

Willkommen

Moodle-Kurs

- Kursname „Algorithmen und Datenstrukturen“
- Selbsteinschreibung mit dem Kennwort
4LG0D4T

Oliver Hofmann

- Professor an der TH Nürnberg seit 2015, zuvor an der TH Würzburg/Schweinfurt seit 2012
- Dipl. Informatiker
- Modulverantwortlicher "Software Engineering"
- Studiengangsleiter "Media Engineering"

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppenarbeit

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 1:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Setze $i \leftarrow 1$
2. Addiere 1 zu i
3. Addiere 1 zu i
4. Addiere 1 zu i

...

1. Setze $i \leftarrow 1$
2. Addiere 1 zu i
3. Wiederhole Schritt 2 solange $i < 10$

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 2:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Gib n ein
2. Wenn $n < 1$ gehe zu Schritt 4
3. Subtrahiere 1 von n
4. Gehe zu Schritt 2
5. Gib n aus

1. Gib n ein
2. Wenn $n < 1$ gehe zu Schritt 5
3. Subtrahiere 1 von n
4. Gehe zu Schritt 2
5. Gib n aus

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 3:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Nimm Brot.
2. Lege eine angemessene Menge Käse darauf.
3. Erwärme das Sandwich ein bisschen.
4. Serviere es schön.

1. Nimm 1 Scheibe Brot
2. Lege zwei Scheiben Käse darauf
3. Erwärme das Sandwich auf 80 Grad
4. Serviere es warm auf einem Teller

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 4:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Gib aus "3 + 5".
2. Gib aus " = 8"

1. Gib a und b ein
2. Addiere a und b und gib das Ergebnis aus

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Ein Algorithmus ist

- eine **endliche Folge** von Regeln,
- nach denen sich nach **endlich vielen**,
- **eindeutig festgelegten** Schritten
- die Lösung einer **Klasse von Problemen** ergibt.

Finitheit

Wir können den Algorithmus mit endlichen Ressourcen beschreiben

Terminiertheit

Der Algorithmus endet nach endlicher Zeit

Determiniertheit

Es gibt keinen Interpretationsspielraum

Allgemeingültigkeit

Lösung wird für unterschiedliche Eingaben errechnet

Was ist eine Datenstruktur?

Eine Datenstruktur ist

- Die Art und Weise, die die von einem Algorithmus bearbeiteten Daten organisiert und vorgehalten werden.
- Algorithmen und Datenstrukturen bedingen sich gegenseitig, d.h. für die Durchführung eines Algorithmus ist es i.d.R. notwendig, dass die Daten in einer bestimmten Datenstruktur vorliegen.
- Manchmal ist das Erzeugen der Datenstruktur aufwändiger als die anschließende Durchführung des Algorithmus.

Beispiel

30 Zahlen $\{a_1 \dots a_{30}\}$ zwischen 1 und 250 sollen der Größe nach sortiert werden beginnend bei der größten Zahl.

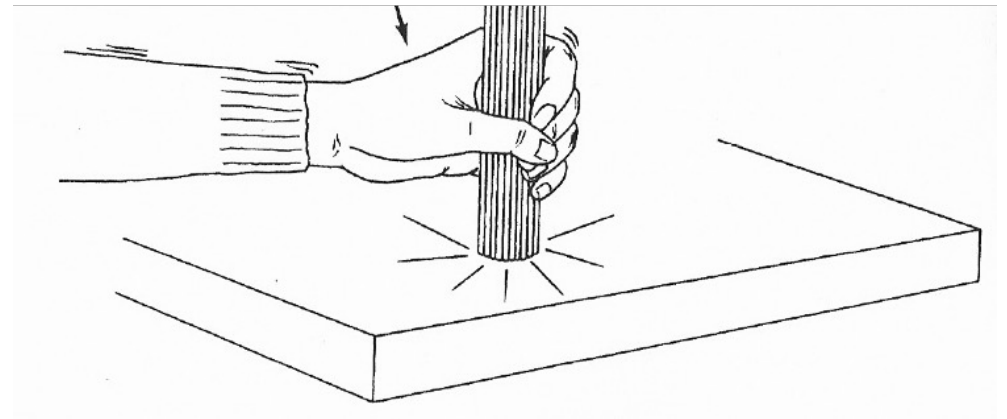
Vorbereiten der Datenstruktur

Für jede Zahl a_i wird eine Spagetti-Nudel auf die Länge $\frac{a_i}{10}$ cm gekürzt.

Durchführen des Algorithmus

Das Bündel Spagetti wird auf den Tisch geschlagen und sukzessive immer die längste Nudel herausgezogen und ihre Länge notiert.

Die Vorbereitung ist hier sicher aufwändiger als die Durchführung!



Ziele der Vorlesung

- Wir lernen für eine Reihe von Problemklassen unterschiedliche Algorithmen mit ihren Datenstrukturen kennen und vergleichen sie.
- Einige der Algorithmen werden wir in den Praktika implementieren, um die jeweiligen Herausforderungen besser zu verstehen.
- Am Ende sollten wir in der Lage sein, für neue und unbekannte Problemklassen brauchbare Algorithmen entwerfen zu können.

Disclaimer: Viele der von uns betrachteten Algorithmen sind bereits in Klassenbibliotheken der unterschiedlichen Programmiersprache enthalten. Das ändert aber nichts daran, dass man sie verstanden haben sollte, um die Lösungsansätze auf neue Problemstellungen anwenden zu können!

Wie stellen wir einen Algorithmus dar?

Wie stellen wir einen Algorithmus dar?

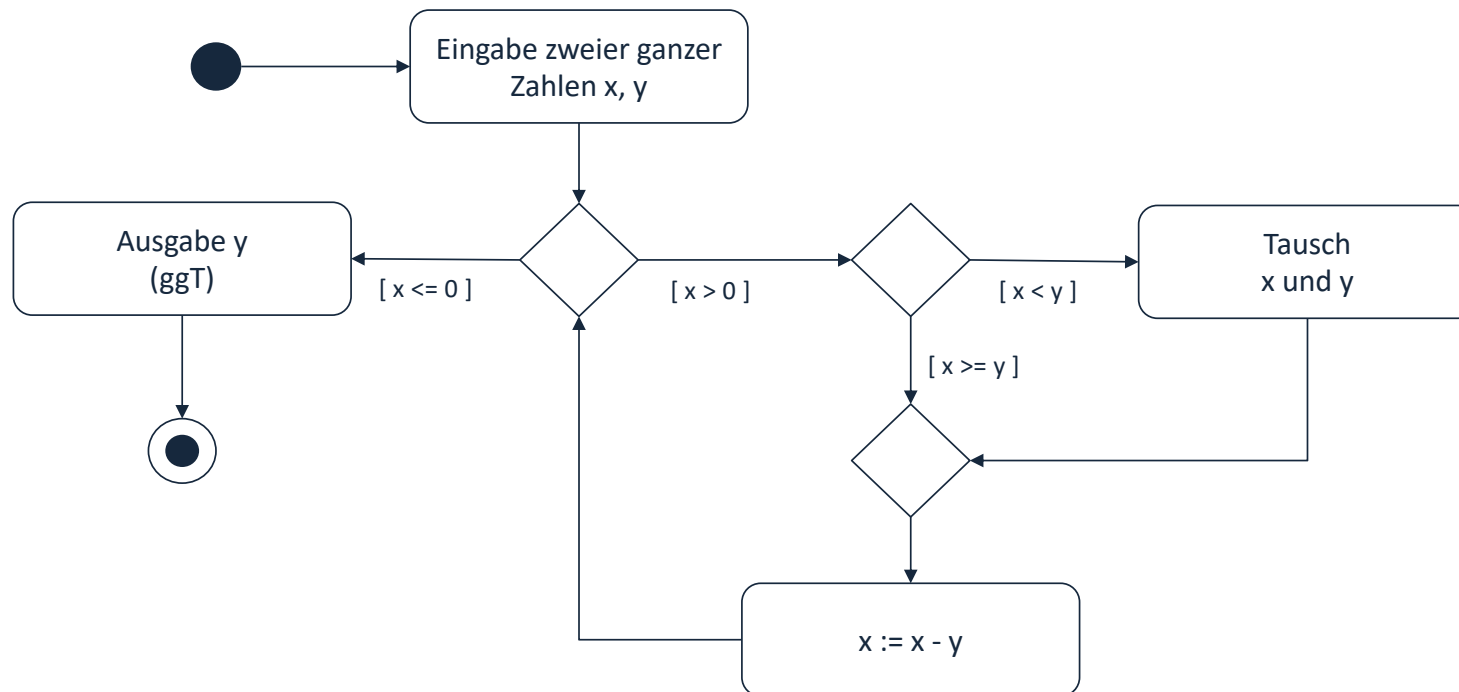
Informelle Darstellung eines Algorithmus

1. Lies zwei ganze Zahlen in die Variablen `,x'` und `,y'` ein!
2. Solange der Wert von `,x'` größer als 0 ist, wiederhole die Schritte 2.1 und 2.2
 - 2.1 Wenn der Wert von `,x'` kleiner als `,y'`, dann vertausche die Inhalte von `,x'` und `,y'`!
 - 2.2 Weise `,x'` die Differenz von `,x'` minus `,y'` zu!
3. Gib den Wert von `,y'` als ggT aus!

Es handelt sich hierbei um den Euklid-Algorithmus zur Berechnung des größten gemeinsamen Teilers (ggT) zweier ganzer Zahlen.

Wie stellen wir einen Algorithmus dar?

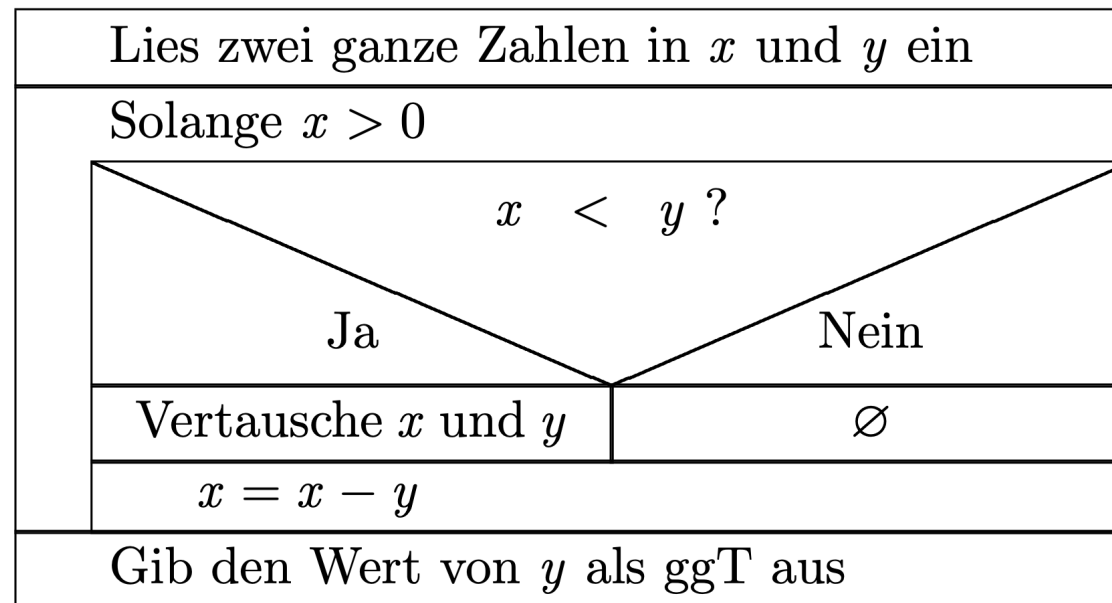
Darstellung als UML Aktivitätsdiagramm



Wie stellen wir einen Algorithmus dar?

Darstellung als Struktogramm von Nassi und Shneiderman (DIN 66261)

Euklid-Algorithmus



Wie stellen wir einen Algorithmus dar?

Darstellung als Pseudocode

Euklid-Algorithmus

Ensure: $x, y \in \mathbb{N}$

```
1: while  $x > 0$  do  
2:   if  $x < y$  then  
3:      $temp \leftarrow x$   
4:      $x \leftarrow y$   
5:      $y \leftarrow temp$   
6:   end if  
7:    $x \leftarrow x - y$   
8: end while  
9: return  $y$ 
```

▷ y enthält den gesuchten ggT

- Ein Pseudocode ist eine semi-formale Sprache, mit der man die Ablaufstrukturen eines Programms grob durch Texte und fest vorgegebene Schlüsselwörter beschreiben kann.
- Die Schlüsselwörter lehnen sich in der Regel an die einer höheren Programmiersprache an.

Implementieren eines Algorithmus

Definition

Das Formulieren eines Algorithmus in einer maschinenlesbaren Form, so dass der Algorithmus von einem Rechner ausgeführt werden kann, nennt man Implementieren des Algorithmus.

Der entstehende Programmcode ist ebenfalls eine Darstellung des Algorithmus, jedoch enthält er i.d.R. weitergehende Implementierungsdetails, die nicht Kernbestandteil des Algorithmus sind.

Implementieren eines Algorithmus

Beispiel-Implementierung in Python

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$	
Ja	Nein
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	



```
def euklid_ggt(x, y):
    while x > 0:
        if x < y:
            (x, y) = (y, x)
        x -= y
    return y
```

Implementierungsdetail:
Variablentausch mit Hilfe von Tupeln

Eigenschaften von Algorithmen

Eigenschaften von Algorithmen

In dieser Veranstaltung interessieren uns vorrangig zwei Eigenschaften:

Korrektheit

Ein Algorithmus ist korrekt, wenn er die Aufgabe richtig löst.

Effizienz

Ein Algorithmus ist effizient, wenn er vergleichsweise wenige Schritte/Operationen und wenig Speicher benötigt.



Diese Eigenschaften sind nicht abhängig von der Implementierung!
Ein Algorithmus wird nicht korrekter oder effizienter durch die Nutzung einer anderen Programmiersprache!

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Nein	
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Korrektheitsüberlegung

In jedem Schleifendurchlauf wird x und ggf. auch y verändert. Durch die Subtraktion der kleineren von der größeren Zahl werden die Werte sukzessive kleiner.

Die Schleife bricht ab, sobald x kleiner oder gleich 0 wird.

→ Der Algorithmus **terminiert**.

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Korrektheitsüberlegung

Seien x_t bzw. y_t die Werte vor dem t . Schleifendurchlauf und x_{t+1} bzw. y_{t+1} die Werte nach dem t . Schleifendurchlauf.

Schleifeninvariante

$ggT(x_t, y_t) = ggT(x_{t+1}, y_{t+1})$, d.h. der ggT wird durch die Schleife nicht verändert.

Beweis

Sei $g = ggT(x_t, y_t)$, dann gilt

$$x_t = g \cdot a$$

$$y_t = g \cdot b$$

O.B.d.A.: Sei $x_t > y_t$, dann

$$x_{t+1} = x_t - y_t$$

$$= g \cdot a - g \cdot b$$

$$= g \cdot (a - b)$$

$$y_{t+1} = y_t = g \cdot b$$

→ g ist weiterhin ein Teiler von x_{t+1} und y_{t+1}

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Korrektheitsüberlegung

Seien x_t bzw. y_t die Werte vor dem t . Schleifendurchlauf und x_{t+1} bzw. y_{t+1} die Werte nach dem t . Schleifendurchlauf.

Schleifeninvariante

$ggT(x_t, y_t) = ggT(x_{t+1}, y_{t+1})$, d.h. der ggT wird durch die Schleife nicht verändert.

Beweis (Fortsetzung)

Es kann keinen neuen Teiler $\hat{g}$ geben, der x_{t+1} und y_{t+1} teilt und größer ist als g (und somit ein neuer ggT), denn sonst:

$$\begin{aligned}y_{t+1} &= \hat{g} \cdot \hat{b} = y_t \\x_{t+1} &= \hat{g} \cdot \hat{a} = x_t - y_t = x_t - \hat{g} \cdot \hat{b} \\x_t &= \hat{g} \cdot \hat{a} + \hat{g} \cdot \hat{b} = \\&= \hat{g} \cdot (\hat{a} + \hat{b})\end{aligned}$$

D.h. $\hat{g}$ wäre bereits ein Teiler von x_t und y_t gewesen und damit g nicht der ggT $\rightarrow$ Widerspruch!

q.e.d

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Nein	
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Korrektheitsüberlegung

- Der Algorithmus terminiert.
- Der ggT wird durch die Schleife nicht verändert.
- Beim letzten Schleifendurchlauf galt

$$x = y$$

→ Der ggT zweier gleicher Zahlen ist der Wert dieser Zahlen

q.e.d.

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus effizient?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Nein	
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Um die Effizienz von Algorithmen vergleichen zu können, benötigen wir ein **abstraktes Maschinenmodell**, anhand dem der nötige Speicher und die Anzahl der ausgeführten Befehle bestimmt werden können.

Die Nutzung eines realen Rechners und die Messung einer Implementierung sind nicht zielführend, weil sich die Aussagen bei Weiterentwicklungen der Architektur oder der Programmiersprache ändern könnten.

Abstraktes Maschinenmodell

RAM (Random Access Machine)

Wir verwenden für die Effizienzbetrachtung eine RAM (Random Access Machine) als abstraktes Maschinenmodell.

- Sie besitzt eine unendliche Anzahl von Speicherzellen
- Jede Speicherzelle kann einen Integer unbegrenzter Größe speichern
- Der Befehlssatz der RAM umfasst lesende und schreibende Zugriffe auf die Speicherzellen, Vergleiche von Speicherinhalten und Sprünge
- Jeder dieser Befehle benötigt eine gewisse Laufzeit
- Die Laufzeitkomplexität eines Algorithmus ergibt sich aus der Anzahl der ausgeführten Befehle
- Die Speicherkomplexität ergibt sich aus der Anzahl der benutzten Speicherzellen

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus effizient?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Vertausche x und y $x = x - y$ Nein $\emptyset$	
Gib den Wert von y als ggT aus	

Effizienzüberlegung zum Speicherbedarf (Speicherkomplexität)

Es werden nur Speicherzellen für die beiden Eingabewerte benötigt, die dann im weiteren Verlauf wiederverwendet werden.

→ Mit weniger Speicher geht es wohl nicht, daher effizient.

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus effizient?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja $x < y ?$ Nein	
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Effizienzüberlegung zur Laufzeit (Laufzeitkomplexität)

Im schlimmsten Fall ist eine der beiden Zahlen 1. Dann wird bei jedem Schleifendurchlauf die größere Zahl ermittelt (1 Vergleich) um 1 dekrementiert (1 Subtraktion).

Es ergibt sich also im Worst Case:

$\max(x, y)$ Vergleiche und $\max(x, y)$ Subtraktionen

Ob dies effizient ist, kann man erst beurteilen, wenn man alternative Algorithmen in einer ähnlichen Form untersucht hat.

Was passiert im Praktikum?

Python

Hinweise zur Nutzung

- Verfügbar für Linux, Mac OS und Windows
- Wir verwenden die Version 3.12 oder neuer
- Viele Algorithmen und Datenstrukturen der Vorlesung sind in der Python-Bibliothek bereits enthalten. Diese verwenden wir aber nicht für unser Praktikum! Auch einfache Listen sind verboten! Stattdessen sind alle unsere Werte Instanzen der Klasse `AlgoDatValue` und Arrays nutzen die Klasse `AlgoDatArray`.

Python

Hinweise zur Nutzung

```
def euklid_ggt(x, y):  
    while x > 0:  
        if x < y:  
            (x, y) = (y, x)  
        x -= y  
    return y
```

```
if __name__ == "__main__":  
    print(euklid_ggt(AlgoDatValue(12), AlgoDatValue(8)))  
  
    AlgoDatValue.summary()
```

Warum?

Wir machen uns zwar das Leben damit zunächst etwas schwerer, können aber die Effizienz besser beurteilen, weil keine „Magie“ in den Bibliotheken stattfindet.

Python

Hinweise zur Nutzung

```
def euklid_ggt(x, y):  
    while x > 0:  
        if x < y:  
            (x, y) = (y, x)  
        x -= y  
    return y  
  
if __name__ == "__main__":  
    print(euklid_ggt(AlgoDatValue(12), AlgoDatValue(8)))  
  
    AlgoDatValue.summary()
```

Ausgabe:

```
4  
Memory: 2  
Read: 18  
Write: 5  
Compare: 7  
Add: 0  
Sub: 3
```

Ziele der heutigen Lerneinheit

Sie verstehen...

- warum verschiedene Algorithmen für dasselbe Problem unterschiedlich effizient sein können
- was die Landau-Symbole O , Ω und Θ aussagen und wie sie sich voneinander unterscheiden
- wie aus der Struktur eines rekursiven Algorithmus eine Rekurrenzgleichung entsteht

Sie können...

- die Anzahl dominanter Operationen eines Algorithmus in Abhängigkeit von n bestimmen
- einen Algorithmus einer Komplexitätsklasse zuordnen und dies begründen
- das Mastertheorem anwenden

Effizienz von Algorithmen

Effizienz von Algorithmen

Sind alle Algorithmen für eine Problemklasse gleich effizient?



Wir wissen bereits: Algorithmen auch für eine Problemklasse können sich unterscheiden hinsichtlich

- der Anzahl der ausgeführten Befehle (Laufzeitkomplexität)
- der Anzahl der benutzten Speicherzellen (Speicherkomplexität)

Maximale Abschnittssumme

Beispielaufgabenstellung

Es soll die maximale Abschnittssumme für eine Abfolge von n ganzen Zahlen gefunden werden. Die Zahlen liegen in einem Feld Z mit den Indizes $0 \dots n - 1$ im Speicher vor.

Hat man z.B. die Zahlenfolge

-59, 52, 46, 14, -50, 58, -87, -77, 34, 15

liefert folgende Teil-Zahlenfolge die maximale Abschnittssumme:

$$\mathbf{52 + 46 + 14 + -50 + 58 = 120}$$

Es ist offensichtlich, dass die Anzahl der Befehle zur Ermittlung der maximalen Abschnittssumme mit der Länge n der Zahlenfolge zunimmt.

Maximale Abschnittssumme

Erster Lösungsansatz

Ein möglicher Lösungsansatz ist, alle Teilfolgen zu betrachten, deren Summen zu berechnen und das Maximum auszuwählen.

Betrachtet man die Anzahl der Additionen in Zeile 7, so erkennt man, dass deren Anzahl aufgrund der drei verschachtelten Schleifen in etwa proportional zu n^3 zunimmt.

Algorithmen mit einem solchen Zeitverhalten nennt man **kubische Algorithmen**.

MAXFOLGE1(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2:  $max \leftarrow -\infty$ 
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:   for  $j \leftarrow i$  to  $n - 1$  do
5:      $sum \leftarrow 0$ 
6:     for  $k \leftarrow i$  to  $j$  do
7:        $sum \leftarrow sum + Z[k]$ 
8:     end for
9:     if  $sum > max$  then
10:       $max \leftarrow sum$ 
11:       $links \leftarrow i$ 
12:       $rechts \leftarrow j$ 
13:    end if
14:  end for
15: end for
16: return ( $max, links, rechts$ )
```

Maximale Abschnittssumme

Zweiter Lösungsansatz

Eine Verbesserung ist es, nicht mehr jede Teilfolge explizit zu betrachten, sondern von jedem möglichen Startpunkt beginnend aufzusummieren.

Betrachtet man nun die Anzahl der Additionen in Zeile 6, so nimmt deren Anzahl nur noch in etwa proportional zu n^2 zu.

Algorithmen mit einem solchen Zeitverhalten nennt man **quadratische Algorithmen**.

MAXFOLGE2(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2:  $max \leftarrow -\infty$ 
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:    $sum \leftarrow 0$ 
5:   for  $j \leftarrow i$  to  $n - 1$  do
6:      $sum \leftarrow sum + Z[j]$ 
7:     if  $sum > max$  then
8:        $max \leftarrow sum$ 
9:        $links \leftarrow i$ 
10:       $rechts \leftarrow j$ 
11:    end if
12:  end for
13: end for
14: return ( $max, links, rechts$ )
```

Maximale Abschnittssumme

Dritter Lösungsansatz „Teile-und-Herrsche“

Folgende Überlegung bringt uns zu einer noch besseren Zeitkomplexität:

Man teilt das Feld $Z[l \dots r]$ in der Mitte m , so dass zwei Teilabschnitte $Z[l \dots m]$ und $Z[m + 1 \dots r]$ entstehen. Der gesuchte Abschnitt mit der maximalen Summe kann sich nun

1. vollständig im Abschnitt $Z[l \dots m]$ oder
2. vollständig im Abschnitt $Z[m + 1 \dots r]$ befinden oder
3. sich über diese beiden Teilabschnitte erstrecken.

Maximale Abschnittssumme

FINDE-ZWISCHEN(Z, l, m, r)

```
1:  $linksMax \leftarrow -\infty$ 
2:  $sum \leftarrow 0$ 
3: for  $i \leftarrow m$  downto  $l$  do
4:    $sum \leftarrow sum + Z[i]$ 
5:   if  $sum > linksMax$  then
6:      $linksMax \leftarrow sum$ 
7:      $links \leftarrow i$ 
8:   end if
9: end for
10:  $rechtsMax \leftarrow -\infty$ 
11:  $sum \leftarrow 0$ 
12: for  $i \leftarrow m + 1$  to  $r$  do
13:    $sum \leftarrow sum + Z[i]$ 
14:   if  $sum > rechtsMax$  then
15:      $rechtsMax \leftarrow sum$ 
16:      $rechts \leftarrow i$ 
17:   end if
18: end for
19: return ( $linksMax + rechtsMax, links, rechts$ )
```

Lösung für den 3. Fall

FINDE-ZWISCHEN behandelt 3. Fall, indem von m nach links und von $m + 1$ nach rechts die max. Abschnittssumme gesucht wird.

Es muss aus jedem Teilabschnitt min. ein Element enthalten sein, sonst wäre es nicht der 3. Fall!

Maximale Abschnittssumme

Dritter Lösungsansatz „Teile-und-Herrsche“

MAXFOLGE3(Z, l, r)

```

1: if  $l = r$  then                                ▷ Abbruchbedingung für die Rekursion
2:   return ( $Z[l], l, r$ )
3: end if
4:  $m \leftarrow (l + r) / 2$                             ▷ Ganzzahlige Division
5: ( $linksMax, linksL, linksR$ )  $\leftarrow$  MAXFOLGE3( $Z, l, m$ )    ▷ 1. Fall
6: ( $rechtsMax, rechtsL, rechtsR$ )  $\leftarrow$  MAXFOLGE3( $Z, m + 1, r$ )  ▷ 2. Fall
7: ( $zwiMax, zwiL, zwiR$ )  $\leftarrow$  FINDE-ZWISCHEN( $Z, l, m, r$ )    ▷ 3. Fall
8: if  $linksMax \geq rechtsMax$  and  $linksMax \geq zwiMax$  then
9:   return ( $linksMax, linksL, linksR$ )
10: end if
11: if  $rechtsMax \geq linksMax$  and  $rechtsMax \geq zwiMax$  then
12:   return ( $rechtsMax, rechtsL, rechtsR$ )
13: end if
14: return ( $zwiMax, zwiL, zwiR$ )

```

Es fällt auf, dass nur in der Funktion FINDE-ZWISCHEN Schleifen enthalten sind, in denen in Abhängigkeit der Größe von n Additionen durchgeführt werden – allerdings nicht verschachtelt.

Durch das wiederholte Halbieren des Feldes wird FINDE-ZWISCHEN etwa $\lg n$ - mal

aufgerufen, so dass die Anzahl der Additionen etwa proportional zu $n \lg n$

ist. ($\lg$ ist Logarithmus zur Basis 2)

Maximale Abschnittssumme

Vierter Lösungsansatz

Beim vierten Lösungsansatz wird die Folge nur einmal von links nach rechts durchlaufen und die Werte in *aktSum* aufsummiert.

Der erreichte maximale Summenwert wird in *max* gespeichert.

Wird *aktSum* kleiner als 0, so ist es besser, die Folge neu zu beginnen.

Die Anzahl der Additionen ist jetzt proportional zu *n*.

Solche Algorithmen nennt man **lineare Algorithmen**.

MAXFOLGE4(*Z*)

```
1:  $n \leftarrow \text{length}(Z)$ 
2:  $max \leftarrow -\infty$ 
3:  $aktSum \leftarrow 0$ 
4:  $aktLinks \leftarrow 0$ 
5: for  $i \leftarrow 0$  to  $n - 1$  do
6:    $aktSum \leftarrow aktSum + Z[i]$ 
7:   if  $aktSum > max$  then
8:      $max \leftarrow aktSum$ 
9:      $links \leftarrow aktLinks$ 
10:     $rechts \leftarrow i$ 
11:   end if
12:   if  $aktSum < 0$  then
13:      $aktSum \leftarrow 0$ 
14:      $aktLinks \leftarrow i + 1$ 
15:   end if
16: end for
17: return ( $max, links, rechts$ )
```

Maximale Abschnittssumme

**Was bedeutet das bei einer
Zahlenfolge der Länge
 $n = 10000 = 10^4$?**

Die Anzahl der Additionen bei MAXFOLGE1 beträgt etwa $10^{4^3} = 1$ Billion

Die Anzahl der Additionen bei MAXFOLGE2 beträgt etwa 10^{4^2} (10000 mal schneller als MAXFOLGE1)

Die Anzahl der Additionen bei MAXFOLGE3 beträgt etwa $10^4 \cdot \lg 10^4 \approx 10^4 \cdot 13,3$ (751 mal schneller als MAXFOLGE2 und 7,5 Millionen mal schneller als MAXFOLGE 1)

Die Anzahl der Additionen bei MAXFOLGE4 beträgt etwa 10^4 (100 Millionen mal schneller als MAXFOLGE1)

**Test mit einer Rust-
Implementierung auf einem 3,4
GHz Intel Core i5**

7009 33 5140

MAXFOLGE1 (kubisch): 4738.40s

7009 33 5140

MAXFOLGE2 (quadratisch): 1.32s

7009 33 5140

MAXFOLGE3 (Teile-und-Herrsche): 4.85ms

7009 33 5140

MAXFOLGE4 (Linear): 302.49µs

Landau-Symbole

Landau-Symbole

Hinführung

Bei der Betrachtung der Komplexität versuchen wir, einen Term zu entwickeln, der die Anzahl der auszuführenden Anweisungen bzw. der benutzten Speicherzellen in Abhängigkeit von der Größe des Problems berechnet.

Beispiel

$$t(n) = 1,5n^2 + 0,5n - 1$$

Bei großen n ist nur der führende Term wichtig. Auch ignoriert man konstante Koeffizienten des führenden Terms. M.a.W.:

$$1,5n^2 + 0,5n - 1 \rightarrow \text{quadratische Komplexität}$$

Landau-Symbole

Historie

Das große O (großes Omikron) wurde Ende des 19. Jh von dem deutschen Zahlentheoretiker **Paul Bachmann** als Symbol für "Ordnung von" eingeführt.

Irrtümlicherweise wurde Notation nach dem ebenfalls deutschen Zahlentheoretiker **Edmund Landau** benannt, der diese Anfang des 20. Jh bekannt machte.

Mit den Landau-Symbole lassen sich Algorithmen hinsichtlich ihres (minimalen oder maximalen) Bedarfs an Zeit und/oder Speicher bezüglich eines bestimmten Maschinenmodells analysieren.

Man ermittelt dabei die Schranken, in denen sich Laufzeit und/oder Speicherplatzbedarf hält, wenn man die Problemgröße n vergrößert.

Landau-Symbole

Bedeutung

Notation	Bedeutung	Sprechweise
$f \in O(g)$	f wächst höchstens so schnell wie g	„groß <i>O</i> “
$f \in \Omega(g)$	f wächst mindestens so schnell wie g	„groß <i>Omega</i> “
$f \in \Theta(g)$	f wächst genauso schnell wie g	„groß <i>Theta</i> “

Mit den Landau-Symbolen beschreibt man eine Menge von Funktionen, deren Wachstumsraten vergleichbare Wachstumsraten haben wie die in Klammern angegebene Funktion.

Landau-Symbole

O-Notation

$O(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c \text{ und } n_0, \text{ so dass für alle } n \geq n_0 :$

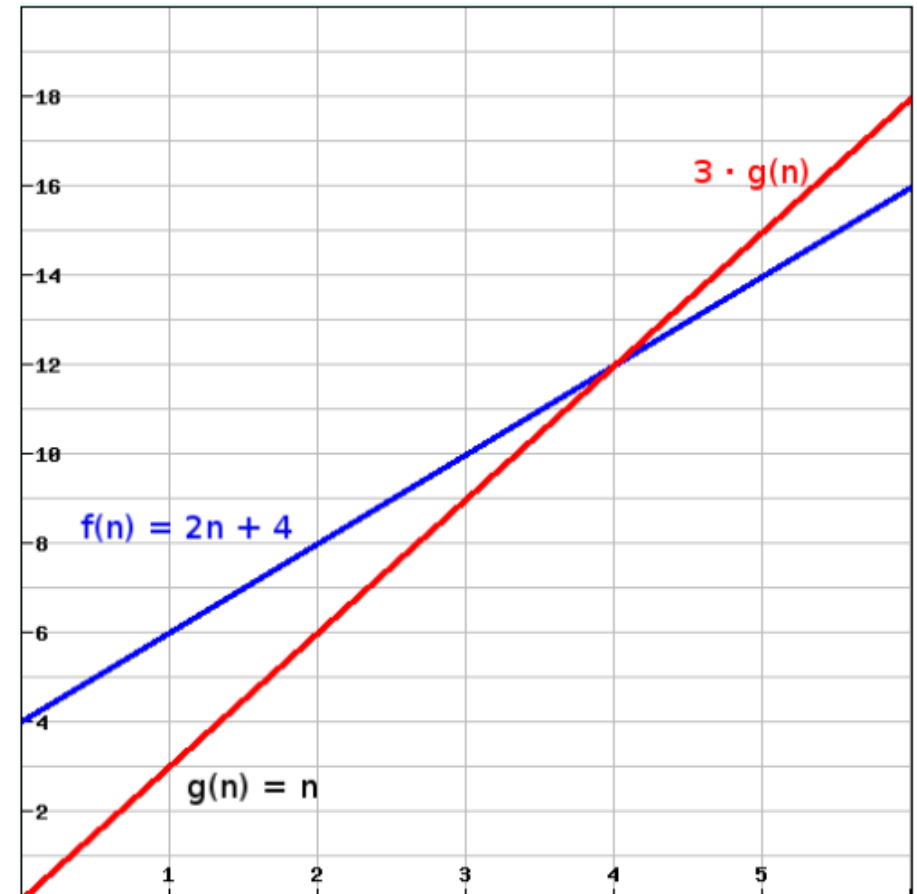
$$0 \leq f(n) \leq c \cdot g(n) \}$$

Beispiel

Gegeben $f(n) = 2n + 4$ und $g(n) = n$. Wir finden $c = 3$ und $n_0 = 4$, so dass

$$f(n) \leq 3 \cdot g(n) \text{ für alle } n \geq n_0$$

$$\rightarrow f(n) \in O(n)$$



Landau-Symbole

Θ -Notation

$$\Theta(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c_1, c_2 \text{ und } n_0, \text{ so dass für alle } n \geq n_0: \\ 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Man bezeichnet hier $g(n)$ auch als asymptotisch scharfe Schranke von $f(n)$, da f genauso schnell wie g wächst. Dies bedeutet, dass man die Wachstumsrate des Ressourcenbedarfs (bis auf konstante Faktoren) genau bestimmen kann.

Landau-Symbole

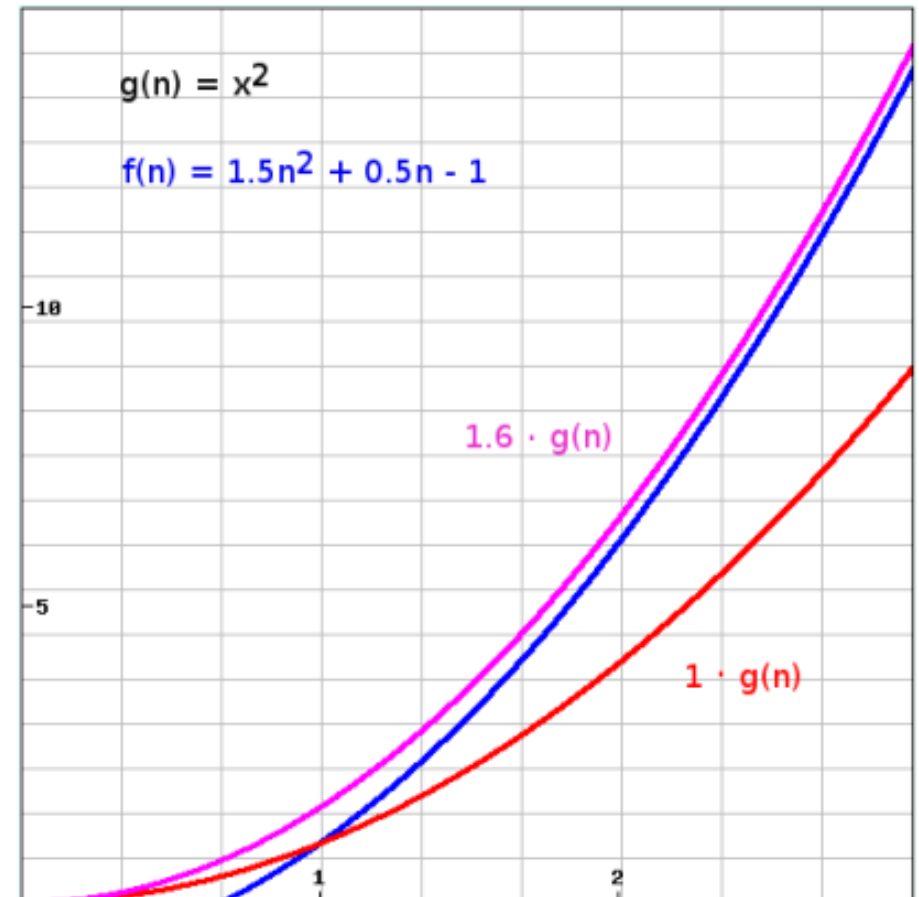
Θ -Notation

$\Theta(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c_1, c_2 \text{ und } n_0, \text{ so dass f\"ur alle } n \geq n_0 :$

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$$

Beispiel

Gegeben $f(n) = 1,5n^2 + 0,5n - 1$ und $g(n) = n^2$. Wir finden $c_1 = 1$, $c_2 = 1,6$ und $n_0 = 1$



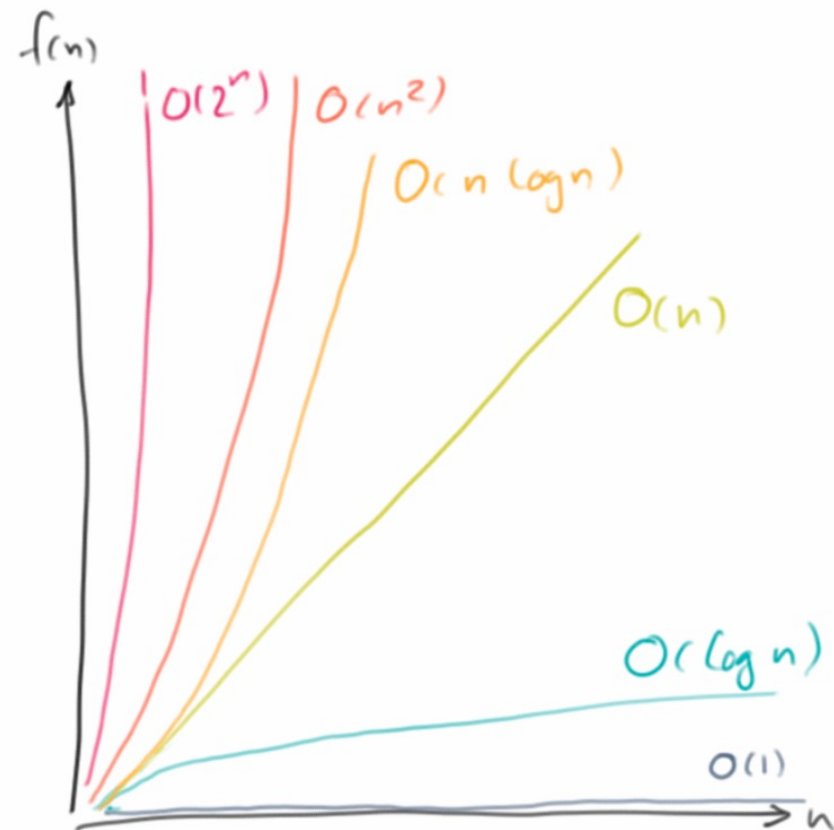
Landau-Symbole

Komplexitätsklasse

Eine **Komplexitätsklasse** fasst alle Algorithmen zusammen, deren Ressourcenbedarf dieselbe asymptotische Wachstumsrate hat.

Beispiel:

"MAXFOLGE4 hat lineare Komplexität"
bedeutet formal:
die Laufzeit von MAXFOLGE4 $\in \Theta(n)$.



Landau-Symbole

Ω -Notation

$$\Omega(g(n)) = \{ f(n) : \text{es existieren positive Konstanten } c \text{ und } n_0, \text{ so dass für alle } n \geq n_0: \\ 0 \leq c \cdot g(n) \leq f(n) \}$$

Die Ω -Notation gibt eine untere Schranke für die Wachstumsrate an.

$f(n) \in \Omega(g(n))$ bedeutet, dass der Ressourcenbedarf für alle $n \geq n_0$ mindestens proportional zu $g(n)$ wächst – unabhängig von der konkreten Eingabe.

In der Praxis wird Ω häufig genutzt, um die Best-Case-Laufzeit zu beschreiben.

Ω hm

Landau-Symbole

Beziehungen zwischen den Notationen

$$\Theta(f) \subseteq O(f)$$

$$\Theta(f) \subseteq \Omega(f)$$

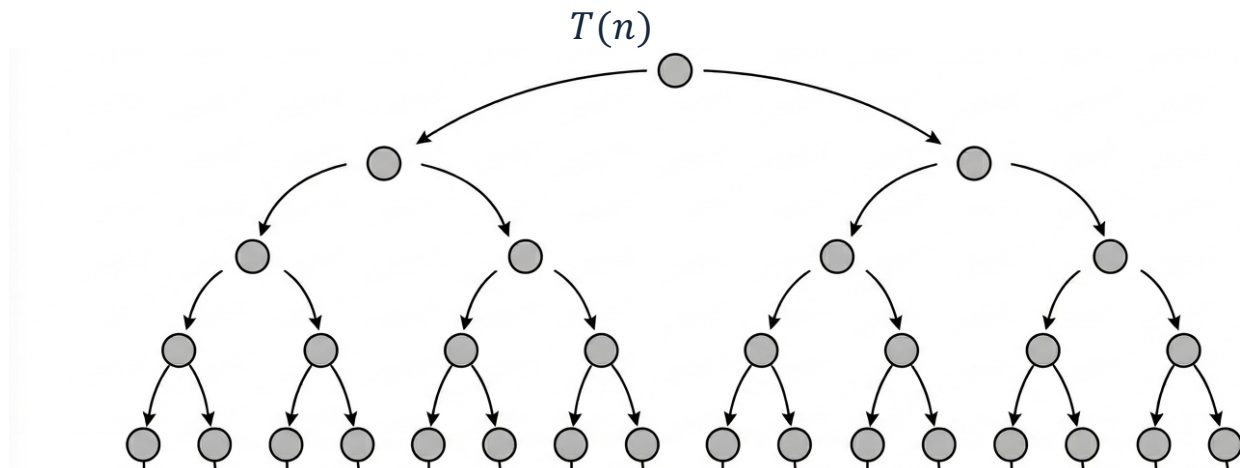
$$\Theta(f) = O(f) \cap \Omega(f)$$

Mastertheorem

Mastertheorem

Rekurrenz

Relativ einfach lässt sich meist die **Rekurrenz** (oder auch **Rekurrenzformel**) bestimmen, die den Ressourcenbedarf $T(n)$ auf den Bedarf für kleinere n zurückführt.



Mastertheorem

Rekurrenz bei MAXFOLGE3

Relativ einfach lässt sich meist die **Rekurrenz** (oder auch **Rekurrenzformel**) bestimmen, die den Ressourcenbedarf $T(n)$ auf den Bedarf für kleinere n zurückführt.

```

1: if  $l = r$  then                                ▷ Abbruchbedingung für die Rekursion
2:   return  $(Z[l], l, r)$ 
3: end if

```

Basisfall: $T(1) = 1$

```

5:  $(linksMax, linksL, linksR) \leftarrow \text{MAXFOLGE3}(Z, l, m)$            ▷ 1. Fall
6:  $(rechtsMax, rechtsL, rechtsR) \leftarrow \text{MAXFOLGE3}(Z, m + 1, r)$    ▷ 2. Fall
7:  $(zwiMax, zwiL, zwiR) \leftarrow \text{FINDE-ZWISCHEN}(Z, l, m, r)$        ▷ 3. Fall

```

Rekurrenz: $T(n) = 2 \cdot T(n/2) + n$

Mastertheorem

Allgemeine Rekurrenz

Die allgemeine Form $T(n) = a \cdot T(n/b) + f(n)$ kann zerlegt werden:

Parameter	Bedeutung	bei MAXFOLGE3
a	Anzahl der Teilprobleme	2
b	Verkleinerungsfaktor	2
$f(n)$	Kosten außerhalb der Rekursion	FINDE-ZWISCHEN $\in O(n)$

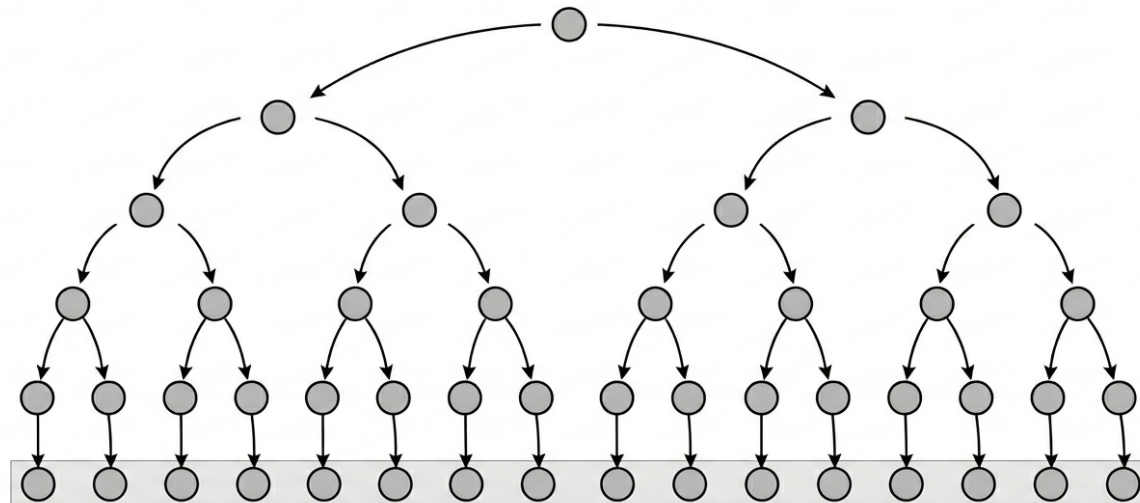
Mastertheorem

Zusammenhänge

Anzahl der Ebenen (Höhe des Baums): $\log_b n$; bei **MAXFOLGE3** ist $b = 2$, daher $\lg n$

Mit jeder Ebene entstehen a Teilprobleme, damit Anzahl der Blätter $a^{\log_b n} = n^{\log_b a}$

Bei **MAXFOLGE3**: $2^{\lg n} = n$



Mastertheorem

Grundidee

Der Aufwand für den rekursiven Algorithmus setzt sich aus zwei Komponenten zusammen:

1. An jedem "Blatt" des Rekursionsbaums fällt die Kosten des Basisfalls an.
2. Auf jeder Ebene des Rekursionsbaums fallen die Kosten außerhalb der Rekursion an (Teilen/Zusammenführen).

Die Gesamtkosten bzw. die Komplexitätsklasse hängt davon ab, welche Komponente dominiert.

Mastertheorem

Fallunterscheidung

1. Fall: Rekursion dominiert

$f(n)$ wächst langsamer als $n^{\log_b a} \rightarrow T(n) \in \Theta(n^{\log_b a})$

2. Fall: Gleichstand

$f(n)$ wächst genauso schnell $\rightarrow T(n) \in \Theta(f(n) \cdot \log_b n)$ (auf jeder Ebene $f(n)$)

3. Fall: Teilen/Zusammenführen dominiert

$f(n)$ wächst schneller $\rightarrow T(n) \in \Theta(f(n))$

Mastertheorem

Checkliste

1. Zuerst a , b und $f(n)$ identifizieren
2. Dann $n^{\log_b a}$ berechnen
3. $f(n)$ mit $n^{\log_b a}$ vergleichen
4. Passenden Fall anwenden

MAXFOLGE3

$$a = 2, b = 2 \text{ und } f(n) = n$$
$$n^{\log_b a} = n$$

Beide gleich $\rightarrow$ Fall 2

$$\Theta(f(n) \cdot \lg n)$$

Bubble-Sort

Sortiervverfahren

Bubble-Sort

BUBBLE-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for
```

Lösungsidee

Das Feld wird von hinten nach vorne durchlaufen und die nebeneinander liegenden Speicherzellen verglichen. Stimmt die Reihenfolge nicht, werden die Inhalte vertauscht.

Mit dem ersten Durchgang wandert das kleinste Element an die erste Stelle. Der zweite Durchgang muss dann nur noch das Feld ab der zweiten Stelle bearbeiten.

Sortiervverfahren

Bubble-Sort

BUBBLE-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for
```

Vor Sortierung

85 72 48 6 22 10 37 76

Schritt 1: 6 85 72 48 10 22 37 76

Sortiervverfahren

Bubble-Sort

BUBBLE-SORT(Z)

```

1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for

```

Vor Sortierung

85 72 48 6 22 10 37 76

Schritt 1:	6	85	72	48	10	22	37	76
Schritt 2:	6	10	85	72	48	22	37	76
Schritt 3:	6	10	22	85	72	48	37	76
Schritt 4:	6	10	22	37	85	72	48	76
Schritt 5:	6	10	22	37	48	85	72	76
Schritt 6:	6	10	22	37	48	72	85	76
Schritt 7:	6	10	22	37	48	72	76	85

Sortiervverfahren

Bubble-Sort

BUBBLE-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 2$  do
3:   for  $j \leftarrow n - 1$  downto  $i + 1$  do
4:     if  $Z[j - 1] > Z[j]$  then
5:        $\text{temp} \leftarrow Z[j]$ 
6:        $Z[j] \leftarrow Z[j - 1]$ 
7:        $Z[j - 1] \leftarrow \text{temp}$ 
8:     end if
9:   end for
10: end for
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Laufzeitkomplexität

Aufwändig sind der Vergleich (Zeile 4) und die Zuweisungen (Zeilen 5-6). Beides findet innerhalb zweier verschachtelter Schleifen statt. Die Anzahl der Vergleiche wächst mit $O(n^2)$. Diese Laufzeit ist unabhängig von der Vorbelegung des Feldes Z .

Sortierv Verfahren

Bubble-Sort (Verbesserung)

BUBBLE-SORT2(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: repeat
3:    $\text{sortiert} \leftarrow \text{true}$ 
4:   for  $i \leftarrow 0$  to  $n - 2$  do
5:     if  $Z[i] > Z[i + 1]$  then
6:        $\text{sortiert} \leftarrow \text{false}$ 
7:        $\text{temp} \leftarrow Z[i]$ 
8:        $Z[i] \leftarrow Z[i + 1]$ 
9:        $Z[i + 1] \leftarrow \text{temp}$ 
10:    end if
11:  end for
12:   $n \leftarrow n - 1$ 
13: until  $\text{sortiert} = \text{true}$  or  $n \leq 1$ 
```

Verbesserte Lösungsidee

Das Feld wird von vorne nach hinten durchlaufen und die nebeneinander liegenden Speicherzellen verglichen. Stimmt die Reihenfolge nicht, werden die Inhalte vertauscht.

Diese Durchläufe werden wiederholt, bis im letzten Durchlauf keine Vertauschung mehr notwendig war.

Sortierv Verfahren

Bubble-Sort (Verbesserung)

BUBBLE-SORT2(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: repeat
3:    $\text{sortiert} \leftarrow \text{true}$ 
4:   for  $i \leftarrow 0$  to  $n - 2$  do
5:     if  $Z[i] > Z[i + 1]$  then
6:        $\text{sortiert} \leftarrow \text{false}$ 
7:        $\text{temp} \leftarrow Z[i]$ 
8:        $Z[i] \leftarrow Z[i + 1]$ 
9:        $Z[i + 1] \leftarrow \text{temp}$ 
10:    end if
11:  end for
12:   $n \leftarrow n - 1$ 
13: until  $\text{sortiert} = \text{true}$  or  $n \leq 1$ 
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Laufzeitkomplexität

Im Sonderfall, dass das Feld Z bereits sortiert ist, bricht der Algorithmus nach nur einem Durchlauf ab: $O(n)$
In allen anderen Fällen bleibt es jedoch bei $O(n^2)$.

Select-Sort

Sortiervverfahren

Select-Sort

SELECT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\text{min} \leftarrow i$ 
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:     if  $Z[j] < Z[\text{min}]$  then
6:        $\text{min} \leftarrow j$ 
7:     end if
8:   end for
9:    $\text{temp} \leftarrow Z[i]$ 
10:   $Z[i] \leftarrow Z[\text{min}]$ 
11:   $Z[\text{min}] \leftarrow \text{temp}$ 
12: end for
```

Lösungsidee

Zunächst wird kleinste Element ermittelt und an 1. Stelle gesetzt. Dann wird das zweitkleinste Element und an 2. Stelle gesetzt usw.

Für jede Position i wird das Feld ab der Position $i + 1$ durchsucht und ggf. das gefundene kleinste Element an die Position i getauscht.

Sortiervverfahren

Select-Sort

SELECT-SORT(Z)

```

1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\text{min} \leftarrow i$ 
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:     if  $Z[j] < Z[\text{min}]$  then
6:        $\text{min} \leftarrow j$ 
7:     end if
8:   end for
9:    $\text{temp} \leftarrow Z[i]$ 
10:   $Z[i] \leftarrow Z[\text{min}]$ 
11:   $Z[\text{min}] \leftarrow \text{temp}$ 
12: end for

```

Vor Sortierung

44 81 90 63 35 43 38 38

Schritt 1:	35	81	90	63	44	43	38	38
Schritt 2:	35	38	90	63	44	43	81	38
Schritt 3:	35	38	38	63	44	43	81	90
Schritt 4:	35	38	38	43	44	63	81	90
Schritt 5:	35	38	38	43	44	63	81	90
Schritt 6:	35	38	38	43	44	63	81	90
Schritt 7:	35	38	38	43	44	63	81	90
Schritt 8:	35	38	38	43	44	63	81	90

Sortierv Verfahren

Select-Sort

SELECT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $\text{min} \leftarrow i$ 
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:     if  $Z[j] < Z[\text{min}]$  then
6:        $\text{min} \leftarrow j$ 
7:     end if
8:   end for
9:    $\text{temp} \leftarrow Z[i]$ 
10:   $Z[i] \leftarrow Z[\text{min}]$ 
11:   $Z[\text{min}] \leftarrow \text{temp}$ 
12: end for
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Laufzeitkomplexität

Zur ersten Bestimmung des Minimums sind $n - 1$ Vergleiche notwendig, bei der zweiten $n - 2$ Vergleiche usw. Somit gilt insgesamt:

$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n \cdot (n - 1)}{2}$ Vergleiche – unabhängig, ob das Feld bereits vorsortiert ist $\rightarrow O(n^2)$.

Insert-Sort

Sortiervverfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Lösungsidee

Man geht wie ein Schafkopf-Spieler vor, der Karten einzeln aufnimmt und jede in das bereits auf der Hand befindliche Blatt an der richtigen Stelle einfügt.

Am Anfang ist (trivialerweise) nur die erste Position (Index 0) sortiert. Beginnend bei $i = 1$ werden alle anderen Elemente in die sortierte Liste eingefügt. Um Platz für $Z[i]$ zu schaffen, müssen ggf. die sortierten Elemente um eine Stelle nach rechts geschoben werden.

Sortiervverfahren

Insert-Sort

INSERT-SORT(Z)

```

1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for

```

Vor Sortierung

44 39 29 58 27 65 1 79 46 10 62 81

Schritt 1:	39	44	29	58	27	65	1	79	46	10	62	81
Schritt 2:	29	39	44	58	27	65	1	79	46	10	62	81
Schritt 3:	29	39	44	58	27	65	1	79	46	10	62	81
Schritt 4:	27	29	39	44	58	65	1	79	46	10	62	81
Schritt 5:	27	29	39	44	58	65	1	79	46	10	62	81
Schritt 6:	1	27	29	39	44	58	65	79	46	10	62	81
Schritt 7:	1	27	29	39	44	58	65	79	46	10	62	81
Schritt 8:	1	27	29	39	44	46	58	65	79	10	62	81
Schritt 9:	1	10	27	29	39	44	46	58	65	79	62	81
Schritt 10:	1	10	27	29	39	44	46	58	62	65	79	81
Schritt 11:	1	10	27	29	39	44	46	58	62	65	79	81

Sortiervverfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Speicherkomplexität

Der Sortieralgorithmus benötigt über das gegebene Feld hinaus nur temporär einzelne Speicherzellen unabhängig von der Größe von n : $O(1)$

Sortierv Verfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Laufzeitkomplexität

Innerhalb der for-Schleife werden im Kopf der while-Schleife Vergleiche durchgeführt.

Best Case:

Die Laufbedingung in Zeile 5 ist nie erfüllt, dann werden nur $O(n)$ Vergleiche durchgeführt. Das ist der Fall, wenn das Feld bereits sortiert ist.

Sortierv Verfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Laufzeitkomplexität

Innerhalb der for-Schleife werden im Kopf der while-Schleife Vergleiche durchgeführt.

Worst Case:

Jedes neu betrachtete Element muss komplett an den Anfang durchgeschoben werden (d.h. das Feld war umgekehrt sortiert. Dann sind es

$$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n \cdot (n - 1)}{2} \text{ Vergleiche}$$

$\rightarrow O(n^2)$.

Sortiervverfahren

Insert-Sort

INSERT-SORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $elem \leftarrow Z[i]$ 
4:    $j \leftarrow i$ 
5:   while  $j > 0$  and  $Z[j - 1] > elem$  do
6:      $Z[j] \leftarrow Z[j - 1]$ 
7:      $j \leftarrow j - 1$ 
8:   end while
9:    $Z[j] \leftarrow elem$ 
10: end for
```

Effizienzüberlegung

Laufzeitkomplexität

Innerhalb der for-Schleife werden im Kopf der while-Schleife Vergleiche durchgeführt.

Average Case:

Im Mittel muss jedes neu betrachtete Element bis zur Mitte der sortierten Anfangssequenz geschoben werden:

$$\frac{n-1}{2} + \frac{n-2}{2} + \dots + \frac{2}{2} + \frac{1}{2} = \frac{n \cdot (n-1)}{4} \text{ Vergleiche}$$

$\rightarrow O(n^2)$.

Ziele der heutigen Lerneinheit

Sie verstehen...

- die grundlegenden Qualitätsmerkmale von Sortialgorithmen wie **Stabilität** und die **In-Place**-Eigenschaft.
- die Bedeutung der verschiedenen Laufzeitszenarien (Best, Worst und Average Case)

Sie können...

- die Funktionsweise und die theoretischen Laufzeitkomplexitäten der elementaren Verfahren **Bubble-Sort**, **Select-Sort** und **Insert-Sort** gegenüberstellen.
- die Sortiertheitsbedingung formal anwenden, um die Korrektheit eines Sortierergebnisses zu prüfen.

Sortiervverfahren

Sortiervverfahren

Problemklasse

Gesucht ist ein effizienter Algorithmus, der Elemente

- für die eine Ordnungsrelation " $\leq$ " besteht und
 - die in einem Array Z mit wahlfreiem Zugriff abgelegt sind
- in eine Reihenfolge bringt, so dass gilt:

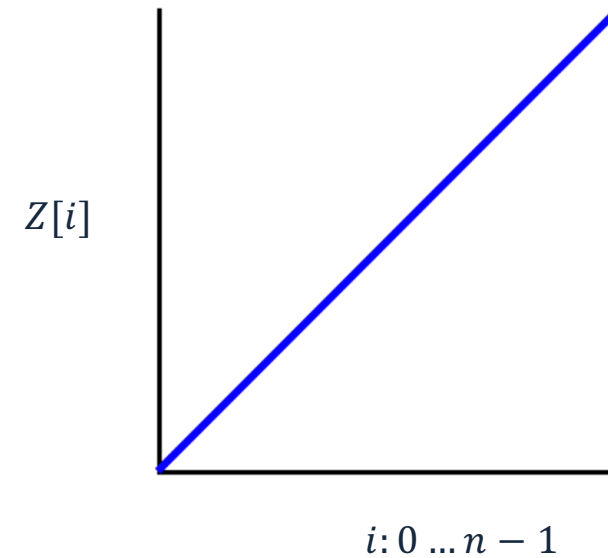
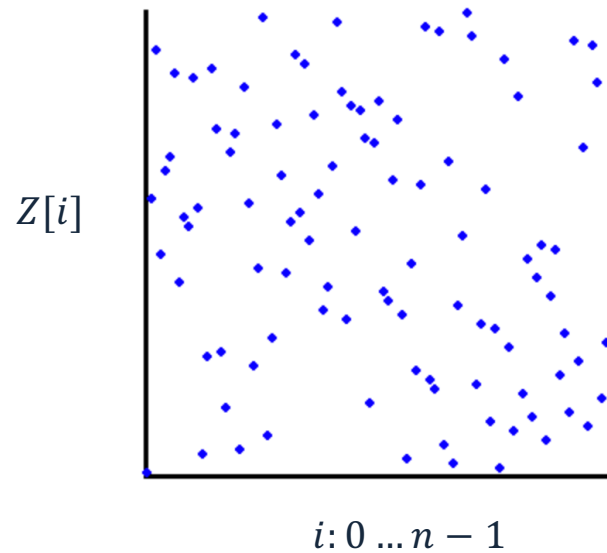
$$\forall i \in \{0, \dots, n - 2\}: Z[i] \leq Z[i + 1]$$

(Sortiertheitsbedingung)

Sortiervverfahren

Visualisierung

Beispiel: Ein Feld der Länge n , in dem die Zahlen $\{0, \dots, n - 1\}$ zunächst zufällig verteilt sind, wird sortiert.



Sortiervverfahren

Korrektheit

Der Algorithmus ist korrekt,

- wenn er terminiert
- wenn Z am Ende eine Permutation des ursprünglichen Arrays ist, für die gilt: $\forall i \in \{0, \dots, n-2\}: Z[i] \leq Z[i+1]$ (**Sortiertheitsbedingung**)

Korrektheitsbeweis

Die Korrektheit kann oft mit einem Induktionsbeweis nachgewiesen werden: Welche **Aussage**

- gilt vor dem ersten Schleifendurchlauf der äußersten Schleife?
- bleibt nach jedem Schleifendurchlauf erhalten (**Schleifeninvariante**)?
- impliziert bei Terminierung die Sortiertheitsbedingung?

Sortiervverfahren

Stabilität

Definition

Ein Sortieralgorithmus gilt als **stabil**, wenn er bei Elementen, die nach der Ordnungsrelation gleich sind, die zuvor vorliegende Reihenfolge der Elemente relativ zueinander beibehält.

Hinweis: **Gleichheit** bei einer Ordnungsrelation ist gegeben, wenn gilt:

$$a \leq b \wedge b \leq a$$

a und b sind im Sinner der Ordnungsrelation nicht unterscheidbar.

Beispiel

Hat man z.B. eine alphabetisch sortierte Liste von Einwohnern einer Stadt, so garantiert ein stabiler Sortieralgorithmus, der diese Liste nach Alter sortiert, dass gleichaltrige Personen danach immer noch alphabetisch geordnet sind.

Vorher (nach Name sortiert):

Anna (20), Bernd (20), Clara (25)

Nach stabiler Sortierung nach Alter:

Anna (20), Bernd (20), Clara (25)

Nach instabiler Sortierung:

Bernd (20), Anna (20), Clara (25) ← Reihenfolge zerstört

Sortiervverfahren

In-Place

Definition

Ein Sortieralgorithmus heißt **in-place**, wenn er nur eine konstante Menge an zusätzlichem Speicher benötigt, unabhängig von der Eingabegröße.

Der zusätzliche Speicherbedarf ist $O(1)$.

Hinweise:

- Das Eingabearray selbst zählt nicht als zusätzlicher Speicher
- Temporäre Variablen (z.B. für einen Swap) sind erlaubt

Gegenbeispiel

$$Z = [4, 2, 3, 1]$$

→ neues Array Y wird erzeugt

$$Y = [1, 2, 3, 4]$$

Dieser Algorithmus benötigt zusätzlichen Speicher $O(n)$.

Sortiervverfahren

Laufzeit

Kostenmaß

Für den Vergleich von Sortiervverfahren wird normalerweise die **Anzahl der Vergleiche** herangezogen.

Hintergrund: Der Aufwand der **Vertauschungen** ist stark abhängig von der Größe der Objekte. Außerdem lassen sich einige formale Überlegungen besser mit den Vergleichen durchführen.

Die einheitliche Währung zur Gegenüberstellung von Sortieralgorithmen ist daher die Anzahl der Vergleiche.

Sind Vertauschungen unwichtig?

Nein!

Insbesondere bei großen Objekten können sie entscheidend sein. Sie sind Gegenstand oft Gegenstand einer weiteren, genaueren Laufzeitanalyse.

Sortiervverfahren

Best Case, Worst Case, Average Case

Warum?

Die Laufzeit eines Algorithmus ist nicht immer gleich, sondern hängt davon ab, wie die Eingabedaten aussehen.

Der Best Case beschreibt die **Eingabe, bei der der Algorithmus am schnellsten arbeitet.**

Der Worst Case ist die **Eingabe, bei der der Algorithmus am meisten Arbeit hat.**

Der Average Case beschreibt das **typische Verhalten bei zufälligen Eingaben.**

Achtung:

Die Case-Betrachtungen fokussieren immer auf **eine bestimmte Eingabe.**

Aussage Best Case: Wie wenig Arbeit ist unter optimalen Bedingungen möglich?

Die O , Θ , Ω -Darstellungen beziehen sich immer auf **alle möglichen Eingaben.**

Aussage untere Schranke Ω : Wie wenig Arbeit ist grundsätzlich erzwingbar, unabhängig von der Eingabe?

Sortiervverfahren

Gruppenarbeit

Sie werden sich jetzt in drei Gruppen mit unterschiedlichen elementaren Sortiervverfahren beschäftigen:

- Bubble-Sort
- Select-Sort
- Insert-Sort

Dabei sollen Sie schrittweise:

1. Die Idee des Algorithmus verstehen
2. Die Korrektheit über eine Schleifeninvariante zeigen
3. Die Laufzeit des Algorithmus analysieren
4. Die Eigenschaften Stabilität und In-Place ermitteln
5. Eine Präsentation Ihrer Erkenntnisse vorbereiten



Sortiervverfahren

Präsentationen



Insert-Sort



Select-Sort



Bubble-Sort

Sortiervverfahren

Laufzeit elementarer Sortiervverfahren

Sortiervverfahren	Worst Case	Average Case	Best Case	Stabil?
Insert-Sort	$O(n^2)$	$O(n^2)$	$O(n)$	ja
Select-Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	nein
Bubble-Sort	$O(n^2)$	$O(n^2)$	$O(n)$	ja

Ziele der heutigen Lerneinheit

Sie verstehen...

- die Grundprinzipien von
 - **Quicksort**,
 - **Heapsort** und
 - **Counting Sort**
- Die untere Schranke für das Sortierproblem bei Vergleichs-sortierv Verfahren

Sie können...

- die drei Sortierv Verfahren anwenden und nachvollziehen
- für eine gegebene Problemstellung ein geeignetes Sortierv Verfahren auswählen und begründen

Quicksort

Sortierv Verfahren

Quicksort

QUICKSORT(Z, l, r)

```
1: if  $l < r$  then  
2:    $q \leftarrow \text{PARTITION}(Z, l, r)$   
3:   QUICKSORT( $Z, l, q - 1$ )  
4:   QUICKSORT( $Z, q + 1, r$ )  
5: end if
```

Lösungsidee

Quicksort arbeitet nach dem Prinzip „Teile und Herrsche“: Er zerlegt die zu sortierende Datenmenge in zwei Teile und sortiert dann diese beiden Teile unabhängig voneinander.

Bei der Partitionierung muss allerdings sichergestellt werden, dass in der einen Teilmenge "kleinere" Elemente und in der anderen Teilmenge "größere" Elemente enthalten sind, um unabhängig sortieren zu können.

Dies geschieht durch (zufällige) Auswahl eines Pivot-Elements, das bei der Partitionierung als Vergleichswert für kleinere und größere Elemente herangezogen wird.

Sortiervverfahren

Quicksort – Partitionierung

Rollenspiel:

Stellen Sie sich "unsortiert" in einem Array auf. Eine geeignete Zahlenfolge ist:

4, 2, 7, 1, 6, 3, 8, 5

PARTITION(Z, l, r)

```
1:  $pivot \leftarrow Z[r]$ 
2:  $i \leftarrow l$ 
3:  $j \leftarrow r - 1$ 
4: while  $i < j$  do
5:   while  $Z[i] < pivot$  do
6:      $i \leftarrow i + 1$ 
7:   end while
8:   while  $j > l$  and  $Z[j] \geq pivot$  do
9:      $j \leftarrow j - 1$ 
10:  end while
11:  if  $i < j$  then
12:    SWAP( $Z[i], Z[j]$ )
13:     $i \leftarrow i + 1$ 
14:     $j \leftarrow j - 1$ 
15:  end if
16: end while
17: if  $i = j$  and  $Z[i] < pivot$  then
18:    $i \leftarrow i + 1$ 
19: end if
20: if  $Z[i] \neq pivot$  then
21:   SWAP( $Z[i], Z[r]$ )
22: end if
23: return  $i$ 
```

Sortiervverfahren

Quicksort – Partitionierung (Alternative)

Rollenspiel:

Stellen Sie sich "unsortiert" in einem Array auf. Eine geeignete Zahlenfolge ist:

4, 2, 7, 1, 6, 3, 8, 5

PARTITION2(Z, l, r)

```
1:  $pivot \leftarrow Z[r]$ 
2:  $i \leftarrow l - 1$ 
3: for  $j \leftarrow l$  to  $r - 1$  do
4:   if  $Z[j] \leq pivot$  then
5:      $i \leftarrow i + 1$ 
6:     SWAP( $Z[i], Z[j]$ )
7:   end if
8: end for
9: SWAP( $Z[i + 1], Z[r]$ )
10: return  $i + 1$ 
```

Sortierv Verfahren

Quicksort - Laufzeit

QUICKSORT(Z, l, r)

```
1: if  $l < r$  then  
2:    $q \leftarrow \text{PARTITION}(Z, l, r)$   
3:   QUICKSORT( $Z, l, q - 1$ )  
4:   QUICKSORT( $Z, q + 1, r$ )  
5: end if
```

Effizienzüberlegung

Grundsätzlich: Es handelt sich um ein Divide-et-Impera-Verfahren, d.h.

- die Anzahl der Aufteilungen und
 - die Komplexität zur Ermittlung der Aufteilungen
- ist miteinander zu multiplizieren.

Die Anzahl der Aufteilungen hängt davon ab, wie gut das jeweils gewählte Pivot-Element die Menge partitioniert.

Sortiervverfahren

Quicksort - Laufzeit

Effizienzüberlegung Partitionierung

Auch wenn auf den ersten Blick verschachtelte Schleifen vorhanden sind, wird das gesamte Array nur einmal durchlaufen:

- i läuft von unten nach oben
- j läuft von oben nach unten

bis sich die beiden Indizes treffen. M.a.W.: die Additionen/Subtraktionen/Vergleiche werden in der Größenordnung n ausgeführt. Die anschließenden Verschiebungen fallen nicht ins Gewicht:

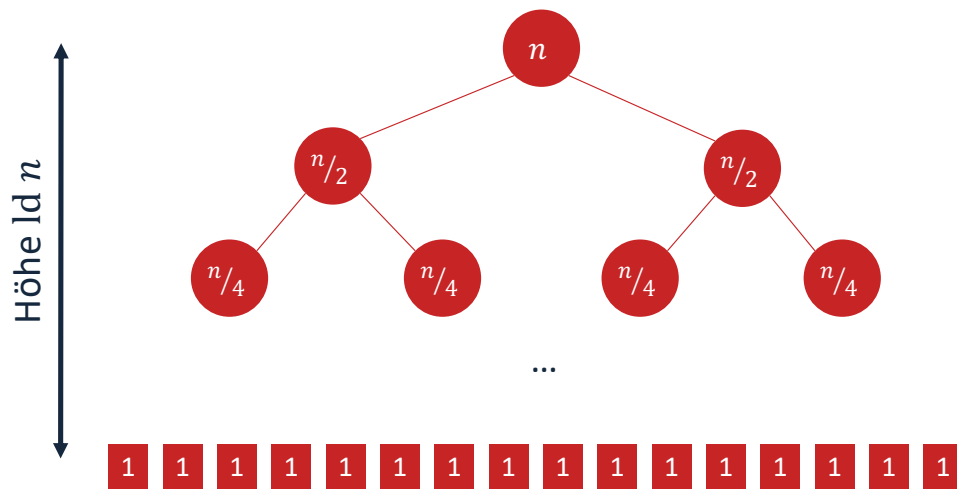
$$O(n)$$

PARTITION(Z, l, r)

```
1:  $pivot \leftarrow Z[r]$ 
2:  $i \leftarrow l$ 
3:  $j \leftarrow r - 1$ 
4: while  $i < j$  do
5:   while  $Z[i] < pivot$  do
6:      $i \leftarrow i + 1$ 
7:   end while
8:   while  $j > l$  and  $Z[j] \geq pivot$  do
9:      $j \leftarrow j - 1$ 
10:  end while
11:  if  $i < j$  then
12:    SWAP( $Z[i], Z[j]$ )
13:     $i \leftarrow i + 1$ 
14:     $j \leftarrow j - 1$ 
15:  end if
16: end while
17: if  $i = j$  and  $Z[i] < pivot$  then
18:    $i \leftarrow i + 1$ 
19: end if
20: if  $Z[i] \neq pivot$  then
21:   SWAP( $Z[i], Z[r]$ )
22: end if
23: return  $i$ 
```

Sortiervverfahren

Quicksort - Laufzeit



Effizienzüberlegung Rekursion

Im **Best Case** wird die Menge durch das Pivot-Element in der Mitte halbiert. Daraus ergibt sich ein Zerlegungsbaum der Höhe

$$\lg n$$

In jeder Zeile des Baums sind n Elemente für die nächste Partitionierung zu bearbeiten:

1. Zeile: n

2. Zeile: $n/2 + n/2 = n$

3. Zeile: $n/4 + n/4 + n/4 + n/4 = n$

...

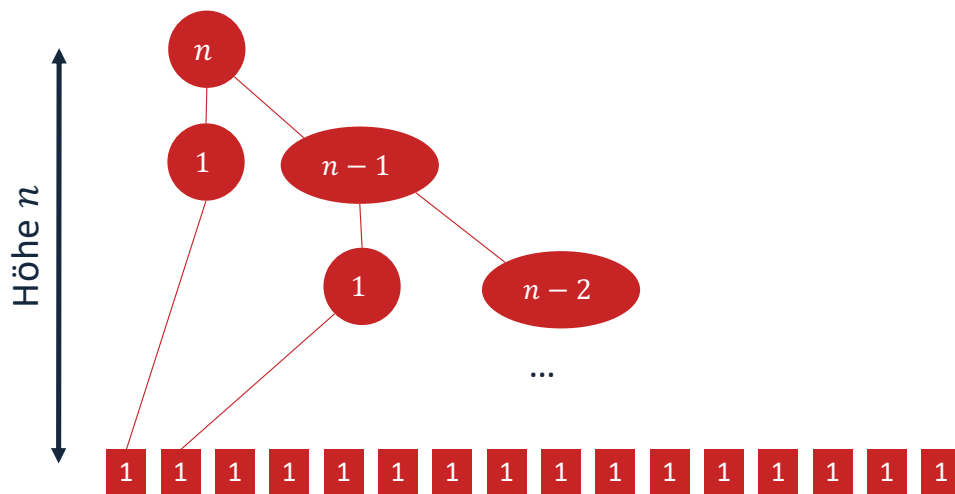
Letzte Zeile: $n \cdot 1 = n$

Da die Partitionierung mit $O(n)$ erfolgt, ergibt sich insgesamt

$$O(n \lg n)$$

Sortiervverfahren

Quicksort - Laufzeit



Effizienzüberlegung Rekursion

Im **Worst Case** teilt Partitionierung jeweils nur ein Element ab. Dadurch ergibt sich eine Höhe n für den Zerlegungsbaum.

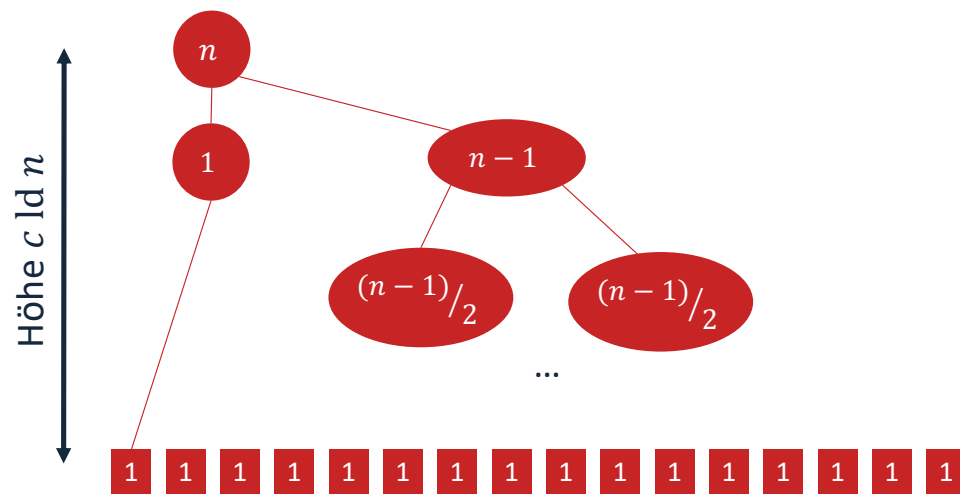
In jeder Zeile des Baums sind im Mittel $n/2$ Elemente für die nächste Partitionierung zu bearbeiten.

Da die Partitionierung mit $O(n)$ erfolgt, ergibt sich insgesamt

$$O(n^2)$$

Sortiervverfahren

Quicksort - Laufzeit



Effizienzüberlegung Rekursion

Im **Average Case** sind gute und schlechte Partitionierungen im Baum verteilt. Damit wird die Tiefe des Baums nicht n erreichen, sondern weiterhin mittels $c \lg n$ mit $c > 1$ angegeben werden können.

Die genaue mathematische Analyse über alle Permutationen, die auftreten können, ergibt $c \approx 1,39$.

Es ergibt sich wiederum

$$O(n \lg n)$$

Sortiervverfahren

Quicksort - Laufzeit

Sortiervverfahren	Worst Case	Average Case	Best Case
Quicksort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$

Hinweis:

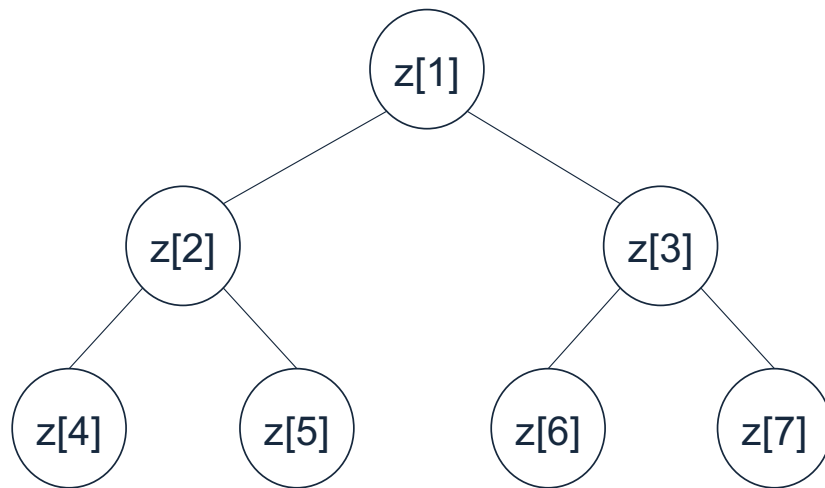
$$\log n = \frac{\lg n}{\lg 10}$$

M.a.W. zwischen den Logarithmen kann mit konstantem Faktor umgerechnet werden, d.h. für Landau-Symbole irrelevant.

Heapsort

Sortiervverfahren

Heapsort – Definition "Heap"



Unter einem **Heap** versteht man die Abbildung eines binären (d.h. jeder Knoten hat max. 2 Kinder) Baumes auf ein Array. Die Wurzel des Baumes wird im ersten Element des Arrays abgelegt. Die weiteren Knoten sind dann zeilenweise angeordnet.

Regeln für die Navigation in einem Heap

- Der Elternknoten eines Knoten i hat den Index $i/2$ (abgerundet)
- Der linke Kindknoten eines Knoten i hat den Index $2i$: $\text{LEFT}(i) \rightarrow 2i$
- Der rechte Kindknoten eines Knoten i hat den Index $2i+1$: $\text{RIGHT}(i) \rightarrow 2i+1$

Hinweis: Um die Berechnungen einfacher darstellen zu können, ist in diesem Kapitel der erste Arrayindex "1". Bei einer Implementierung ist dies entsprechend zu berücksichtigen.

Sortiervverfahren

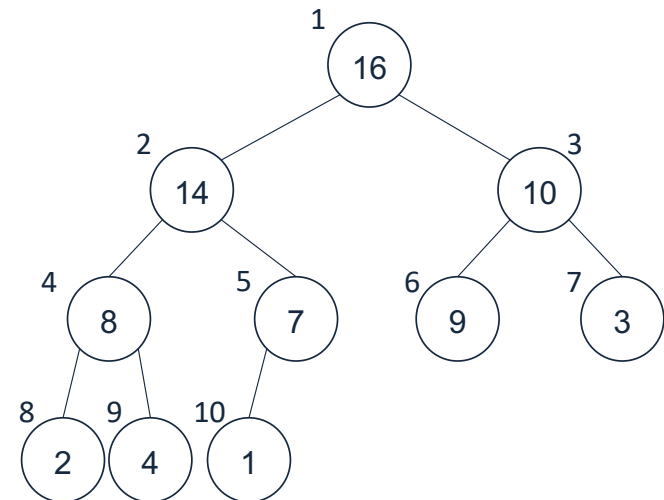
Heapsort – Definition "Max-Heap"

Ein Array erfüllt die **Max-Heap**-Eigenschaft, wenn jeder Elternknoten größer oder gleich seinen Kindknoten ist. Für jeden Knoten i gilt:

$$z[i] \geq z[2i] \text{ und } z[i] \geq z[2i + 1]$$

Beispielarray mit Max-Heap-Eigenschaft:

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Sortierverfahren

Heapsort

HEAPSORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$            ▷ Achtung:  $Z[1 \dots n]$ 
2: MAKE-MAX-HEAP( $Z$ )
3:  $\text{heapsize} \leftarrow n$ 
4: for  $i \leftarrow n$  downto 2 do
5:   SWAP( $Z[1], Z[i]$ )
6:    $\text{heapsize} \leftarrow \text{heapsize} - 1$ 
7:   MAX-HEAPIFY( $Z, 1, \text{heapsize}$ )
8: end for
```

Lösungsidee

Stelle im unsortierten Array durch Vertauschungen die Max-Heap-Eigenschaft her (MAKE-MAX-HEAP). Das erste Element ist dann das größte im Array.

Vertausche das erste Element (Maximum) mit dem letzten Element und stelle im verbliebenen Array (ohne das letzte Element) wieder die Max-Heap-Eigenschaft her (MAX-HEAPIFY).

Wiederhole den letzten Schritt, bis das Array sortiert ist.

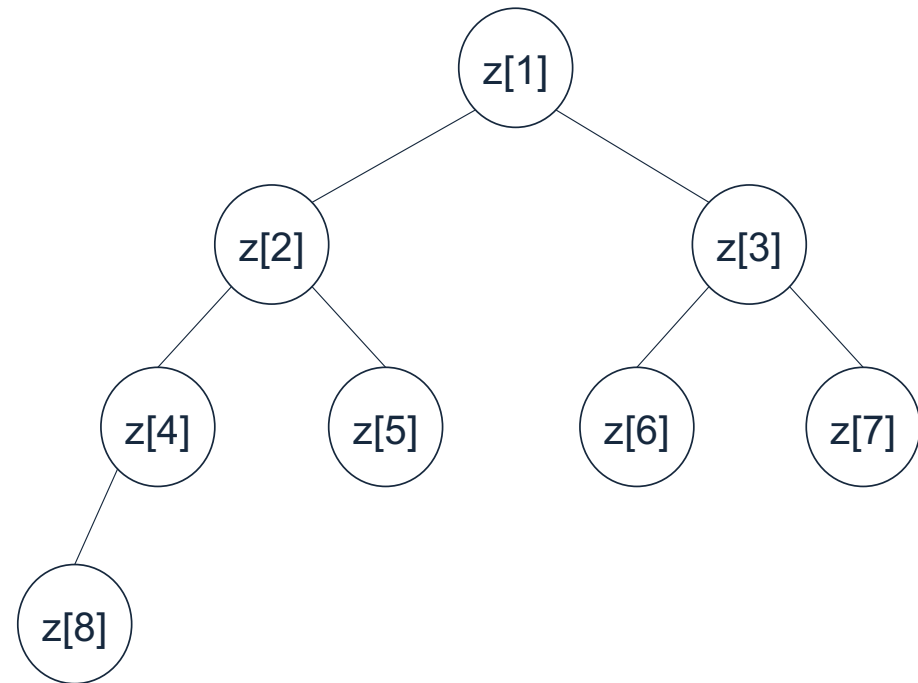
Sortiervverfahren

Heapsort – Herstellen der Max-Heap-Eigenschaft

Rollenspiel:

Stellen Sie sich "unsortiert" in einem Array auf.

Beginnend am Ende stellen wir die Max-Heap-Eigenschaft her, indem wir den Wert ggf. "absinken" lassen.

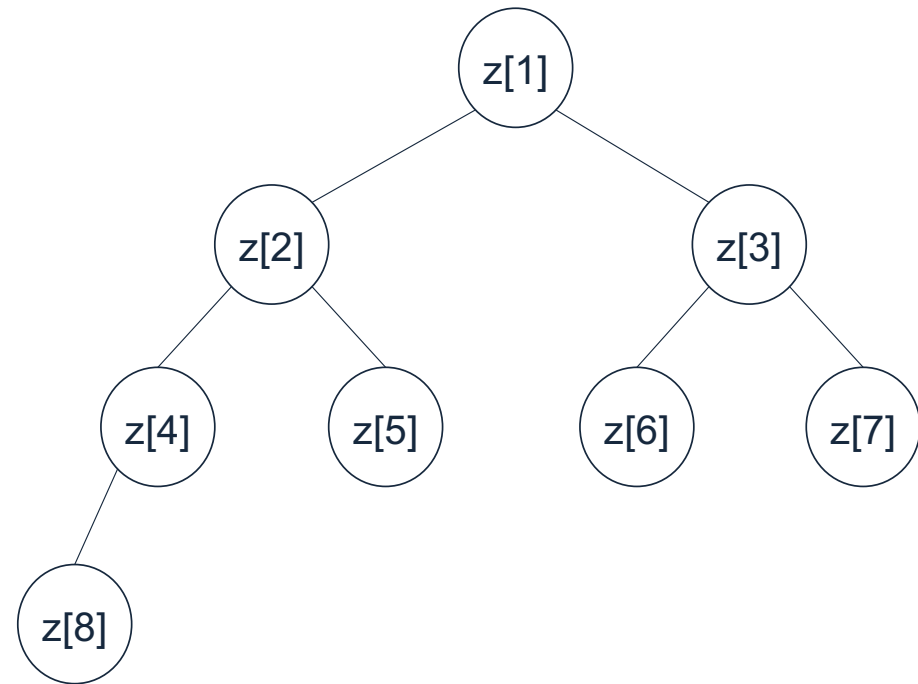


Sortiervverfahren

Heapsort – Tausch des größten Elements nach hinten

Rollenspiel:

Das erste Element geht ans Ende,
der zu sortierende Heap
verkleinert sich.



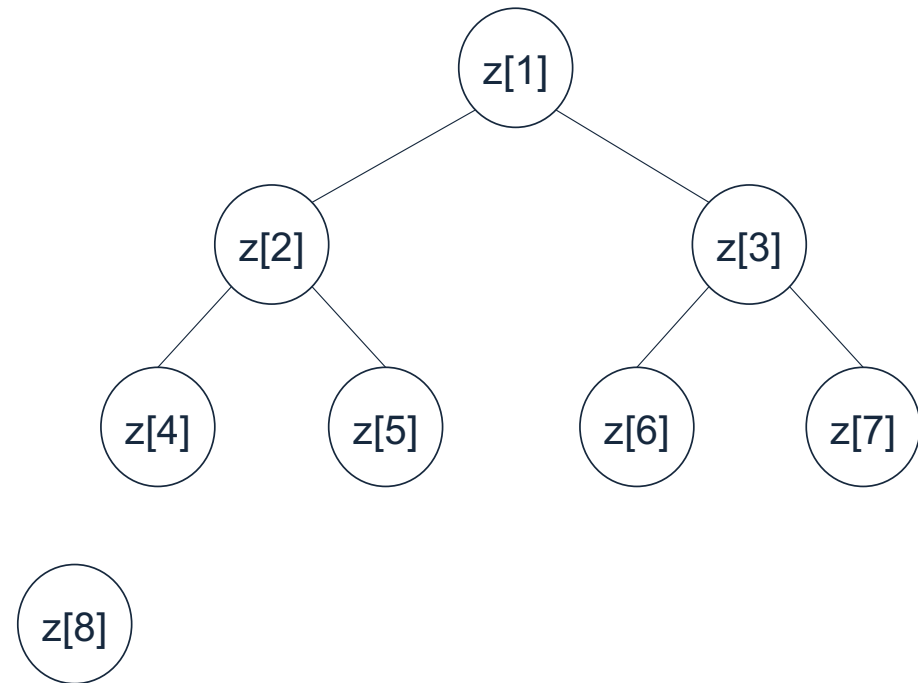
Sortiervverfahren

Heapsort – Wiederherstellen der Max-Heap-Eigenschaft

Rollenspiel:

Das an $z[1]$ eingefügte Element verstößt u.U. gegen die Max-Heap-Eigenschaft und muss absinken.

Danach wieder Tausch ans Ende.



Sortierv Verfahren

Heapsort – Wiederherstellen der Max-Heap-Eigenschaft

MAX-HEAPIFY($Z, i, \text{heapsize}$)

```
1:  $l \leftarrow \text{LEFT}(i)$            ▷ Linker Kindknoten
2:  $r \leftarrow \text{RIGHT}(i)$         ▷ Rechter Kindknoten
3:  $\text{max} \leftarrow i$ 
4: if  $l \leq \text{heapsize}$  and  $Z[l] > Z[i]$  then
5:    $\text{max} \leftarrow l$ 
6: end if
7: if  $r \leq \text{heapsize}$  and  $Z[r] > Z[\text{max}]$  then
8:    $\text{max} \leftarrow r$ 
9: end if
10: if  $\text{max} \neq i$  then
11:   SWAP( $Z[i], Z[\text{max}]$ )
12:   MAX-HEAPIFY( $Z, \text{max}, \text{heapsize}$ )
13: end if
```

Effizienzüberlegung

Die Tiefe des Heap ist $\ln n$, d.h. die Rekursion findet max. $\ln n$ mal statt.

Damit besitzt MAX-HEAPIFY eine Laufzeitkomplexität von $O(\log n)$

Hinweis: Die maximale Laufzeit wird erreicht, wenn das fehlerhafte Element ganz nach unten durchgereicht werden muss.

Sortiervverfahren

Heapsort – Initiales Herstellen der Max-Heap-Eigenschaft

MAKE-MAX-HEAP(Z)

```
1:  $n \leftarrow \text{length}(Z)$  ▷ Achtung:  $Z[1 \dots n]$   
2: for  $i \leftarrow n/2$  downto 1 do  
3:   MAX-HEAPIFY( $Z, i, n$ )  
4: end for
```

Effizienzüberlegung

Mit zuvor vorgestellten MAX-HEAPIFY kann man ein Array $Z[1..n]$, das Werte in beliebiger Reihenfolge enthält, so umformen, dass es danach Max-Heap-Eigenschaft besitzt. Dies erreicht man mit dem nebenstehenden Algorithmus MAKE-MAX-HEAP.

Glücklicherweise ist der oberflächliche Betrachtung, dass $\frac{n}{2}$ mal MAX_HEAPIFY mit $O(\log n)$ falsch, weil viele der MAX_HEAPIFY-Aufrufe nur sehr geringe Höhen haben. Es ergibt sich für MAKE-MAX-HEAP eine Laufzeitkomplexität von

$$O(n)$$

Sortierv Verfahren

Heapsort

HEAPSORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$            ▷ Achtung:  $Z[1 \dots n]$ 
2: MAKE-MAX-HEAP( $Z$ )
3:  $\text{heapsize} \leftarrow n$ 
4: for  $i \leftarrow n$  downto 2 do
5:   SWAP( $Z[1], Z[i]$ )
6:    $\text{heapsize} \leftarrow \text{heapsize} - 1$ 
7:   MAX-HEAPIFY( $Z, 1, \text{heapsize}$ )
8: end for
```

Effizienzbetrachtung

Da MAKE-MAX-HEAP eine Laufzeitkomplexität von $O(n)$ besitzt, dominiert die Schleife (Zeilen 4-8). Dort wird in der Größenordnung n mal MAX-HEAPIFY aufgerufen, das die Komplexität $O(\log n)$ besitzt. Somit ergibt sich insgesamt eine Laufzeitkomplexität von

$$O(n \log n)$$

Hinweis: Eine Begrenzung der Vertauschungen wie bei MAKE-MAX-HEAP kann hier nicht angegeben werden, weil bei jedem Durchlauf das zur Wurzel bewegte Element bis zu einem Blatt durchgereicht werden kann.

Sortiervverfahren

Heapsort - Laufzeit

Sortiervverfahren	Worst Case	Average Case	Best Case
Quicksort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Untere Schranke für das Sortierproblem

Sortiervverfahren

Untere Schranke für das Sortierproblem

- Wir haben Sortieralgorithmen identifiziert, die im Durchschnitt die Zeitkomplexität $O(n \log n)$ aufweisen.
- Ist dies bereits das Beste, was man erreichen kann?
- In diesem Fall wäre das Sortierproblem „ $\Omega(n \log n)$ -hart“
- Wir werden im Folgenden beweisen, dass allein mit Vergleichen im Rahmen einer Ordnungsrelation (z.B. $\leq$) kein besseres Verfahren existieren kann.

Sortiervverfahren

Untere Schranke für das Sortierproblem

- **Annahme:** In einem Array der Länge n sind die Elemente $e_1 \dots e_n$ enthalten, die paarweise mit einer Ordnungsrelation $\leq$ verglichen werden können
- **Gesucht:** Die Permutation der Elemente, so dass paarweise zwischen allen Nachbarn die Ordnungsrelation gegeben ist.
- **Überlegung:** es gibt $n!$ Permutationen, von denen genau eine die Richtige ist.

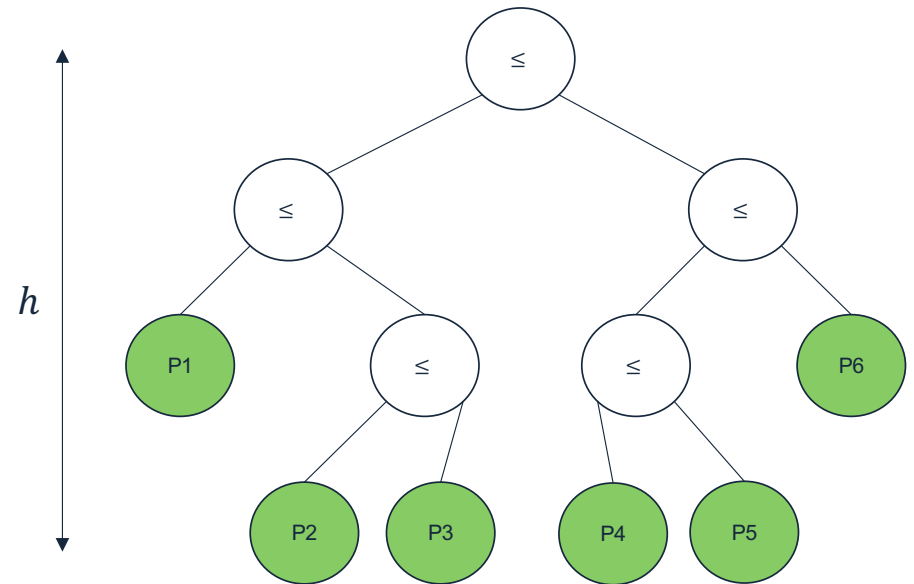
Zur Ermittlung der richtigen Permutation müssen Vergleiche ausgeführt und Entscheidungen getroffen werden.

→ **Entscheidungsbaum** mit den Permutationen als Blätter

Sortiervverfahren

Untere Schranke für das Sortierproblem

- **Entscheidungsbaum** für $n = 3$ mit $n! = 6$ Permutationen.
- Bei optimaler Anordnung der Entscheidungen entsteht ein binärer Baum mit den Permutationen P als Blätter.
- Im **Worst Case** muss zur Identifikation der richtigen Permutation die gesamte Höhe h des Baums durchlaufen werden.



Sortiervverfahren

Untere Schranke für das Sortierproblem

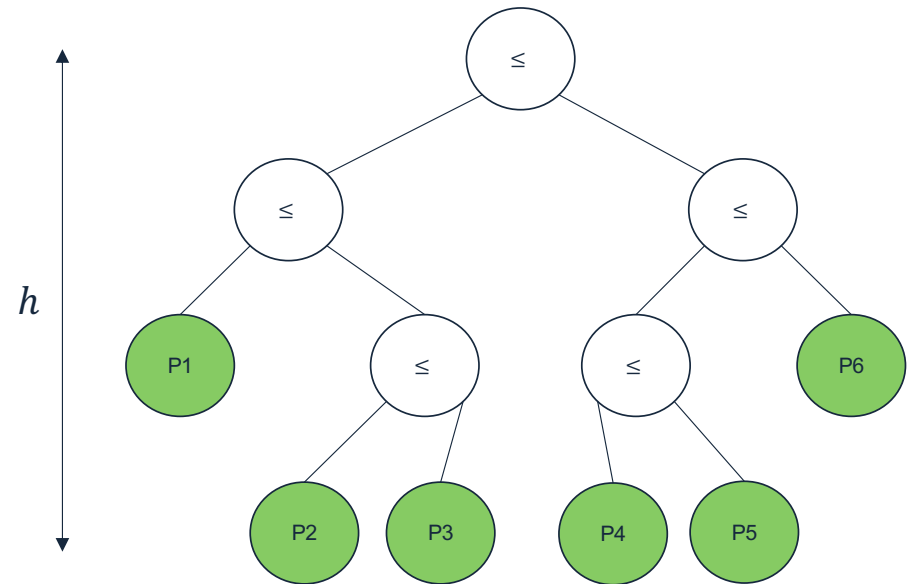
Wie hoch ist der Entscheidungsbaum?

Bei n Elementen ergeben sich $n!$ Permutationen. Der Entscheidungsbaum muss hoch genug sein, dass diese als Blätter geführt werden können.

Ein binärer Entscheidungsbaum der Höhe h hat maximal 2^h Blätter (wenn alle Zweige bis zum Ende durchgezogen werden).

Somit:

$$n! \leq 2^h$$
$$\lg n! \leq h$$



Sortiervverfahren

Untere Schranke für das Sortierproblem

Anzahl der Entscheidungen

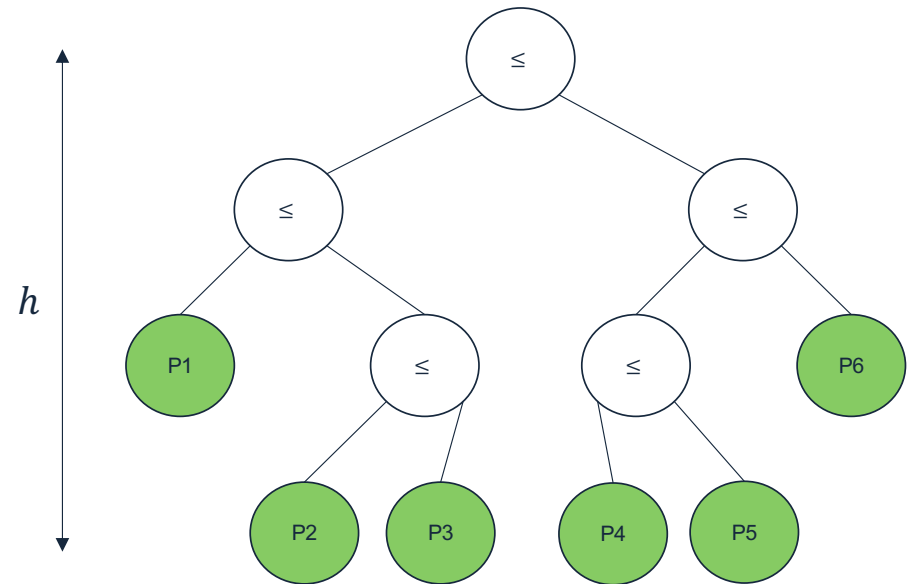
Somit:

$$n! \leq 2^h$$
$$\lg n! \leq h$$

Es müssen also in der Größenordnung $\lg n!$ Entscheidungen getroffen werden, um bei der richtigen Permutation anzukommen.

Ohne Beweis: $\lg n! \in \Theta(n \log n)$

⇒ **Sortierproblem ist „ $\Omega(n \log n)$ -hart“**



Counting Sort

Sortierv Verfahren

Counting-Sort

Es wurde gezeigt, dass ein Sortierv Verfahren, das ausschließlich auf Vergleiche abhebt, keine bessere Laufzeitkomplexität besitzen kann als

$$O(n \log n)$$

Wenn man aber weitere Vorbedingungen für die zu sortierenden Objekte gegeben sind, so kann auch ein besseres Laufzeitverhalten erreicht werden.

Vorbedingungen für den Counting-Sort

Um den Counting-Sort anwenden zu können, sind folgende Vorbedingungen notwendig:

- Sortiert werden positive ganze Zahlen bzw. das Sortierkriterium sind positive ganze Zahlen (d.h. es können noch weitere „Satelitendaten“ in einem struct o.Ä. existieren)
- Man kennt eine obere Schranke für die zu sortierenden Zahlen

Sortiervverfahren

Counting-Sort (Beispiel)

Die Funktionsweise des Counting-Sort soll an einem Beispiel aufgezeigt werden.

Die zu sortierenden Zahlen sind in einem Feld A enthalten; das Ergebnis soll anschließend in einem weiteren Feld B gleicher Größe bereitgestellt werden.
(d.h. es ist kein „in-place“-Algorithmus)

Die obere Schranke der zu sortierenden Zahlen sein k .

1. Schritt:

- Zu sortierendes Array A :

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

- Maximum der zu sortierenden Zahlen: $k = 5$

```

1:  $C \leftarrow \text{new Array}[0 \dots k]$ 
2: for  $i \leftarrow 0$  to  $k$  do
3:    $C[i] \leftarrow 0$ 
4: end for

```

- Initiale Befüllung des Array C :

0	1	2	3	4	5
0	0	0	0	0	0

Sortiervverfahren

Counting-Sort (Beispiel)

2. Schritt:

Anschließend wird die Häufigkeit jedes Wertes gezählt und im Feld C vermerkt.

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

```
5: for  $j \leftarrow 1$  to  $length(A)$  do  
6:    $C[A[j]] \leftarrow C[A[j]] + 1$   
7: end for
```

0	1	2	3	4	5
2	0	2	3	0	1

Sortiervverfahren

Counting-Sort (Beispiel)

3. Schritt:

Nach einem nochmaligen Durchlaufen von C ist jetzt nicht mehr die Anzahl des jeweiligen Wertes, sondern die Anzahl der Werte kleiner oder gleich vermerkt.

```
8: for  $i \leftarrow 1$  to  $k$  do  
9:    $C[i] \leftarrow C[i] + C[i - 1]$   
10: end for
```

0	1	2	3	4	5
2	0	2	3	0	1

0	1	2	3	4	5
2	2	4	7	7	8

Sortiervverfahren

Counting-Sort (Beispiel)

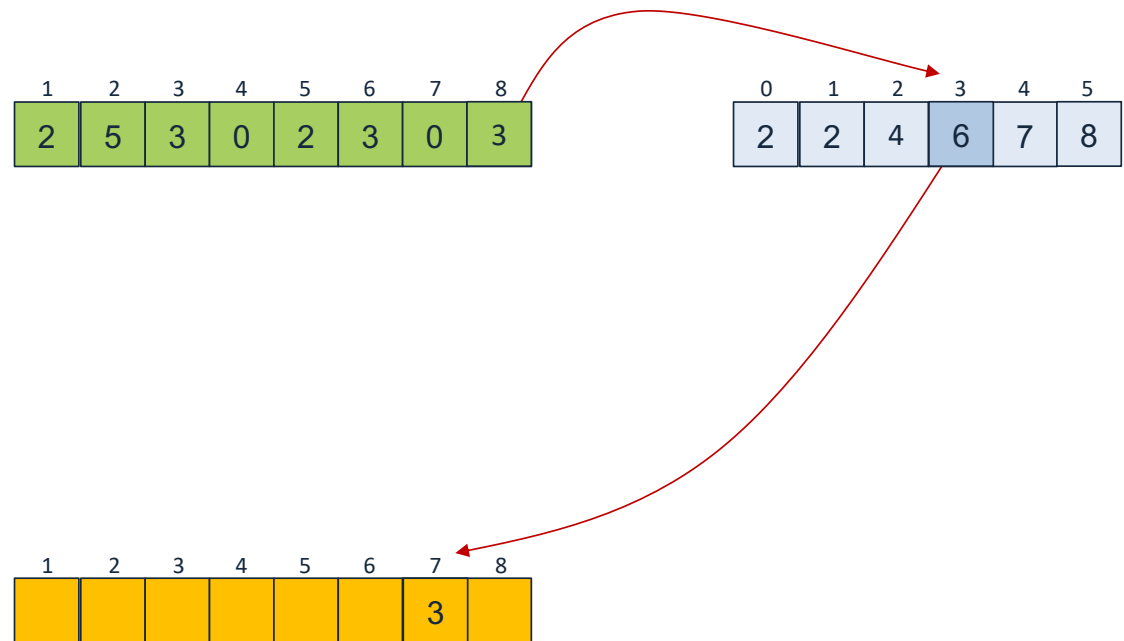
4. Schritt:

Zuletzt wird das Ergebnis-Feld B befüllt.

```

11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for

```



Sortiervverfahren

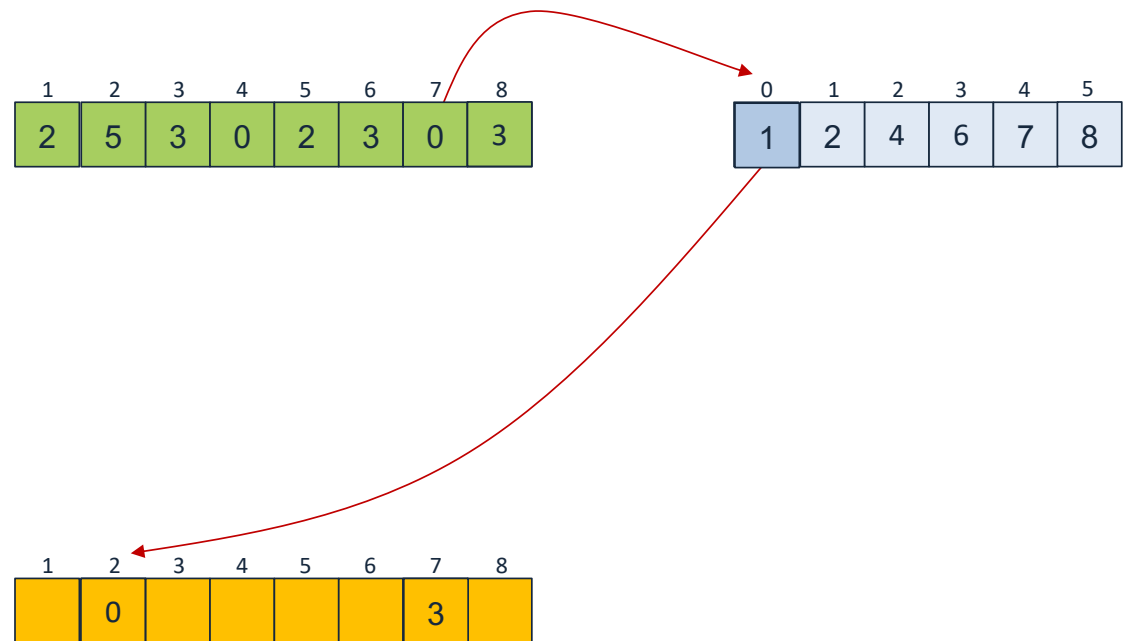
Counting-Sort (Beispiel)

4. Schritt:

Zuletzt wird das Ergebnis-Feld B befüllt.

```

11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for
    
```



Sortiervverfahren

Counting-Sort (Beispiel)

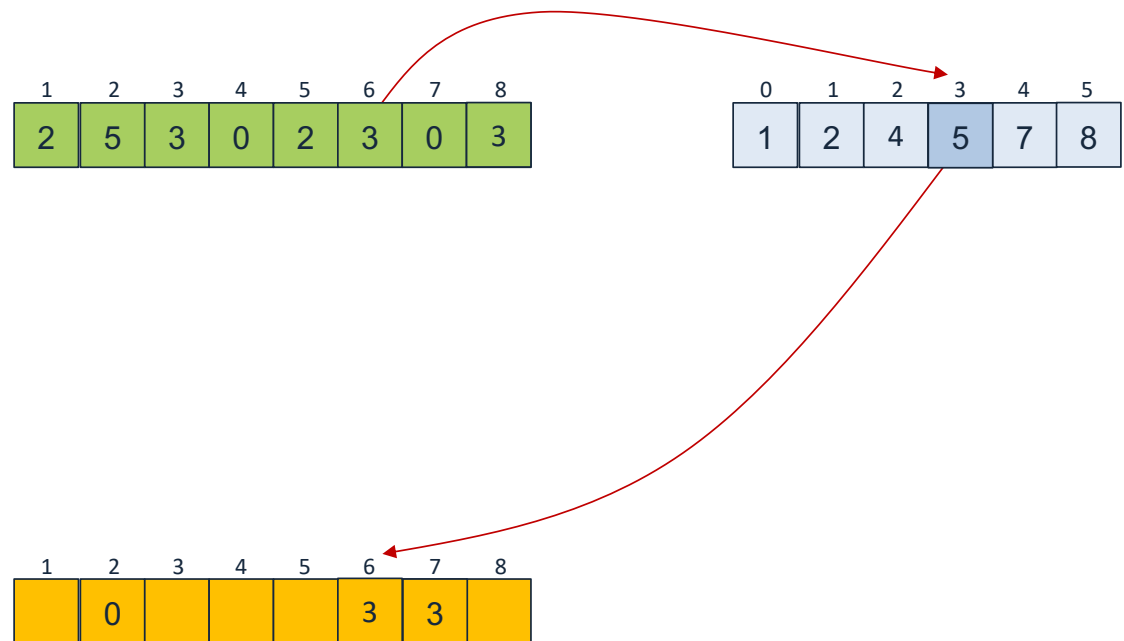
4. Schritt:

Zuletzt wird das Ergebnis-Feld B befüllt.

```

11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for

```



Sortiervverfahren

Counting-Sort

Effizienzüberlegung

Im Algorithmus finden sich nur unverschachtelte Schleifen über n (Länge des Feldes A) und k . Damit ergibt sich zunächst eine Laufzeitkomplexität von

$$O(n + k)$$

Normalerweise wird der Counting-Sort verwendet in Szenarien mit $k \in O(n)$, d.h. bei wachsendem n erhöht sich die Schranke k max. linear. Daraus ergibt sich dann für den Counting-Sort

$$O(n)$$

COUNTING-SORT(A, B, k)

```
1:  $C \leftarrow \text{new Array}[0 \dots k]$ 
2: for  $i \leftarrow 0$  to  $k$  do
3:    $C[i] \leftarrow 0$ 
4: end for
5: for  $j \leftarrow 1$  to  $\text{length}(A)$  do
6:    $C[A[j]] \leftarrow C[A[j]] + 1$ 
7: end for
8: for  $i \leftarrow 1$  to  $k$  do
9:    $C[i] \leftarrow C[i] + C[i - 1]$ 
10: end for
11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for
```

Warum sortieren wir?

Warum sortieren wir?

Durchsuchen einer unsortierten Menge

Um die Anwesenheit eines Werts / Objekts in einer unsortierten Menge festzustellen bzw. auf dessen Attribute zugreifen zu können, müssen wir ggf. die gesamte Menge durchlaufen, im Schnitt aber die halbe Menge (manchmal werden wir schnell fündig, manchmal nicht). Damit ergibt sich:

Laufzeit im Durchschnitt $O(n)$

Warum sortieren wir?

Durchsuchen einer sortierten Menge

BINSEARCH(Z, s, l, r)

```
1: if  $l > r$  then                                ▷  $l$  hat  $r$  überholt
2:   return -1                                     ▷  $s$  nicht in  $Z$  enthalten
3: end if
4:  $m \leftarrow (l + r) / 2$                         ▷ Mitte des aktuellen Bereichs
5: if  $s < Z[m]$  then
6:   return BINSEARCH( $Z, s, l, m - 1$ )
7: else if  $s > Z[m]$  then
8:   return BINSEARCH( $Z, s, m + 1, r$ )
9: else
10:  return  $m$ 
11: end if
```

Bei einer sortierten Menge kann durch eine binäre Suche die Laufzeit auf $O(\log n)$ reduziert werden.

M.a.W.: Sortieren lohnt sich, wenn viele Suchvorgänge erwartet werden und der Aufwand für das Sortieren durch Einsparungen beim Suchen aufgewogen wird.

Binärer Suchbaum

Binärer Suchbaum

Grundidee

Statt es überhaupt erst zu einer (großen) Menge unsortierter Elemente in einem Feld/Array/Vektor kommen zu lassen, wird bereits beim Zusammenstellen der Menge eine besser geeignete Datenstruktur verwendet, effizientes Suchen und Sortieren erlaubt.

Die Datenstruktur soll darüber hinaus dynamisch wachsen können, weil die Anzahl der Elemente a priori nicht bekannt ist.

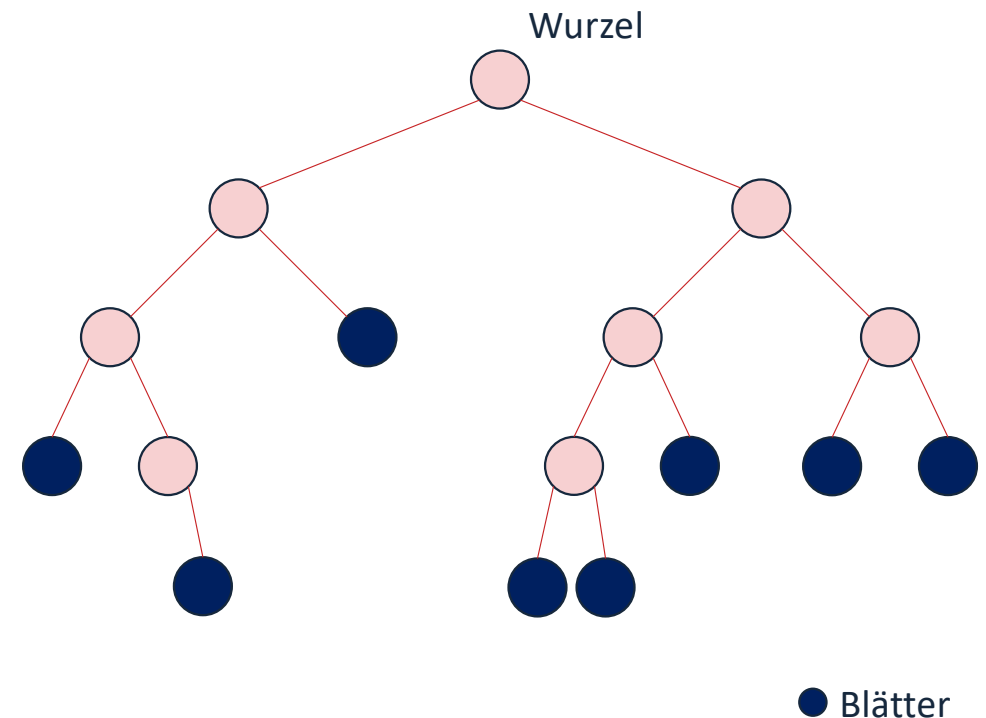
Eine Möglichkeit für eine solche Datenstruktur ist der **binäre Suchbaum**.

Binärer Suchbaum

Binärer Baum

Ein binärer Baum

- besteht aus Knoten mit max. zwei (daher binär) Nachfolgern/Kindern
- Mit Ausnahme der Wurzel besitzt jeder Knoten genau einen Elternknoten
- Knoten ohne Kinder heißen äußere Knoten (oder auch Blätter)
- Anders als beim Heap, bei dem sich der Baum gleichmäßig zeilenweise füllt, kann ein binärer Baum auch sehr unbalanciert sein.
- Ein einfaches Abbilden auf ein Array ist daher nicht möglich
- Typischerweise werden binäre Bäume durch verkettete Elemente auf dem Heap gebildet.



Binärer Suchbaum

Binärer Suchbaum

Ein binärer Baum wird zum binären Suchbaum, wenn die Knoten vergleichbare Werte (Keys) besitzen und gilt:

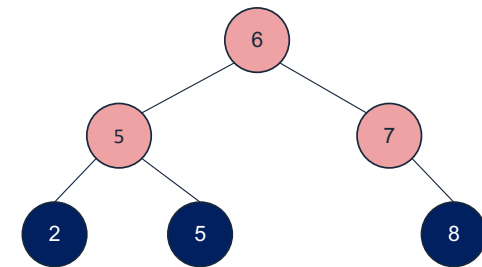
Sei x ist ein Knotenwert in Baum $\Rightarrow$

$\forall v$ im linken Teilbaum: $v \leq x$

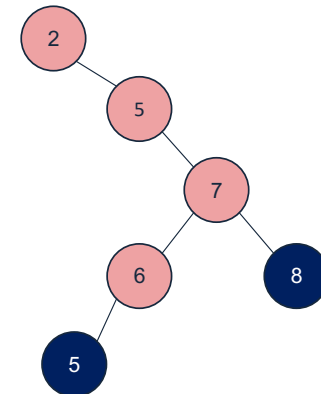
$\forall w$ im rechten Teilbaum: $x \leq w$

Die beiden Beispiele auf der rechten Seite repräsentieren die gleichen Elementmengen, d.h. ein binärer Suchbaum zu einer Menge ist nicht eindeutig.

Beispiel 1



Beispiel 2



Binärer Suchbaum

Sortieren

Die sortierte Ausgabe wird auch

Inorder-Traversierung bzw. **Inorder-Tree-Walk** genannt.

„Inorder“, weil die Verarbeitung des aktuellen Werts zwischen der Verarbeitung des linken und des rechten Teilbaums stattfindet.

Der nebenstehende Algorithmus wird hierzu initial mit dem Wurzelknoten als Parameter aufgerufen.

INORDER-TREE-WALK(x)

```
1: if  $x \neq \text{NIL}$  then  
2:   INORDER-TREE-WALK( $x.\text{left}$ )  
3:   print  $x.\text{key}$   
4:   INORDER-TREE-WALK( $x.\text{right}$ )  
5: end if
```

Jeder Knoten wird durch eine Struktur repräsentiert mit den Attributen:

key: Schlüsselwert des Knoten

left: Verweis zum linken Nachfolger (oder NIL)

right: Verweis zum rechten Nachfolger (oder NIL)

Effizienzbetrachtung

Jeder Knoten wird genau einmal besucht, daher Laufzeit: $O(n)$

Binärer Suchbaum

Suchen

Gesucht wird der Knoten im Baum, der den angegebenen Schlüsselwert besitzt. Existiert ein solcher Knoten, wird eine Referenz auf ihn zurückgegeben, sonst NIL

Der nebenstehende Algorithmus wird hierzu initial mit der Wurzel des Baums als Startknoten aufgerufen.

Effizienzbetrachtung

Laufzeit: $O(h)$, wobei h die Höhe des Baums ist

TREE-SEARCH(*start*, *key*)

```
1: if start = NIL or key = start.key then  
2:   return start  
3: end if  
4: if key < start.key then  
5:   return TREE-SEARCH(start.left, key)  
6: else  
7:   return TREE-SEARCH(start.right, key)  
8: end if
```

Binärer Suchbaum

Einfügen eines Elements

Sei z der einzufügende Knoten und T der betroffene Baum.

Ausgehend vom Wurzelknoten wird durch einen Abstieg ein NIL gesucht, das durch z ersetzt werden kann.

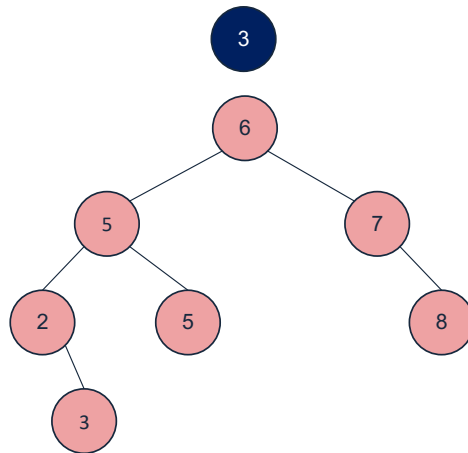
y verweist dann auf den vorgelagerten äußeren Knoten, in dem z verankert werden muss.

TREE-INSERT(T, z)

```
1:  $y \leftarrow \text{NIL}$ 
2:  $x \leftarrow T.\text{root}$ 
3: while  $x \neq \text{NIL}$  do
4:    $y \leftarrow x$ 
5:   if  $z.\text{key} < x.\text{key}$  then
6:      $x \leftarrow x.\text{left}$ 
7:   else
8:      $x \leftarrow x.\text{right}$ 
9:   end if
10: end while
11: if  $y = \text{NIL}$  then
12:    $T.\text{root} \leftarrow z$ 
13: else if  $z.\text{key} < y.\text{key}$  then
14:    $y.\text{left} \leftarrow z$ 
15: else
16:    $y.\text{right} \leftarrow z$ 
17: end if
```

Binärer Suchbaum

Einfügen eines Elements



Effizienzbetrachtung

Laufzeit: $O(h)$, wobei h die Höhe des Baums ist

TREE-INSERT(T, z)

```

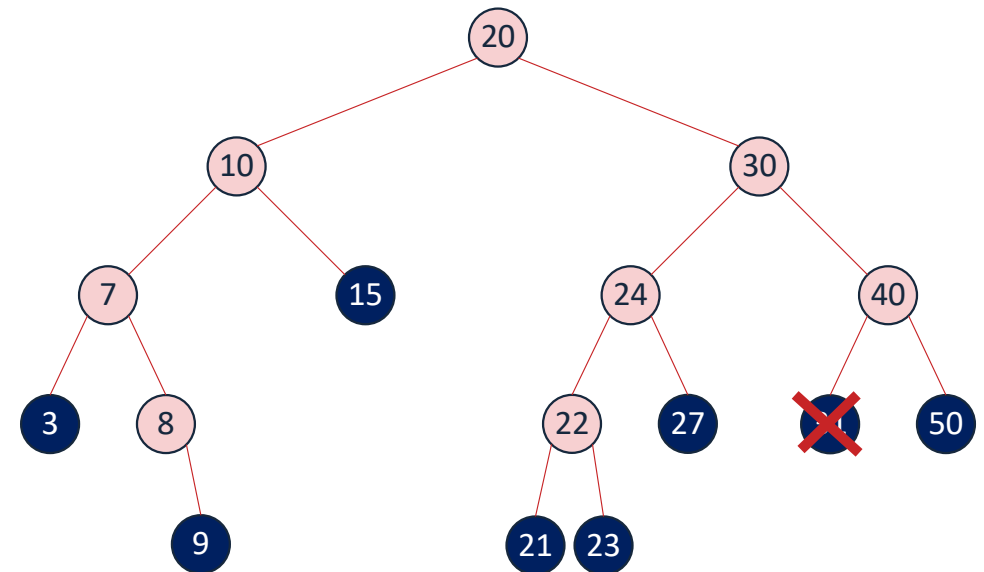
1:  $y \leftarrow \text{NIL}$ 
2:  $x \leftarrow T.\text{root}$ 
3: while  $x \neq \text{NIL}$  do
4:    $y \leftarrow x$ 
5:   if  $z.\text{key} < x.\text{key}$  then
6:      $x \leftarrow x.\text{left}$ 
7:   else
8:      $x \leftarrow x.\text{right}$ 
9:   end if
10: end while
11: if  $y = \text{NIL}$  then
12:    $T.\text{root} \leftarrow z$ 
13: else if  $z.\text{key} < y.\text{key}$  then
14:    $y.\text{left} \leftarrow z$ 
15: else
16:    $y.\text{right} \leftarrow z$ 
17: end if
  
```

Binärer Suchbaum

Löschen eines Elements

1. Fall: Entfernen eines Blatts

Ein Knoten ohne Nachfolger kann einfach entfernt werden, ohne dass die Struktur des verbleibenden Binärbaums beeinflusst wird.

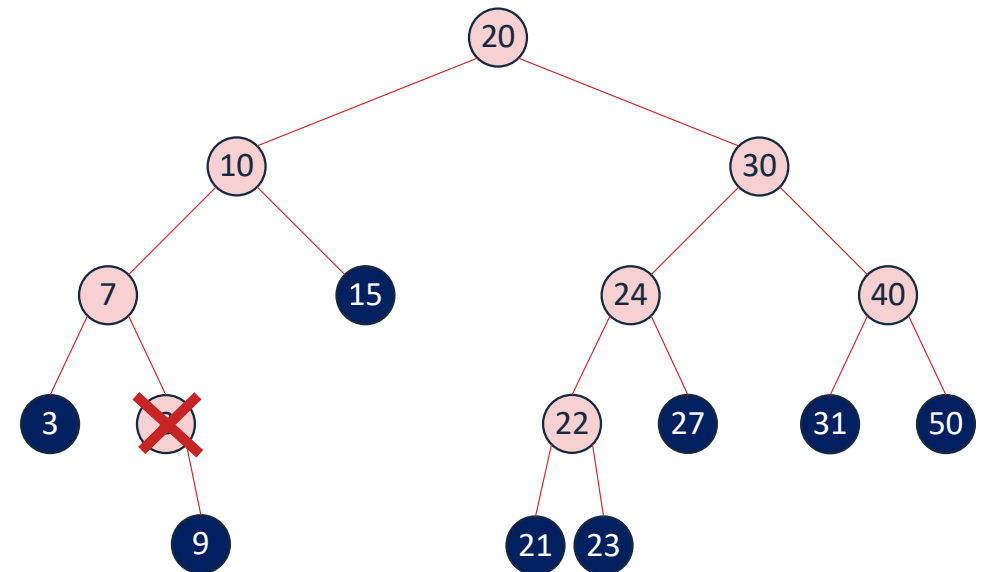


Binärer Suchbaum

Löschen eines Elements

2. Fall: Entfernen eines Knotens mit nur einem Nachfolger

Der Zeiger auf den zu entfernenden Knoten wird auf den nicht leeren Nachfolger-Knoten umgelenkt.



Binärer Suchbaum

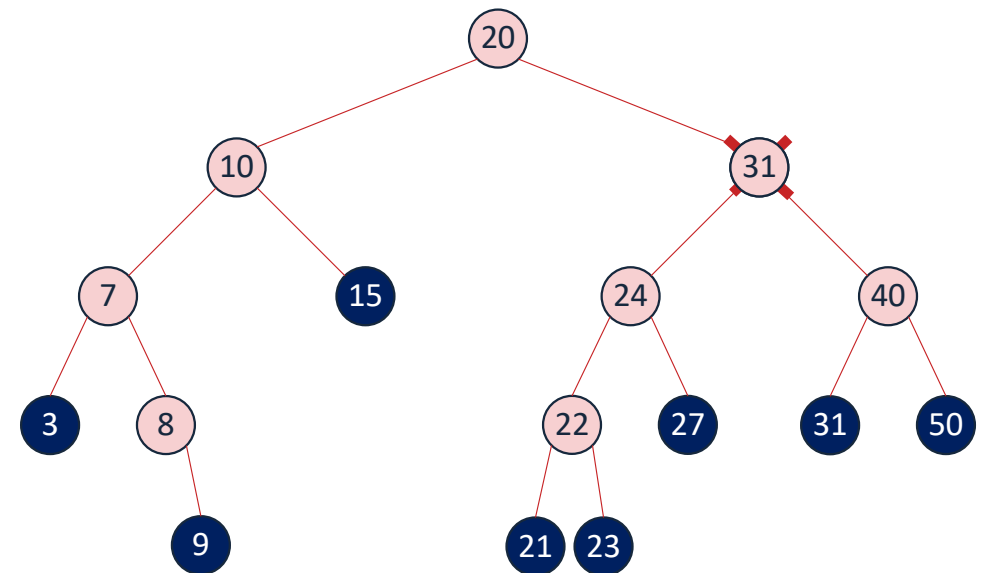
Löschen eines Elements

3. Fall: Entfernen eines Knotens mit zwei Nachfolgern

In diesem Fall muss der zu löschende Knoten durch seinen Nachfolger ersetzt werden, der sich im rechten Teilbaum befindet. Im einfachsten Fall ist dies ein äußerer Knoten.

Falls es sich beim Nachfolger um einen inneren Knoten handelt, kann dieser kein linkes Kind besitzen (sonst wäre das der Nachfolger des zu löschenden Knoten).

Wir entfernen den gefundenen Nachfolger aus dem rechten Teilbaum, indem wir das rechte Kind nach oben rücken lassen, und setzen den Nachfolger an die Stelle des zu löschenden Knoten.



Binärer Suchbaum

Laufzeit-Komplexität der Operationen

Operation	Laufzeit
Traversieren	$O(n)$
Suchen	$O(h)$
Einfügen	$O(h)$
Löschen	$O(h)$

AVL-Bäume

AVL-Bäume

Das Problem mit binären Suchbäumen

Viele Operationen (Suchen, Einfügen, Löschen) sind abhängig von der Höhe h des Baums

Die Höhe ergibt sich durch die Einfügereihenfolge der Elemente

Im Worst Case (Einfügen einer sortierten Folge mit n Elementen) ergibt sich die Höhe n

Das Problem ist vergleichbar mit dem Worst Case beim Quick-Sort

AVL-Bäume

Lösungsansatz

- Ein AVL-Baum ist ein binärer Suchbaum, der bei jedem Einfügen oder Löschen „balanciert“ wird
- Benannt nach Georgi Maximowitsch **A**delson-**V**elski und Jewgeni Michailowitsch **L**andis
- Der Balancefaktor BF eines Knotens k ist definiert durch

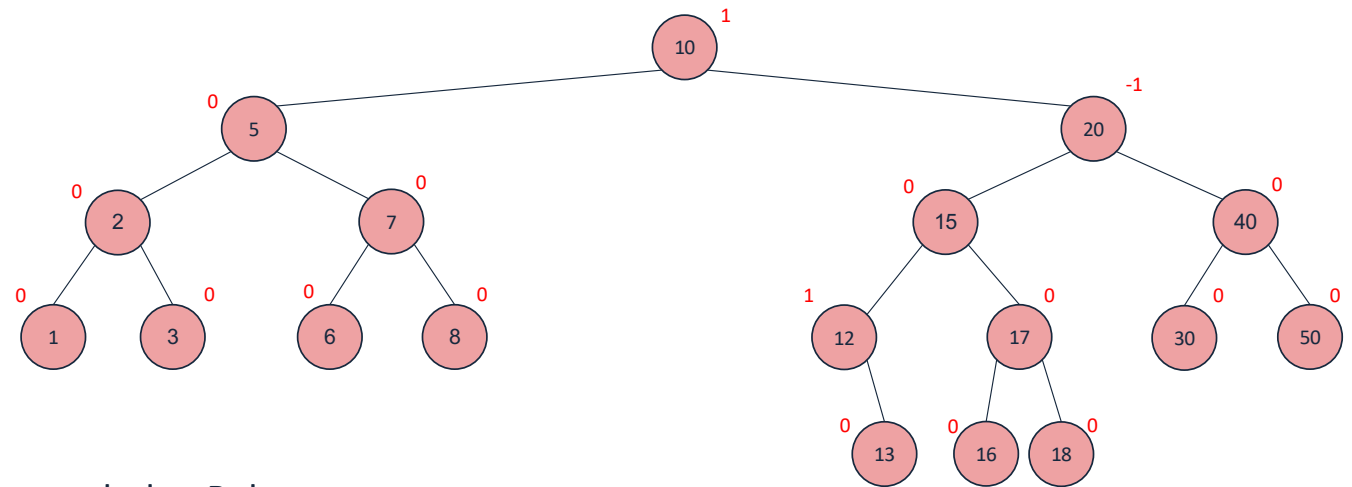
$$BF(k) = HEIGHT(k.right) - HEIGHT(k.left)$$

- Für jeden Knoten k des Baums muss gelten:

$$BF(k) \in \{-1, 0, 1\}$$

AVL-Bäume

Lösungsansatz



Die Datenstruktur für einen Knoten sollte auch den Balance-Faktor und einen Verweis zum Elternknoten enthalten:

- key*: Schlüsselwert des Knoten
- balance*: Schlüsselwert des Knoten
- left*: Verweis zum linken Nachfolger (oder NIL)
- right*: Verweis zum rechten Nachfolger (oder NIL)
- parent*: Verweis zum Elternknoten

AVL-Bäume

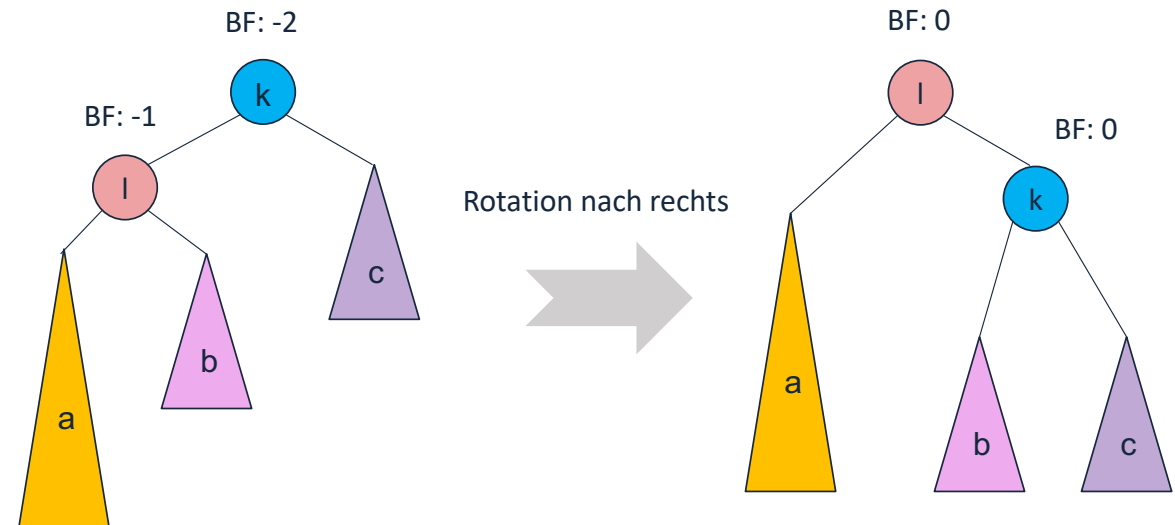
Balancieren durch Rotation

1. Beispiel

Knoten k verletzt das Balance-Kriterium und das Kind mit dem zu tiefen Teilbaum ist ausbalanciert oder in der gleichen Richtung unbalanciert, z.B.:

$$BF(k) < -1 \text{ und } BF(k.left) \leq 0$$

In diesem Fall kann die Balance durch eine einfache Rotation wieder hergestellt werden.



AVL-Bäume

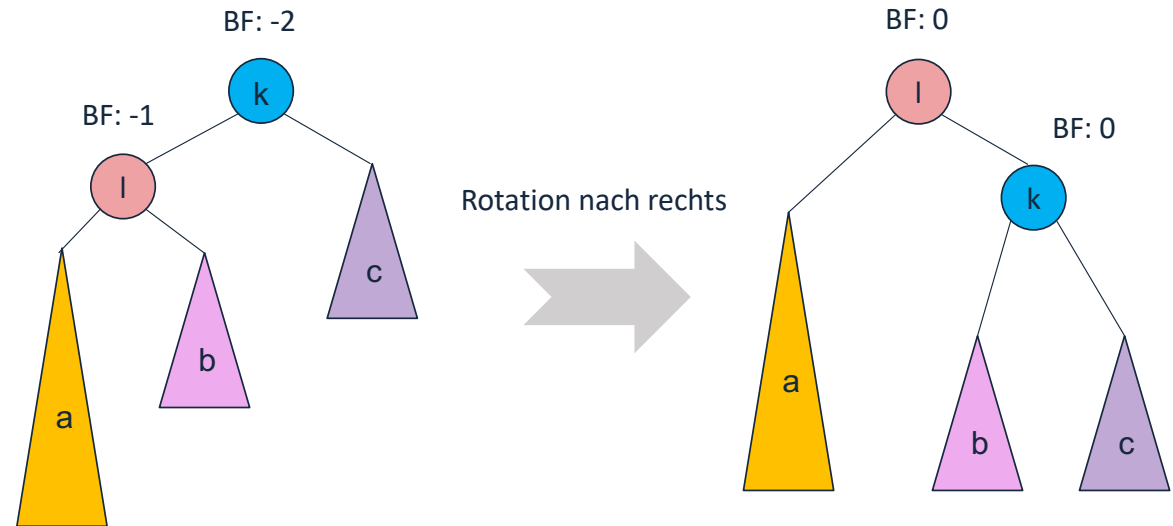
Balancieren durch Rotation

RIGHT-ROTATE(k)

```

1:  $l \leftarrow k.left$ 
2:  $l.parent \leftarrow k.parent$ 
3:  $k.left \leftarrow l.right$ 
4: if  $k.left \neq \text{NIL}$  then
5:    $k.left.parent = k$ 
6: end if
7:  $l.right \leftarrow k$ 
8:  $k.parent \leftarrow l$ 
9: if  $l.parent \neq \text{NIL}$  then
10:  if  $l.parent.right = k$  then
11:     $l.parent.right \leftarrow l$ 
12:  else if  $l.parent.left = k$  then
13:     $l.parent.left \leftarrow l$ 
14:  end if
15: end if
16:  $k.balance \leftarrow \text{HEIGHT}(k.right) - \text{HEIGHT}(k.left)$ 
17:  $l.balance \leftarrow \text{HEIGHT}(l.right) - \text{HEIGHT}(l.left)$ 
18: return  $l$ 

```



AVL-Bäume

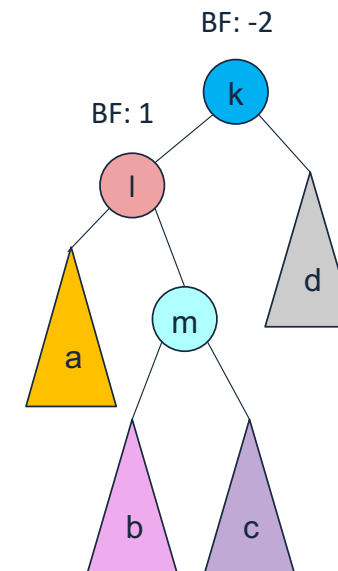
Balancieren durch Rotation

2. Beispiel

Knoten k verletzt das Balance-Kriterium und das Kind mit dem zu tiefen Teilbaum ist in der Gegenrichtung unbalanciert, z.B.:

$$BF(k) < -1 \text{ und } BF(k.\text{left}) > 0$$

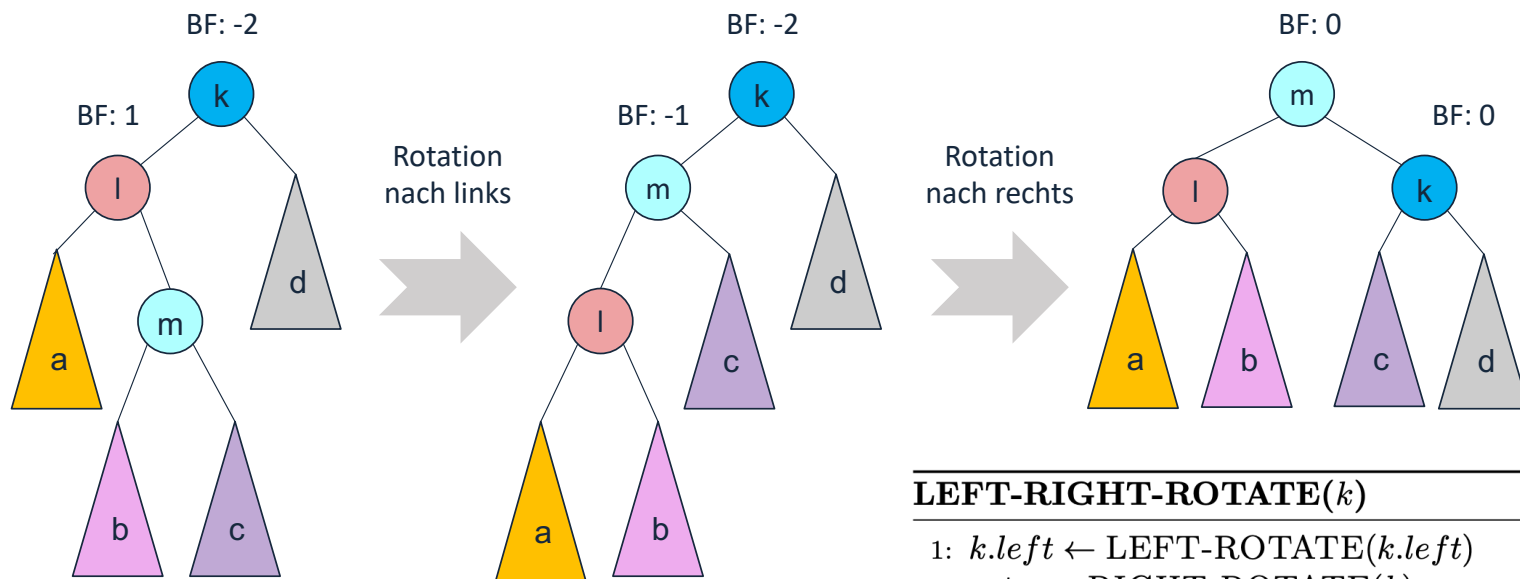
In diesem Fall stellt eine Doppelrotation die Balance wieder her.



AVL-Bäume

Balancieren durch Rotation

2. Beispiel



LEFT-RIGHT-ROTATE(k)

1: $k.left \leftarrow \text{LEFT-ROTATE}(k.left)$

2: **return** RIGHT-ROTATE(k)

AVL-Bäume

Balancieren nach Einfügen oder Löschen

Durch das Einfügen oder Löschen eines Knoten kann der Balance-Faktor mehrerer Knoten bis hinauf zur Wurzel gestört sein.

Die Korrektur erfolgt rekursiv aufsteigend zur Wurzel mit nebenstehendem Algorithmus.

AVL-BALANCE(T, k)

```
1:  $k.balance \leftarrow \text{HEIGHT}(k.right) - \text{HEIGHT}(k.left)$ 
2: if  $k.balance = -2$  then
3:   if  $\text{HEIGHT}(k.left.left) \geq \text{HEIGHT}(k.left.right)$  then
4:      $k \leftarrow \text{RIGHT-ROTATE}(k)$ 
5:   else
6:      $k \leftarrow \text{LEFT-RIGHT-ROTATE}(k)$ 
7:   end if
8: else if  $k.balance = 2$  then
9:   if  $\text{HEIGHT}(k.right.right) \geq \text{HEIGHT}(k.right.left)$  then
10:     $k \leftarrow \text{LEFT-ROTATE}(k)$ 
11:   else
12:     $k \leftarrow \text{RIGHT-LEFT-ROTATE}(k)$ 
13:   end if
14: end if
15: if  $k.parent \neq \text{NIL}$  then
16:    $\text{AVL-BALANCE}(k.parent)$ 
17: else
18:    $T.root \leftarrow k$ 
19: end if
```

AVL-Bäume

Einfügen in einen AVL-Baum

Zunächst wird der neue Knoten wie in einen binären Suchbaum als neues Blatt eingefügt.

Anschließend erfolgt das Balancieren vom Elternknoten des eingefügten Knotens bis zur Wurzel.

Das Balancieren geschieht mit $O(\log n)$.

Damit ist das Einfügen auch im Worst Case mit $O(\log n)$ durchführbar.

AVL-Bäume

Löschen in einen AVL-Baum

Zunächst wird der zu löschende Knoten wie in einen binären Suchbaum identifiziert und entfernt.

Anschließend erfolgt das Balancieren vom Elternknoten des gelöschten Knotens bis zur Wurzel.

Das Balancieren geschieht mit $O(\log n)$.

Damit ist das Löschen auch im Worst Case mit $O(\log n)$ durchführbar.

Ziele der heutigen Lerneinheit

Sie verstehen...

- warum B-Bäume für externe Speicher entwickelt wurden und welche Probleme sie lösen
- wie die Ordnung eines B-Baums die Knotenstruktur bestimmt
- wie die Balancierung des Baums beim Einfügen (Splitten) und Löschen (Verschmelzen/Verschieben) aufrechterhalten wird

Sie können...

- die Suche in einem B-Baum Schritt für Schritt nachvollziehen und die Effizienz in O-Notation angeben
- einen Schlüssel korrekt in einen B-Baum einfügen, inklusive dem Aufteilen voller Knoten
- einen Schlüssel aus einem B-Baum löschen und dabei Unterlauf-Situationen durch Verschieben oder Verschmelzen von Knoten behandeln

B-Bäume

B-Bäume

Speicherstruktur für externe Speicher

- Die bisher betrachteten Bäume gehen davon aus, dass die gesamte Datenmenge in den Hauptspeicher passt
- Zwar schafft eine virtuelle Speicherverwaltung die Illusion eines nahezu unbegrenzten Hauptspeichers, allerdings auf Kosten von häufigem Ein- und Auslagern der referenzierten Speicherseiten
- Der Zugriff auf externe Speicher ist um Größenordnungen langsamer als der Zugriff auf den Hauptspeicher. Daher wird i.d.R. immer eine Seite geladen bzw. gespeichert
- Ziel muss es daher bei großen Datenmengen – neben der Effizienz der Algorithmen – mit wenigen Zugriffen auf externe Speicher auszukommen

B-Bäume

Speicherstruktur für externe Speicher

- Bei einem B-Baum entspricht ein Knoten einer „Seite“ auf dem externen Speicher
- Daher sind Knoten eines B-Baums größer und enthalten mehr als einen Schlüsselwert
- Die Ordnung eines B-Baums legt die Größe der Knoten fest. Ein B-Baum der Ordnung m
 - besitzt Knoten mit **maximal** $2m - 1$ Schlüsselwerten
 - besitzt (mit Ausnahme der Wurzel) Knoten mit **mindestens** $m - 1$ Schlüsselwerten
- Dennoch ist ein B-Baum ein Suchbaum, in dem durch Schlüsselvergleiche ein Abstieg im Baum gesteuert wird
- Entwickelt wurde der B-Baum von R. Bayer und E. McCreight
- **B** steht für balanciert, breit, buschig, Bayer, nicht für binär

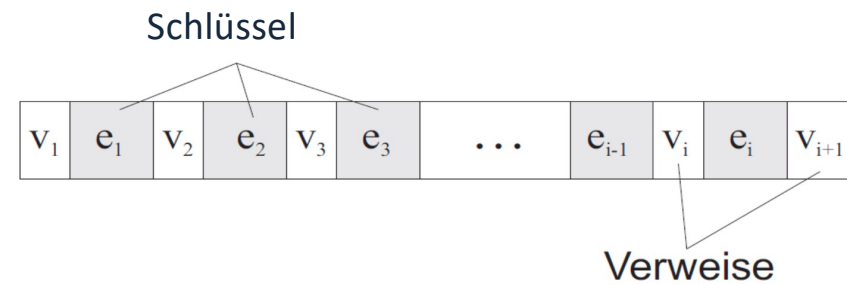
B-Bäume

Aufbau eines Knotens

Die Schlüsselwerte e innerhalb eines Knotens sind sortiert.

Zwischen den i Schlüsselwerten gibt es Verweise auf $i+1$ Unterbäume.

Alle Schlüssel im linken Unterbaum zu einem Schlüssel e_j sind kleiner und alle Schlüssel im rechten Unterbaum zu einem Schlüssel e_j sind größer als e_j .



Die Datenstruktur für einen Knoten:

- n : Aktuelle Anzahl von Schlüsseln im Knoten
- $keys$: Array/Vektor mit den Schlüsseln des Knoten
- $refs$: Array/Vektor mit den Verweisen des Knoten
- $isLeaf$: zeigt an, ob es ein Blattknoten ist

B-Bäume

Aufbau des Baums

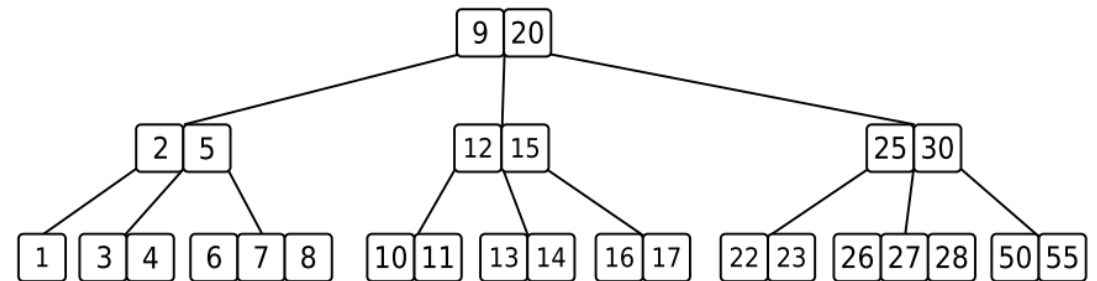
Alle Blattknoten des B-Baums befinden sich in gleicher Tiefe

Die Tiefe ist gleich der Höhe h des Baums

Bei einem B-Baum der Ordnung m gilt

$$h \leq \log_m \frac{n+1}{2}$$

Beispiel für einen Baum der Ordnung 2:



B-Bäume

Anlegen eines leeren Knotens

Beim Anlegen eines leeren Knotens werden die Attribute gemäß des gezeigten Pseudocodes initialisiert.

MAKE-NODE(m)

```
1:  $n \leftarrow 0$   
2:  $keys \leftarrow \mathbf{new\ Array}[2m - 1]$   $\triangleright [0 \dots 2m - 2]$   
3:  $refs \leftarrow \mathbf{new\ Array}[2m]$   $\triangleright [0 \dots 2m - 1]$   
4:  $isLeaf \leftarrow \mathbf{true}$ 
```

B-Bäume

Suche in B-Bäumen

Das Suchen in B-Bäumen ist eine Kombination von

- Verfolgen von Verweisen (wie beim binären Suchbaum)
- Suche in einem sortierten Array

BTREE-SEARCH(*start*, *key*)

```
1:  $i \leftarrow 0$ 
2: while  $i < start.n$  and  $key > start.keys[i]$  do
3:    $i \leftarrow i + 1$ 
4: end while
5: if  $i < start.n$  and  $key = start.keys[i]$  then
6:   return start
7: end if
8: if start.isLeaf then
9:   return NIL
10: end if
11: return BTREE-SEARCH(start.refs[i], key)
```

B-Bäume

Suche in B-Bäumen

Effizienzbetrachtung

Die Anzahl der Zugriffe auf das externe Medium ist gleich der Anzahl der besuchten Knoten:

$$O(\log_m n)$$

Verwendet man eine einfache lineare Suche in den Knoten, so ergibt sich ein Gesamtaufwand von

$$O(\log_m n \cdot m)$$

BTREE-SEARCH(*start*, *key*)

```
1:  $i \leftarrow 0$ 
2: while  $i < start.n$  and  $key > start.keys[i]$  do
3:    $i \leftarrow i + 1$ 
4: end while
5: if  $i < start.n$  and  $key = start.keys[i]$  then
6:   return start
7: end if
8: if start.isLeaf then
9:   return NIL
10: end if
11: return BTREE-SEARCH(start.refs[i], key)
```

B-Bäume

Aufspalten eines B-Baum-Knotens

Beim Einfügen eines neuen Schlüssels in einen B-Baum kann es zu einem Überlauf in einem Knoten kommen, wenn dort bereits $2m-1$ Schlüssel gespeichert sind.

In diesem Fall wird der Knoten aufgespaltet; die Tiefe des Baums ändert sich dadurch nicht.

Im Elternknoten wird ein zusätzlicher Schlüssel eingefügt, damit ein weiterer Verweis auf den neuen Knoten möglich ist.

B-Bäume

Aufspalten eines B-Baum-Knotens

Ausgehend von einem *parent*-Knoten spaltet der nebenstehende Algorithmus den *i*. Kindknoten in einem B-Baum der Ordnung *m* auf.

Voraussetzungen:

- Der Kindknoten besitzt vorher mindestens $2 \cdot (m - 1) + 1 = 2m - 1$ Schlüssel (d.h. er ist voll), damit in den Splittknoten die Mindestanzahl erreicht wird und ein weiterer Schlüssel in den Elternknoten wandern kann
- Der Elternknoten ist noch nicht so voll, dass kein weiterer Schlüssel aufgenommen werden kann

BTREE-SPLIT-CHILD(*parent*, *i*, *m*)

```

1: child  $\leftarrow$  parent.refs[i]
2: h  $\leftarrow$  MAKE-NODE(m)
3: h.isLeaf  $\leftarrow$  child.isLeaf
4: h.n  $\leftarrow$  m - 1
5: for j  $\leftarrow$  0 to m - 2 do
6:   h.keys[j]  $\leftarrow$  child.keys[j + m]
7: end for
8: if not h.isLeaf then
9:   for j  $\leftarrow$  0 to m - 1 do
10:    h.refs[j]  $\leftarrow$  child.refs[j + m]
11:   end for
12:   for j  $\leftarrow$  m to child.n do
13:    child.refs[j]  $\leftarrow$  NIL
14:   end for
15: end if
16: child.n  $\leftarrow$  m - 1
17: for j  $\leftarrow$  parent.n downto i + 1 do
18:   parent.refs[j + 1]  $\leftarrow$  parent.refs[j]
19:   parent.keys[j]  $\leftarrow$  parent.keys[j - 1]
20: end for
21: parent.refs[i + 1]  $\leftarrow$  h
22: parent.keys[i]  $\leftarrow$  child.keys[m - 1]
23: parent.n  $\leftarrow$  parent.n + 1

```

B-Bäume

Aufspalten eines B-Baum-Knotens

Effizienzbetrachtung

Es wird nur eine konstante Anzahl von Knoten bearbeitet ($parent, child, h$), daher ist der Aufwand für den Zugriff auf externen Speicher

$$O(1)$$

Alle Schleifen durch m begrenzt, somit insgesamt

$$O(m)$$

BTREE-SPLIT-CHILD($parent, i, m$)

```

1:  $child \leftarrow parent.refs[i]$ 
2:  $h \leftarrow \text{MAKE-NODE}(m)$ 
3:  $h.isLeaf \leftarrow child.isLeaf$ 
4:  $h.n \leftarrow m - 1$ 
5: for  $j \leftarrow 0$  to  $m - 2$  do
6:    $h.keys[j] \leftarrow child.keys[j + m]$ 
7: end for
8: if not  $h.isLeaf$  then
9:   for  $j \leftarrow 0$  to  $m - 1$  do
10:     $h.refs[j] \leftarrow child.refs[j + m]$ 
11:   end for
12:   for  $j \leftarrow m$  to  $child.n$  do
13:     $child.refs[j] \leftarrow \text{NIL}$ 
14:   end for
15: end if
16:  $child.n \leftarrow m - 1$ 
17: for  $j \leftarrow parent.n$  downto  $i + 1$  do
18:    $parent.refs[j + 1] \leftarrow parent.refs[j]$ 
19:    $parent.keys[j] \leftarrow parent.keys[j - 1]$ 
20: end for
21:  $parent.refs[i + 1] \leftarrow h$ 
22:  $parent.keys[i] \leftarrow child.keys[m - 1]$ 
23:  $parent.n \leftarrow parent.n + 1$ 

```

B-Bäume

Einfügen in einen B-Baum

Der neue Schlüssel wird grundsätzlich in ein Blatt eingetragen.

Beim rekursiven Abstieg zum betroffenen Blatt im Baum wird geprüft, ob in einen vollen Knoten abgestiegen werden soll. Falls ja, wird dieser gespalten.

Von oben kommend wird damit sichergestellt, dass

- der von der Spaltung nach oben durchgereichte Schlüssel nicht auf einen vollen Knoten trifft
- das Blatt noch einen Schlüssel aufnehmen kann

B-Bäume

Einfügen in einen B-Baum

Zunächst muss der Sonderfall betrachtet werden, dass die Wurzel bereits voll ist.

Es erfolgt dann ein Splitt und der Baum wächst nach oben.

Somit bleibt er stets balanciert.

BTREE-INSERT(T, key)

```
1:  $r \leftarrow T.root$ 
2: if  $r.n = 2m - 1$  then
3:    $h \leftarrow \text{MAKE-NODE}(T.m)$ 
4:    $T.root \leftarrow h$ 
5:    $h.isLeaf \leftarrow \text{false}$ 
6:    $h.refs[0] \leftarrow r$ 
7:    $h.n \leftarrow 0$ 
8:   BTREE-SPLIT-CHILD( $h, 0, T.m$ )
9:   BTREE-INSERT-IN-NODE( $h, key, T.m$ )
10: else
11:   BTREE-INSERT-IN-NODE( $r, key, T.m$ )
12: end if
```

B-Bäume

Einfügen in einen B-Baum

Effizienzbetrachtung

Anzahl der Zugriffe auf das externe Medium:

$$O(\log_m n)$$

In jedem Knoten max. m Vergleiche, somit insgesamt:

$$O(\log_m n \cdot m)$$

BTREE-INSERT-IN-NODE(*start*, *key*, *m*)

```

1: i ← start.n
2: if start.isLeaf then
3:   while i ≥ 1 and key < start.keys[i − 1] do
4:     start.keys[i] ← start.keys[i − 1]
5:     i ← i − 1
6:   end while
7:   start.keys[i] ← key
8:   start.n ← start.n + 1
9: else
10:  j ← 0
11:  while j < start.n and key > start.keys[j] do
12:    j ← j + 1
13:  end while
14:  if start.refs[j].n = 2m − 1 then
15:    BTREE-SPLIT-CHILD(start, j, m)
16:    if key > start.keys[j] then
17:      j ← j + 1
18:    end if
19:  end if
20:  BTREE-INSERT-IN-NODE(start.refs[j], key, m)
21: end if

```

B-Bäume

Löschen aus einem B-Baum

Der zu löschende Schlüssel muss zunächst gefunden werden; er kann sich in einem inneren Knoten oder einem Blatt befinden.

Mit dem Entfernen des Schlüssels aus einem inneren Knoten verschwindet auch eine Verzeigerung zu einem möglichen Unterbaum.

Beim Entfernen des Schlüssels kann ein Unterlauf auftreten, wenn die Anzahl der Schlüssel im Knoten den Wert $m-1$ unterschreitet.

B-Bäume

Löschen aus einem B-Baum

Lösungsansatz

Analog zum Einfügen, wo beim rekursiven Abstieg sichergestellt wurde, dass die besuchten Knoten noch Schlüssel aufnehmen können (und ggf. gesplittet wurde)

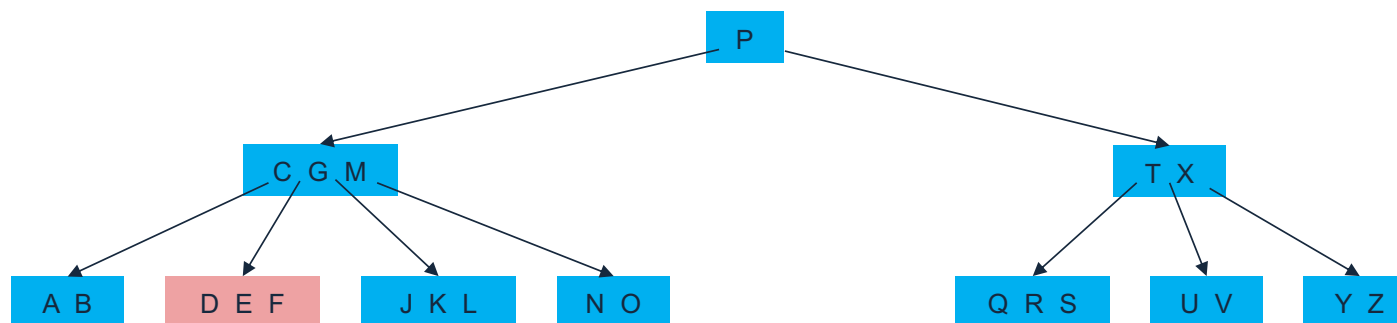
Jetzt wird beim rekursiven Abstieg sichergestellt, dass die besuchten Knoten noch Schlüssel abgeben können. Falls das nicht möglich ist, wird

- vom linken oder rechten Geschwisterknoten ein Schlüssel verschoben, sofern einer der Geschwister Schlüssel abgeben kann
- der Knoten mit einem Geschwister verschmolzen

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (F) aus einem Blatt

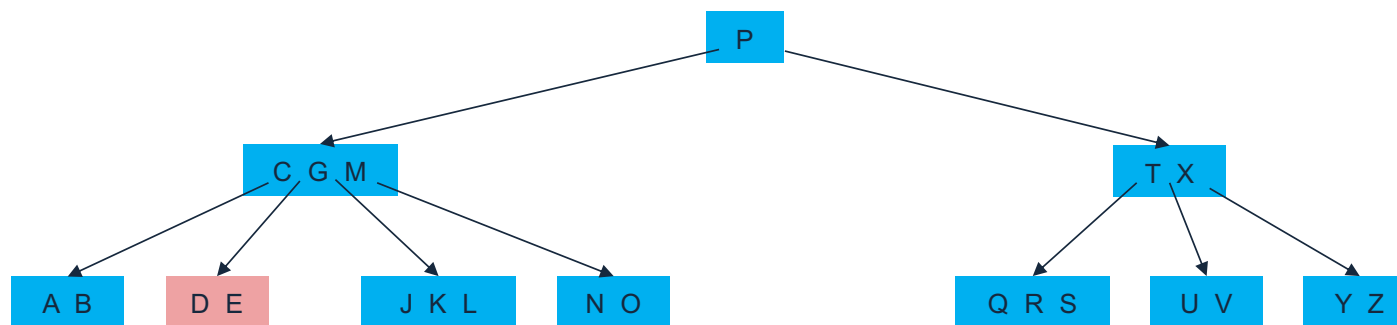


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m - 1 = 2$)
- Keine Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (F) aus einem Blatt

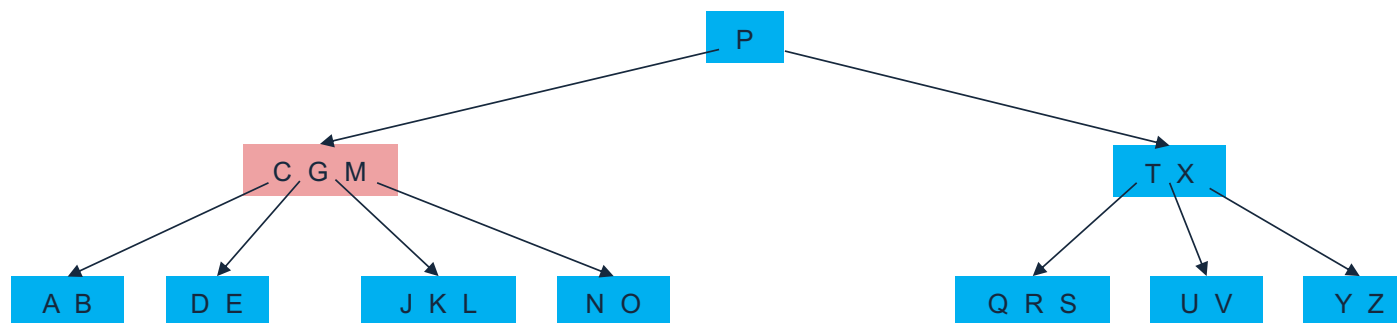


- Der Schlüssel kann einfach entfernt werden

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (M) aus einem inneren Knoten

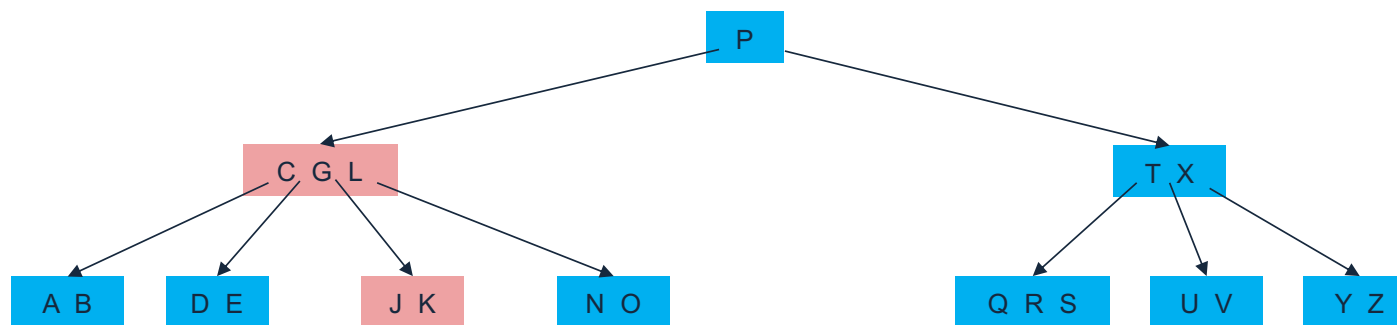


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m - 1 = 2$)
- Es gibt Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (M) aus einem inneren Knoten

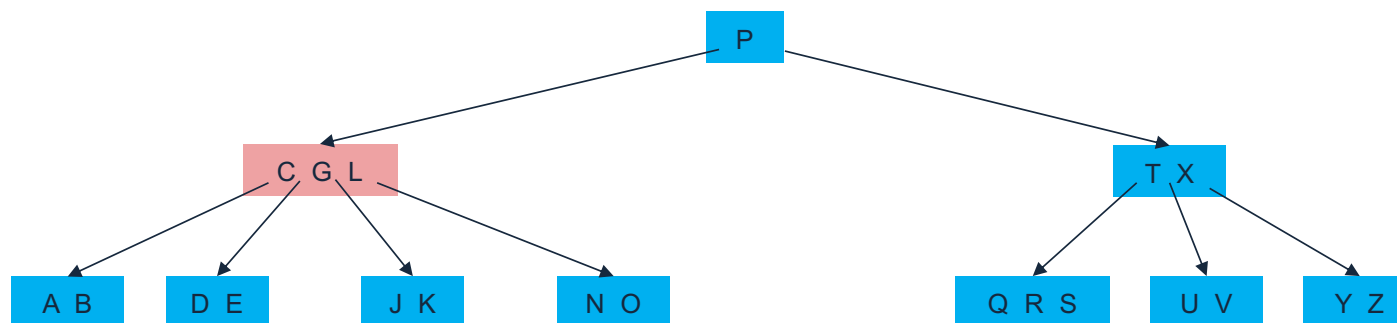


- Der Vorgänger von M rückt nach oben und nimmt den Platz ein
- Der Vorgänger L hat keine Unterbäume und kann verschoben werden

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (G) aus einem inneren Knoten

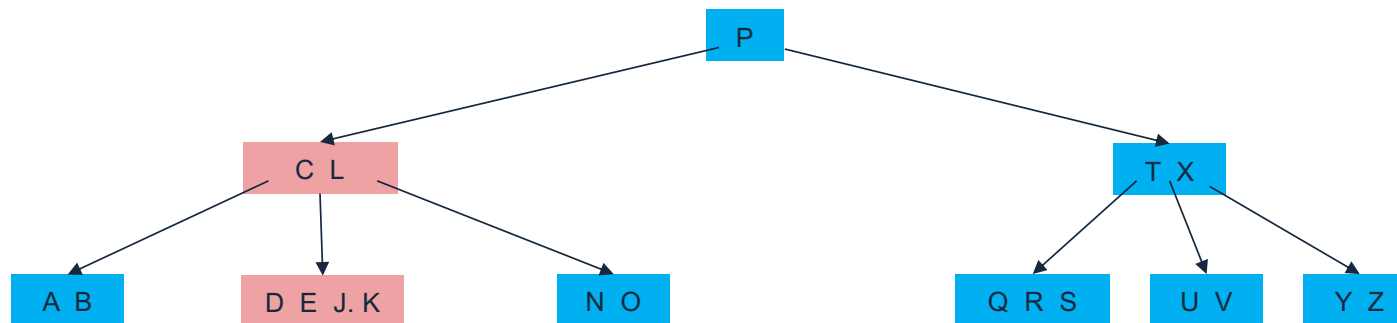


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m-1=2$)
- Es gibt Unterbäume
- Vorgänger kann nicht aufsteigen, weil Mindestgröße unterschritten würde

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (G) aus einem inneren Knoten

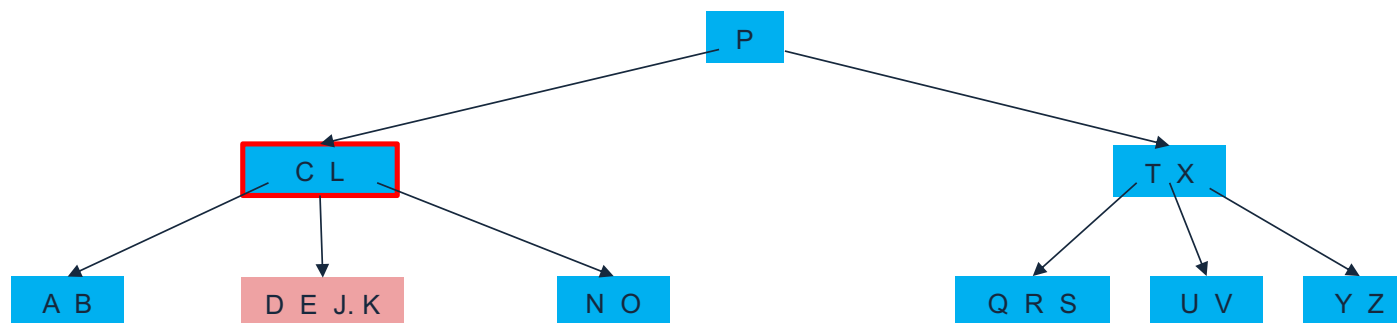


– Verschmelzen der beiden Nachfolger

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (D) aus einem Blatt

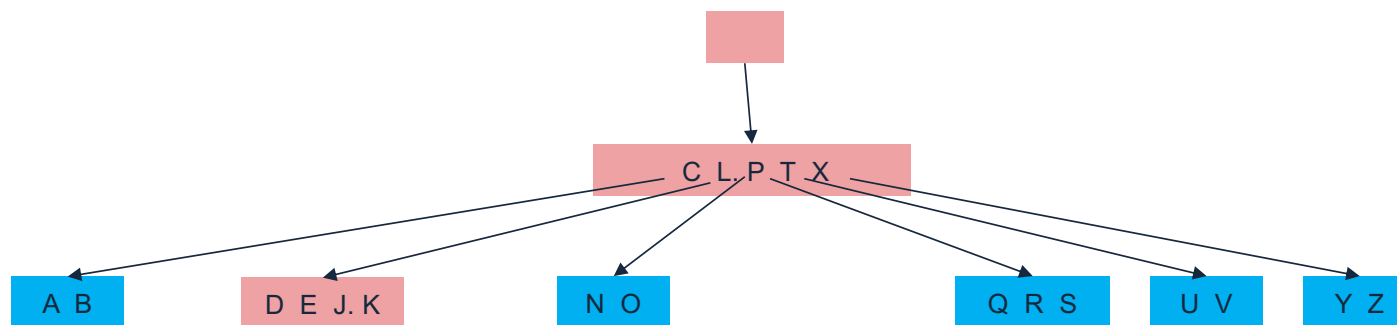


- Der Knoten „C L“ kann keinen Schlüssel mehr abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m-1=2$)
- Es gibt keine Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (D) aus einem Blatt

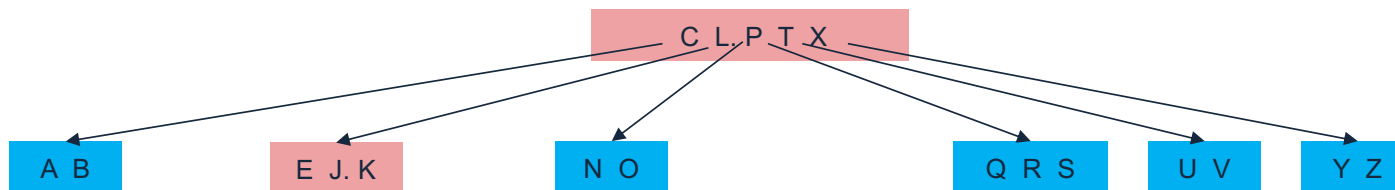


- Verschmelzen von „C L“ mit dem Geschwister „T X“ unter Einbeziehung des ordnenden Schlüssels aus dem Elternknoten

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (D) aus einem Blatt

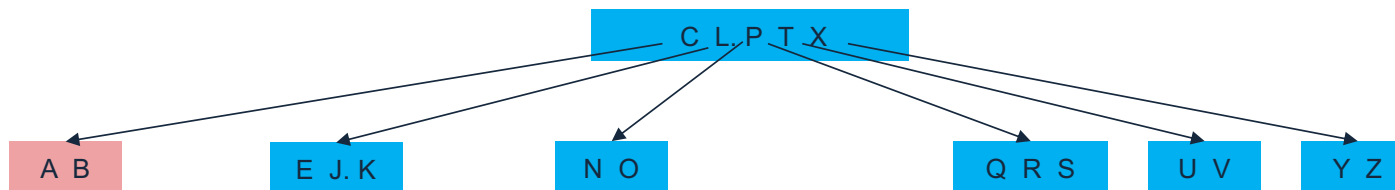


- Danach einfaches Löschen eines Schlüssels aus dem Blatt
- Außerdem kann die leere Wurzel entfernt werden; die Höhe des Baums reduziert sich

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (B) aus einem Blatt

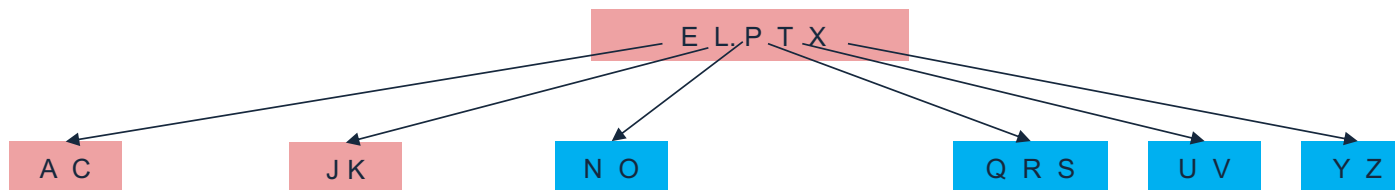


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Minimalen Knotengröße ($m-1=2$) wird unterschritten
- Keine Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (B) aus einem Blatt



- Verschieben eines Knotens aus dem Geschwisterknoten über den Elternknoten

B-Bäume

Relevanz von B-Bäumen

B-Bäume werden in vielen Dateisystemen und Datenbanken eingesetzt, wie z .B.:

- Windows: NTFS
- Mac: HFS, HFS+, APFS
- Linux: ReiserFS, XFS, Ext3FS, JFS
- Datenbanken: Oracle, DB2, MS SQL, INGRES, ...

Ziele der heutigen Lerneinheit

Sie verstehen...

- ... warum Hashtabellen Suche in erwartetem $O(1)$ ermöglichen.
- ... was eine Kollision ist und warum sie bei realen Datenmengen prinzipiell unvermeidbar ist.
- ... wie Kollisionen durch Verkettung und offene Adressierung gelöst werden.

Sie können...

- ... Hashfunktionen nach der Divisions-Rest-Methode entwerfen.
- ... eine Hashfunktion anhand der Kriterien Surjektivität, Gleichverteilung, Effizienz und Unstetigkeit bewerten.
- ... Einfüge-, Such- und Löschooperationen in Hashtabellen mit Verkettung und offener Adressierung schrittweise durchführen.

Hashtabellen

Hashtabellen

Dynamische Mengen

- Wir haben mit **Arrays**, **verketteten Listen** und **Bäumen** bereits Datenstrukturen kennengelernt, die die Wörterbuch-Operationen INSERT, SEARCH und DELETE unterstützen und darüber hinaus auch das Sortieren ermöglichen.
- Kann man auf die sortierte Ausgabe der gesamten Menge verzichten, so stellt eine Hashtabelle eine effiziente Implementierungsvariante dar.
- Insbesondere kann SEARCH im besten Fall in $O(1)$ ausgeführt werden.

Hashtabellen

Prinzipielles Verfahren

- Die Speicherung der Objekte vom Typ T erfolgt in einem Array der Länge m . Dieses Array wird auch **Hashtabelle** genannt.
- Die Festlegung, an welcher Stelle in der Hashtabelle ein Objekt abgelegt wird, ist durch eine **Hashfunktion**

$$h: T \rightarrow \{0, \dots, m - 1\}$$

festgelegt.

Übung

Hashtabellen

Kollision

- Ist die Position, die von der **Hashfunktion** für ein neues Element berechnet wird, bereits belegt, so nennt man dies **Kollision**.
- Kollisionen sind grundsätzlich nicht vermeidbar, denn:
Die Mächtigkeit von T (Anzahl der möglichen zu speichernden Objekte) ist i.d.R. größer als m . Daher gilt:

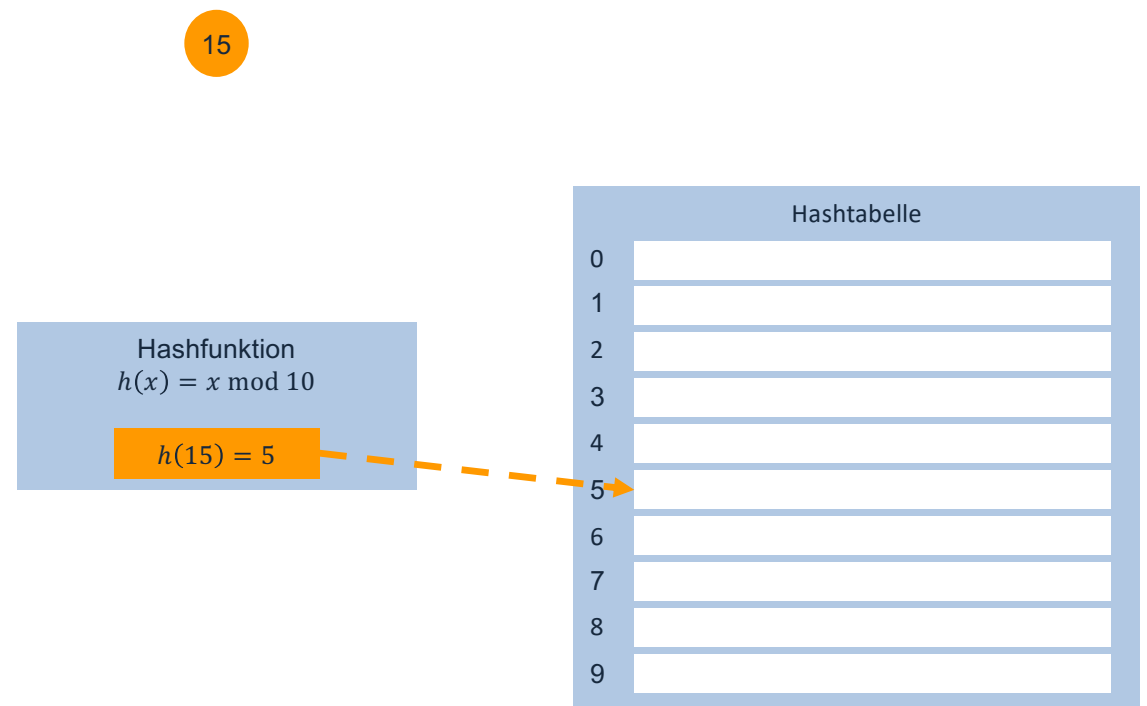
$$\exists o_1, o_2 \in T: o_1 \neq o_2 \wedge h(o_1) = h(o_2)$$

Hashtabellen

Beispiel

Gespeichert werden sollen positive ganze Zahlen in einer Hashtabelle der Größe $m=10$. Die Hashfunktion ist definiert als

$$h(x) = x \bmod 10$$

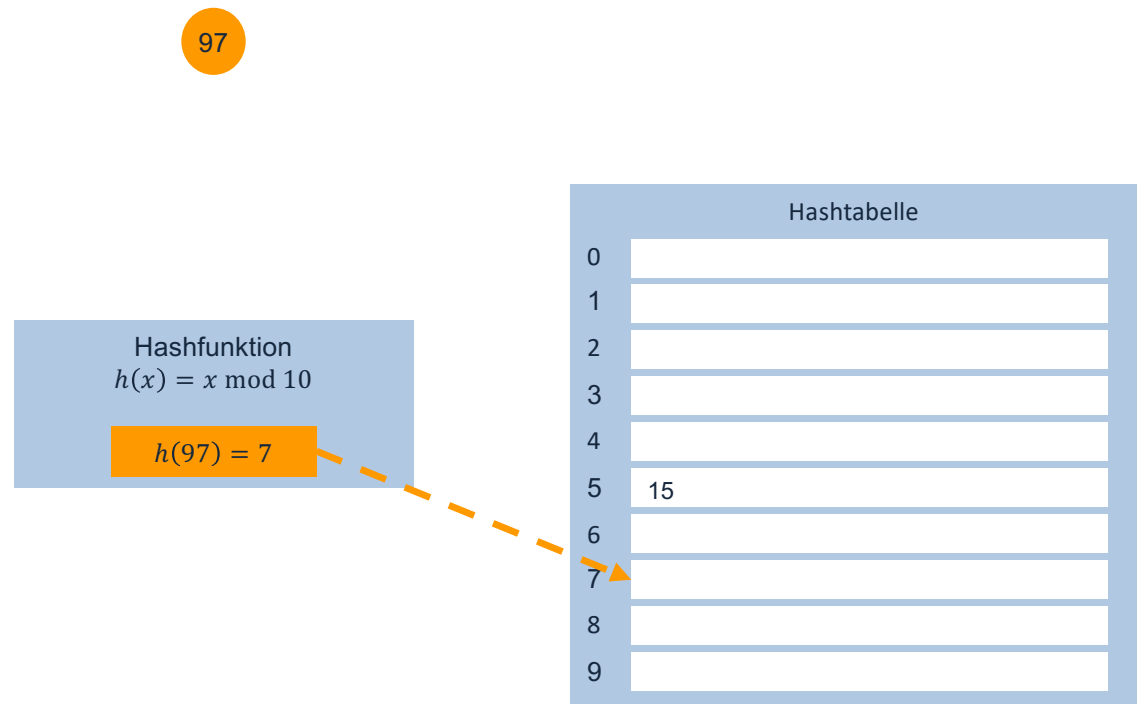


Hashtabellen

Beispiel

Gespeichert werden sollen positive ganze Zahlen in einer Hashtabelle der Größe $m=10$. Die Hashfunktion ist definiert als

$$h(x) = x \bmod 10$$

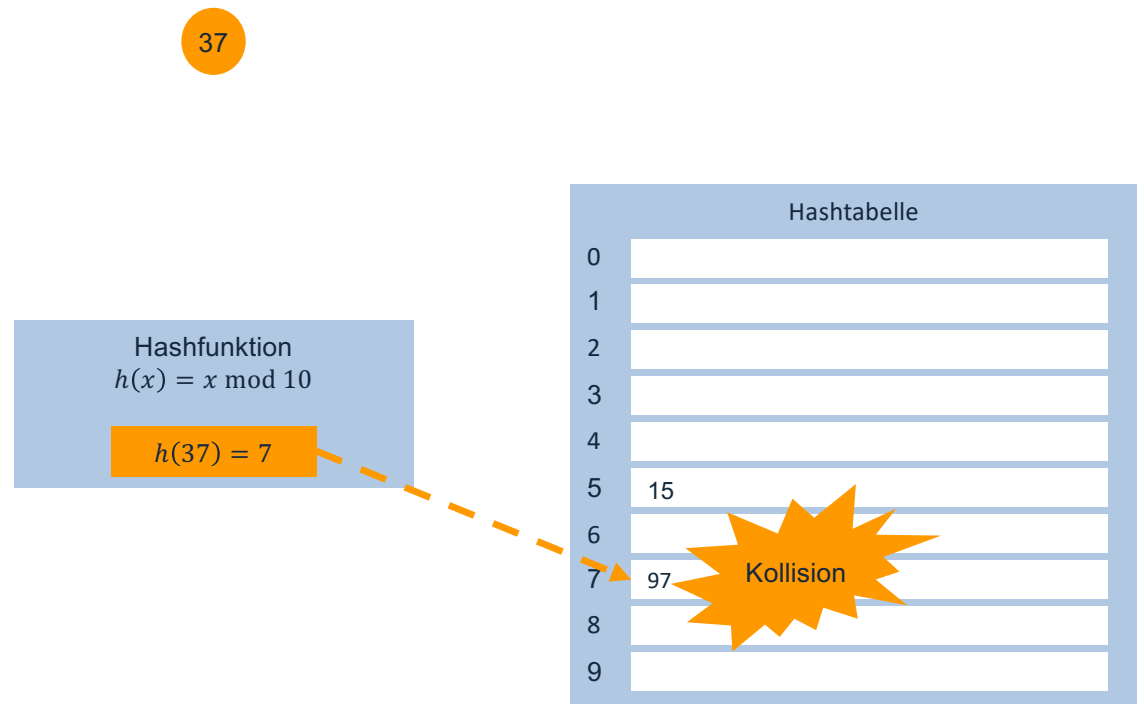


Hashtabellen

Beispiel

Gespeichert werden sollen positive ganze Zahlen in einer Hashtabelle der Größe $m=10$. Die Hashfunktion ist definiert als

$$h(x) = x \bmod 10$$



Wahl der Hashfunktion

Hashtabellen

Wahl der Hashfunktion

- Die Hashfunktion h sollte alle Plätze der Hashtabelle erreichen. (Surjektivität)
- Alle Plätze der Hashtabelle sollten mit der gleichen Wahrscheinlichkeit erreicht werden. (Gleichverteilung)
- Die Hashfunktion sollte effizient berechenbar sein. (ideal: $O(1)$)
- Ähnliche Objekte sollten nicht zu ähnlichen Hashwerten führen. (Unstetigkeit)

Hashtabellen

Wahl der Hashfunktion – Divisions-Rest-Methode

- Sind die zu speichernden Objekte positive ganze Zahlen, bietet sich die Divisions-Rest-Methode an:

$$h(x) = x \bmod m$$

- Entscheidend ist eine gute Wahl von m :

Wählt man für m eine gerade Zahl, dann ist $h(x)$ gerade, wenn x gerade ist, bzw. ungerade, wenn x ungerade ist → schlecht

- Bewährt hat sich die Wahl einer Primzahl für m

Hashtabellen

Wahl der Hashfunktion – Divisions-Rest-Methode

Zeichenketten s der Länge n können durch Aufsummieren ihrer ASCII bzw. UNICODE-Werte zu ganzen Zahlen werden:

$$h(s) = \left(\sum_{i=0}^{n-1} \text{ascii}(s[i]) \right) \bmod m$$

Probleme

- Anagramme liefern den gleichen Hashwert
$$h(\text{„logarithmus“}) = h(\text{„algorithmus“})$$
- Bei kurzen Zeichenketten werden nur relativ kleine Hashwerte erreicht

Hashtabellen

Wahl der Hashfunktion – Divisions-Rest-Methode

Verbesserung für Zeichenketten

$$h(s) = \left(\sum_{i=0}^{n-1} \text{ascii}(s[i]) \cdot 256^{n-1-i} \right) \bmod m$$

Beispiel:

$$h(\text{„test“}) = (\text{ascii}('t') \cdot 256^3 + \text{ascii}('e') \cdot 256^2 + \text{ascii}('s') \cdot 256^1 + \text{ascii}('t') \cdot 256^0) \bmod m$$

Effiziente Berechnung nach dem Horner-Schema:

$$h(\text{„test“}) = ((((\text{ascii}('t') \cdot 256 + \text{ascii}('e')) \cdot 256 + \text{ascii}('s')) \cdot 256 + \text{ascii}('t')) \cdot 256) \bmod m$$

Hashtabellen

Wahl der Hashfunktion – Multiplikative Methode

Sind die zu speichernden Objekte wieder positive ganze Zahlen, so wird das Objekt mit einer irrationalen Zahl A multipliziert. Anschließend wird der gebrochene Anteil des Ergebnisses mit m multipliziert.

$$h(x) = \lfloor (x \cdot A - \lfloor x \cdot A \rfloor) \cdot m \rfloor$$

Für viele typische Eingabedaten hat sich der „goldene Schnitt“ für A bewährt:

$$A = \frac{\sqrt{5} - 1}{2} \approx 0,6180339887 \dots$$

Umgang mit Kollisionen

Hashtabellen

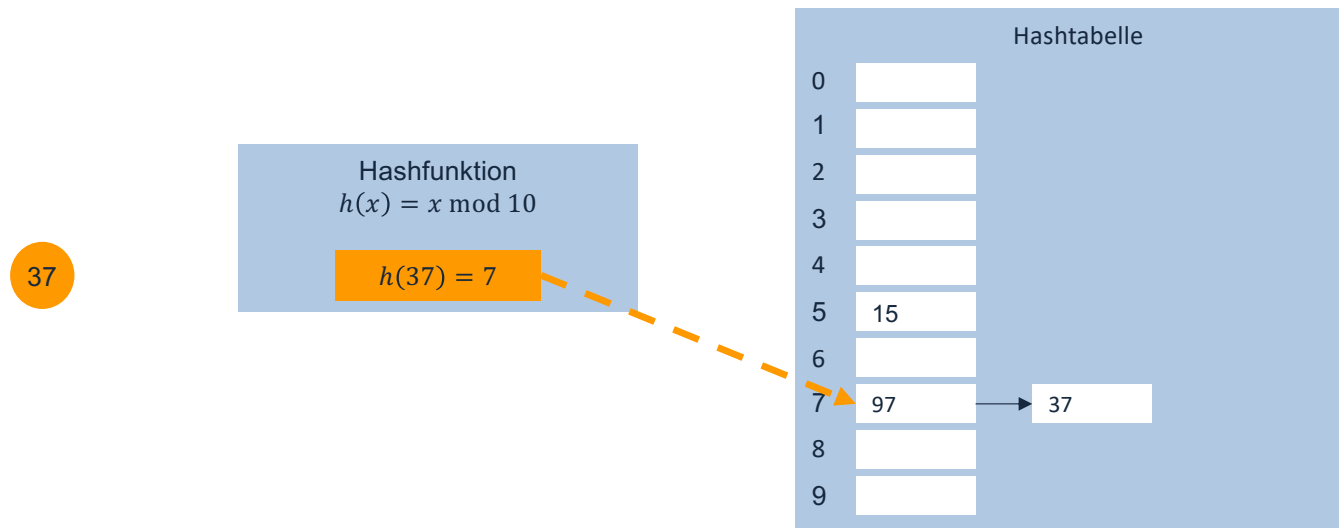
Umgang mit Kollisionen

- Kollisionsauflösung mit Verkettung
- Kollisionsauflösung durch offene Adressierung



Hashtabellen

Kollisionsauflösung mit Verkettung



- Die Hashtabelle enthält nicht die Objekte, sondern verkettete Listen der Objekte
- Die Listen sind unsortiert

Hashtabellen

Kollisionsauflösung mit Verkettung

Einfügen in eine Hashtabelle mit Verkettung ist in $O(1)$ möglich, da das Element an den Anfang der Liste gehängt werden kann.

Bei der **Suche** nach einem Objekt ist mit einer mittleren Länge der Listen von n/m (auch Belegungsfaktor α genannt) zu rechnen. Die durchschnittliche Zeitkomplexität zum Auffinden eines Elements in der Hashtabelle ist hier dann $O(1 + n/m)$

Das **Löschen** ist in konstanter Zeit möglich, wenn bereits das zu löschende Listenelement bekannt ist und die Liste doppelt verkettet ist.

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Die Hashfunktion liefert nicht eine Adresse, sondern eine Folge von Adressen, an denen das Objekt abgelegt werden kann.

$$h : T \times \{0, \dots, m-1\} \\ \rightarrow \{0, \dots, m-1\}$$

37

Hashfunktion
 $h(x, i) = (x + i) \bmod 10$

$$h(37, 0) = 7$$

Hashtabelle

0	
1	
2	
3	
4	
5	15
6	
7	97
8	
9	

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Die Hashfunktion liefert nicht eine Adresse, sondern eine Folge von Adressen, an denen das Objekt abgelegt werden kann.

$$h : T \times \{0, \dots, m-1\} \\ \rightarrow \{0, \dots, m-1\}$$

Hashfunktion
 $h(x, i) = (x + i) \bmod 10$

$h(37, 1) = 8$

Hashtabelle	
0	
1	
2	
3	
4	
5	15
6	
7	97
8	37
9	

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Bei Verwendung von

$$h(x, i) = (h'(x) + c_1 i + c_2 i^2) \bmod m$$

spricht man von ***quadratischem Sondieren***.

$h'(x)$ ist eine übliche Hashfunktion, c_1 und c_2 zwei Konstanten ungleich 0, die so gewählt werden, dass die Sondierungssequenz alle Slots erreicht.

- Quadratisches Sondieren vermeidet die (primäre) Clusterbildung besser als lineares Sondieren.
- Allerdings: Für zwei Schlüssel x_1 und x_2 mit $h'(x_1) = h'(x_2)$ ergibt sich auch hier die gleiche Sondierungssequenz (sekundäre Clusterbildung).

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Bei der Analyse des Laufzeitverhaltens wird von einer optimalen Hashfunktion ausgegangen. In diesem Fall ist mit höchstens

$$\frac{1}{1-\alpha} \text{ Sondierungen}$$

bei einem Zugriff zu rechnen.

Suchen, **Einfügen** und **Löschen** ist damit in $O(1)$ möglich, wenn α als konstant und hinreichend klein angesehen werden kann.

Beim Löschen eines Objekts kann der Slot nicht einfach auf **nil** gesetzt werden, weil dadurch ggf. eine spätere Suche unterbrochen wird. Stattdessen sollte je Slot ein Zustand „besetzt“, „noch nie besetzt“ und „wieder frei“ geführt werden.

Ziele der heutigen Lerneinheit

Sie verstehen...

- was ein Graph ist und wie gerichtete und ungerichtete Graphen sich unterscheiden
- wie BFS vorgeht und warum sie den kürzesten Pfad (in Kantenzahl) findet
- wie DFS vorgeht und was die Zeitmarken aussagen

Sie können...

- einen Graphen als Adjazenzliste oder Adjazenzmatrix darstellen
- BFS und DFS schrittweise auf einem gegebenen Graphen ausführen und die Knoten-/Kantenfärbung nachvollziehen
- eine topologische Sortierung mithilfe der DFS-Endzeiten bestimmen

Graphen

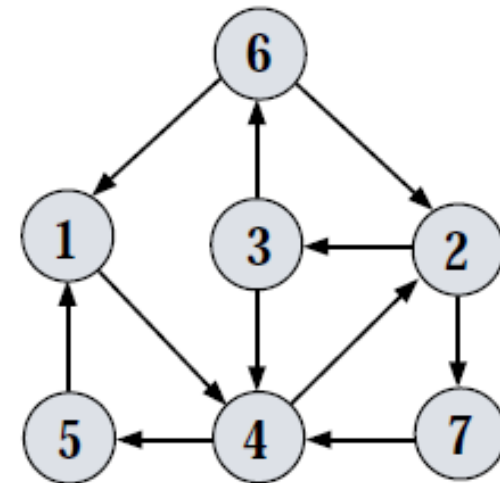
Graphen

Gerichtete Graphen

Ein gerichteter Graph $G=(V, E)$ besteht aus

- einer endlichen Menge von Knoten V (vertices)
- einer endlichen Menge E von geordneten Paaren aus V , den Kanten (edges)

Ist $(q, r) \in E$ eine Kante, so ist $q \in V$ der Anfangsknoten und $r \in V$ der Endknoten dieser Kante.



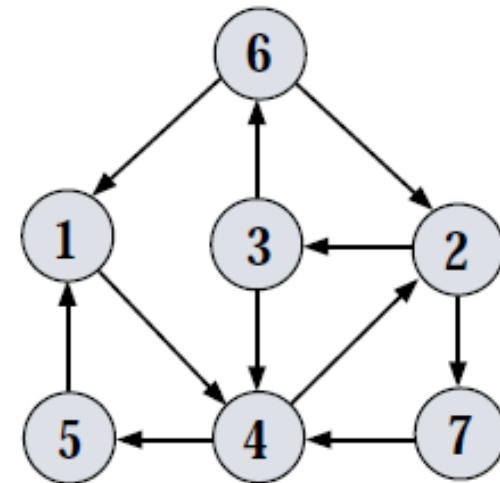
Graphen

Gerichtete Graphen

Im Beispiel:

$G=(V, E)$ mit

- $V=\{1, 2, 3, 4, 5, 6, 7\}$
- $E=\{(1,4), (2,3), (2,7), (3,4), (3,6), (4,2), (4,5), (5,1), (6,1), (6,2), (7,4)\}$

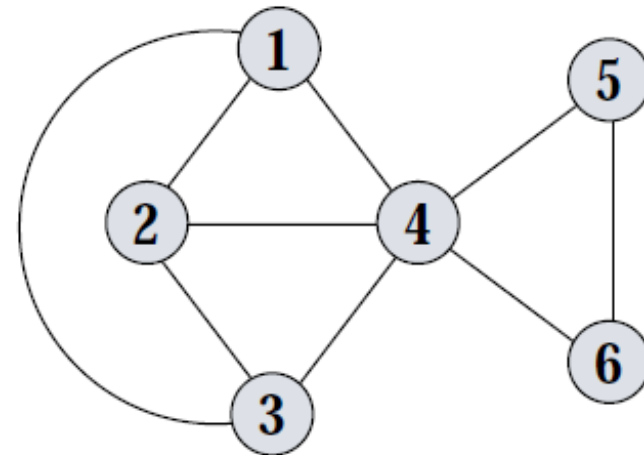


Graphen

Ungerichtete Graphen

Ein ungerichteter Graph $G=(V, E)$ besteht aus

- einer endlichen Menge von Knoten V (vertices)
- einer endlichen Menge E von ungeordneten Paaren aus V , den Kanten (edges)



Graphen

Wichtige Eigenschaften

- Ist $(q, r) \in E$ eine Kante eines Graphen G , so ist der Knoten r **adjazent** zum Knoten q . Ist G ungerichtet, dann ist auch q **adjazent** zum Knoten r
- Ein Graph G , in dem jeder Knoten mit jedem anderen Knoten adjazent ist, heißt **vollständiger Graph**.
- Ein Graph G heißt **dicht (dense)**, wenn er nahezu vollständig ist.
- Ein Graph G heißt **licht (sparse)**, wenn die Kantenanzahl viel kleiner als $|V|^2$ ist.

Graphen

Datenstrukturen

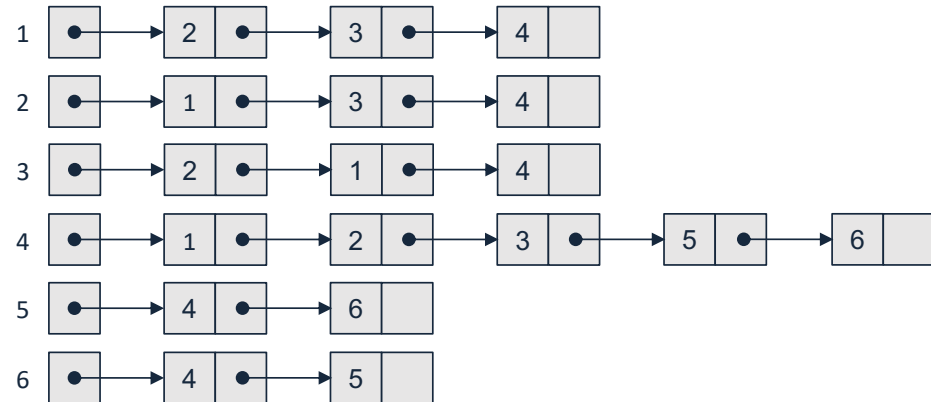
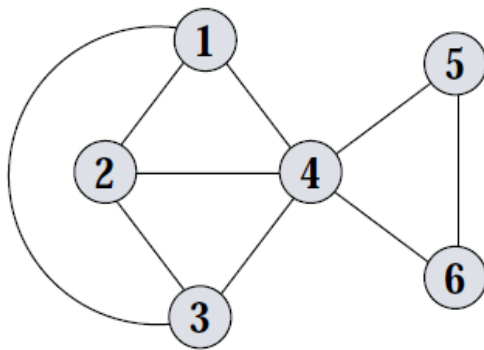
Für die Darstellung von Graphen $G = (V, E)$ gibt es zwei gängige Möglichkeiten:

- Bei lichten Graphen verwendet man gerne ***Adjazenzlisten***.
- Bei dichten Graphen verwendet man meist ***Adjazenzmatrizen***.

Graphen

Adjazenzlisten

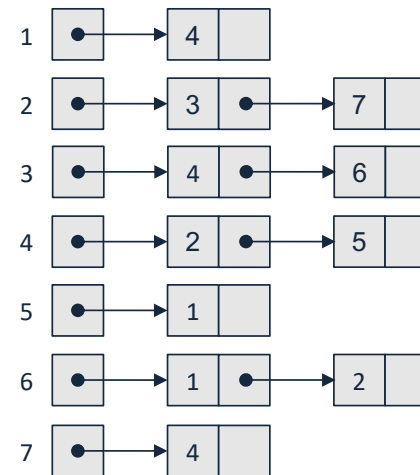
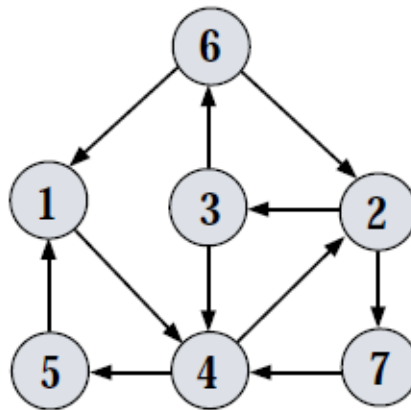
Die Adjazenzlisten-Darstellung eines **ungerichteten** Graphen $G = (V, E)$ besitzt für jeden Knoten $q \in V$ die Liste aller mit q verbundenen (adjazenten) Knoten.



Graphen

Adjazenzlisten

Die Adjazenzlisten-Darstellung eines **gerichteten** Graphen $G = (V, E)$ besitzt für jeden Knoten $q \in V$ die Liste aller direkten Nachfolger r von Knoten q , die also zu q adjazent sind.

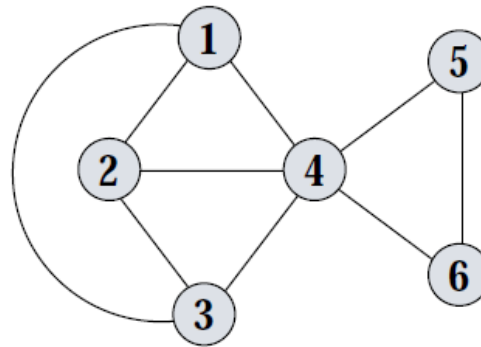


Graphen

Adjazenzmatrix

Die Adjazenzmatrix A eines Graphen $G = (V, E)$ ist eine $|V| \times |V|$ -Matrix mit den Elementen

$$a_{ij} = \begin{cases} 1 & \text{falls } j \text{ adjazent zu } i \\ 0 & \text{sonst} \end{cases}$$



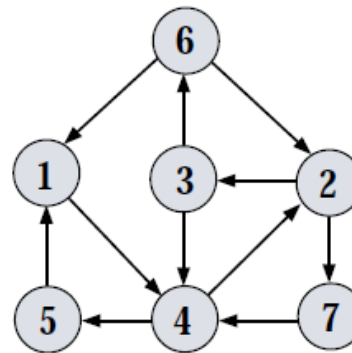
	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	1	1	0	0
3	1	1	0	1	0	0
4	1	1	1	0	1	1
5	0	0	0	1	0	1
6	0	0	0	1	1	0

Graphen

Adjazenzmatrix

Die Adjazenzmatrix A eines Graphen $G = (V, E)$ ist eine $|V| \times |V|$ -Matrix mit den Elementen

$$a_{ij} = \begin{cases} 1 & \text{falls } j \text{ adjazent zu } i \\ 0 & \text{sonst} \end{cases}$$



	1	2	3	4	5	6	7
1	0	0	0	1	0	0	0
2	0	0	1	0	0	0	1
3	0	0	0	1	0	1	0
4	0	1	0	0	1	0	0
5	1	0	0	0	0	0	0
6	1	1	0	0	0	0	0
7	0	0	0	1	0	0	0

Breitensuche

Breitensuche (breadth-first search - BFS)

Zielsetzung

Das Ziel der Breitensuche in einem Graphen ist es, ausgehend von einem Startknoten

- Alle **erreichbaren Knoten** zu identifizieren,
- für jeden erreichbaren Knoten die Distanz (Anzahl der beschrifteten Kanten) zum Start zu ermitteln (auch: **kürzester Pfad**) und
- diesen Pfad festzuhalten

Breitensuche (breadth-first search - BFS)

Vorgehensweise

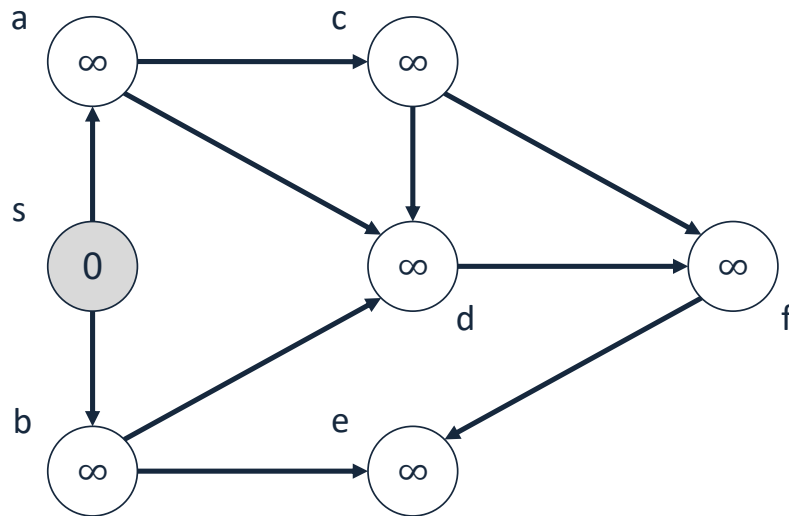
Die Knoten des Graphen werden in drei Kategorien eingeteilt:

- „Weiße“ Knoten sind noch nicht entdeckt
(am Anfang sind alle Knoten weiß)
- „Graue“ Knoten sind entdeckt, besitzen aber evtl. noch weiße Nachbarn
- „Schwarze“ Knoten sind vollständig bearbeitet und können nicht mehr zu unentdeckten Knoten führen

Lösungsidee: Die grauen Knoten werden in einer Queue verwaltet, die den Frontverlauf der Suche wiedergibt.

Breitensuche (breadth-first search - BFS)

Beispiel



Ausgangssituation:

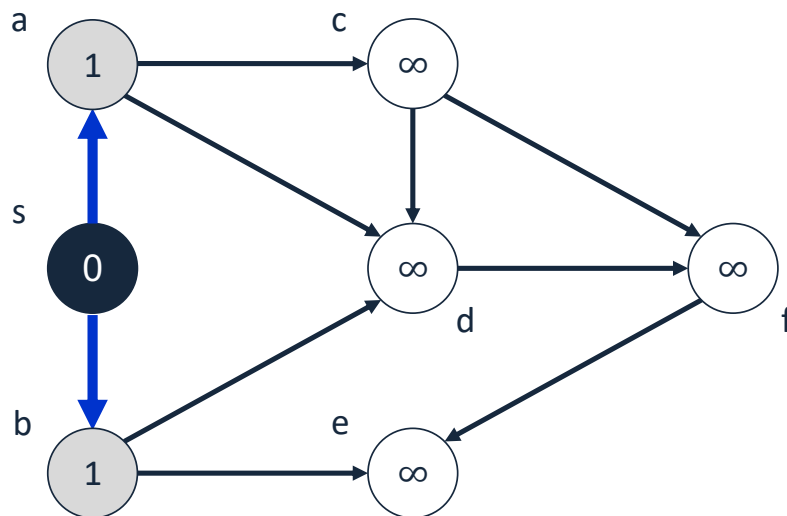
Nur der Startknoten ist grau, besitzt die Distanz 0 und steht in der Queue der zu bearbeitenden Knoten.

Alle anderen Knoten sind weiß und haben die Entfernung „unendlich“.

Queue: s

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

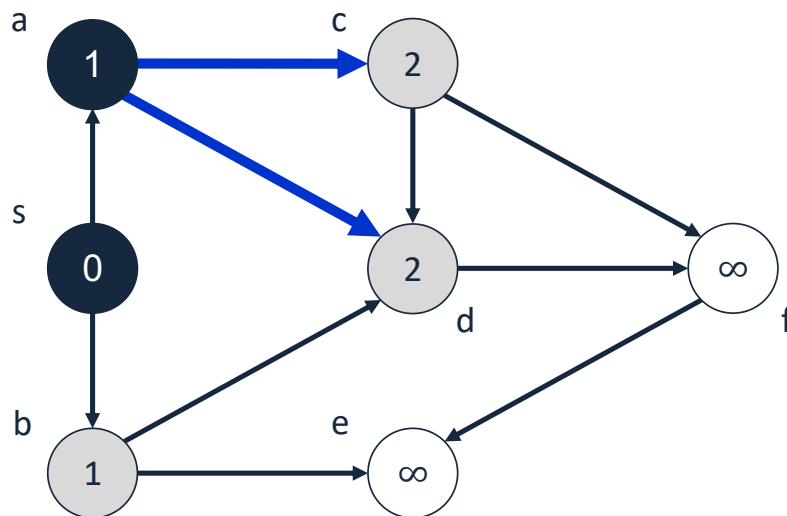
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

a	b
---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

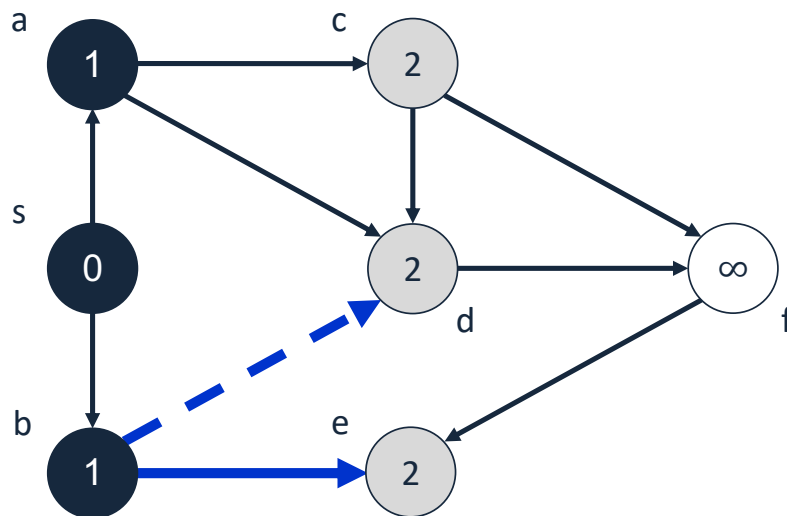
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

b	c	d
---	---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

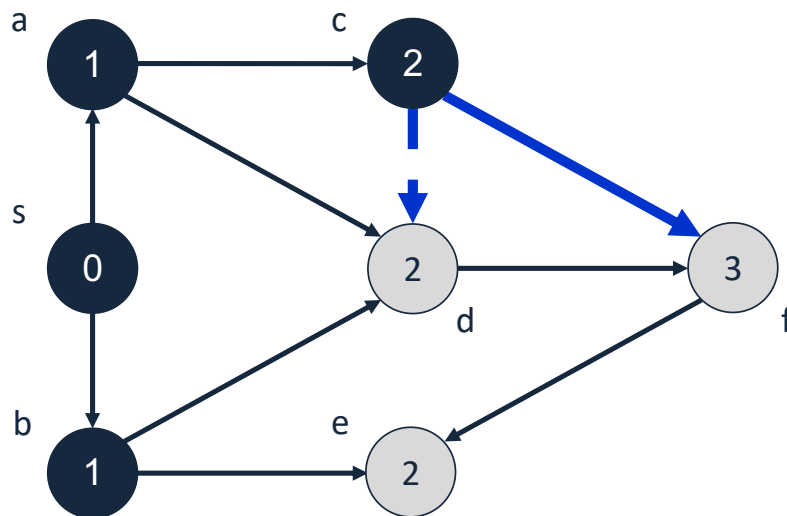
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

c	d	e
---	---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

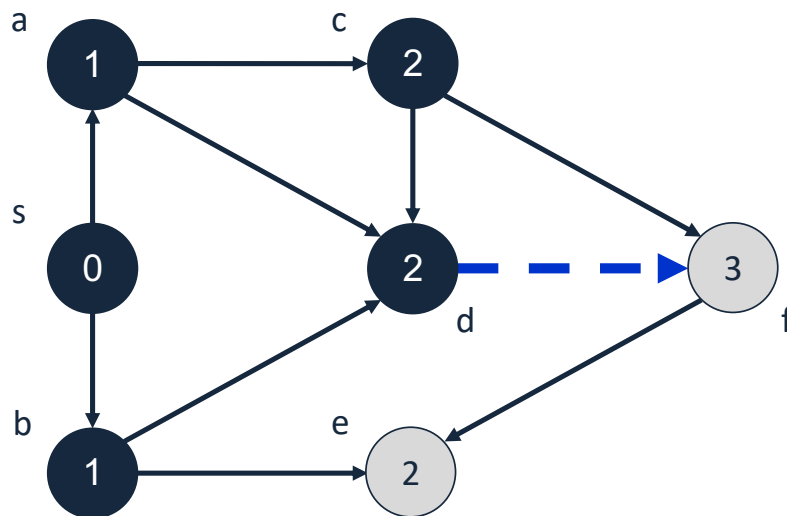
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

d	e	f
---	---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

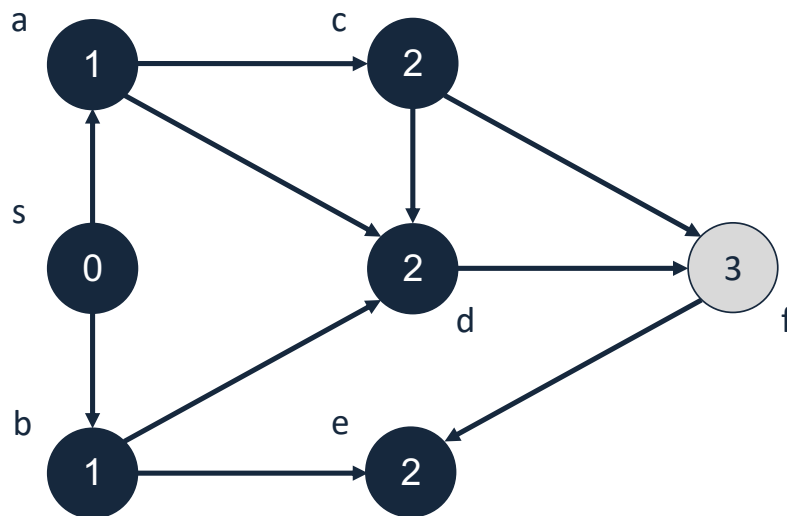
Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue:

e	f
---	---

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

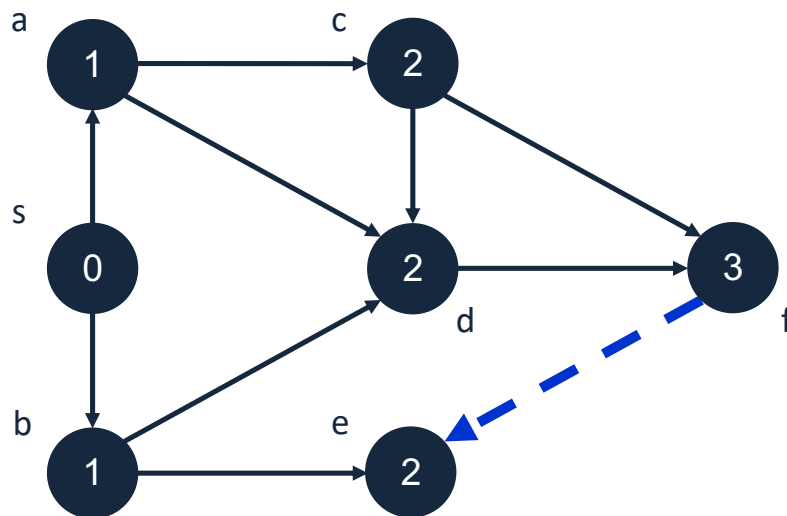
Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue: f

Breitensuche (breadth-first search - BFS)

Beispiel



Nächster Schritt:

Der vorderste Knoten der Queue wird entfernt und schwarz eingefärbt; alle davon erreichbaren Knoten grau eingefärbt und in die Queue eingefügt.

Außerdem wird für die neuen grauen Knoten die Distanz festgelegt.

Queue: leer

Breitensuche (breadth-first search - BFS)

Pseudocode

```

BFS( $V, E, s$ )
1: for each  $v \in V$  do
2:    $v.color \leftarrow \text{WHITE}$ 
3:    $v.dist \leftarrow \infty$ 
4:    $v.pred \leftarrow \text{NIL}$ 
5: end for
6:  $s.color \leftarrow \text{GRAY}$ 
7:  $s.dist \leftarrow 0$ 
8:  $queue \leftarrow \text{new Queue}$  ▷ FIFO Queue
9:  $queue.push(s)$ 
10: while  $queue$  not empty do
11:    $v \leftarrow queue.pop()$ 
12:   for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
13:     if  $u.color = \text{WHITE}$  then
14:        $u.color \leftarrow \text{GRAY}$ 
15:        $u.dist \leftarrow v.dist + 1$ 
16:        $u.pred \leftarrow v$ 
17:        $queue.push(u)$ 
18:     end if
19:   end for
20:    $v.color \leftarrow \text{BLACK}$ 
21: end while

```

Breitensuche (breadth-first search - BFS)

Analyse

- Jeder Knoten wird genau einmal in die Queue aufgenommen und daraus wieder entfernt. Dieser Teil der Lösung besitzt die Zeitkomplexität $O(|V|)$.
- Jede Kante wird nur einmal betrachtet (nämlich beim Entfernen ausgehenden Knotens aus der Queue): $O(|E|)$
- In Summe ergibt sich für die Breitensuche eine lineare Zeitkomplexität von $O(|V| + |E|)$

Tiefensuche

Tiefensuche (depth-first search - DFS)

Vorgehen

- Wie bei der Breitensuche sollen alle erreichbaren Knoten ermittelt werden.
- Der nächste zu bearbeitenden Knoten wird allerdings nicht mit einer Queue festgelegt, sondern durch einen rekursiven Abstieg.
- Die Abbruchbedingung der Rekursion ist gegeben, wenn vom aktuellen Knoten kein weiterer unbesuchter Knoten erreicht werden kann.
- Die Färbungen „weiß“, „grau“ und „schwarz“ werden analog zur Breitensuche verwendet.
- Anders als bei der Breitensuche wird nicht die Distanz zum Startknoten identifiziert. Dagegen erhält jeder Knoten zwei Zeitmarken:
 - Zeitpunkt, zu dem dieser Knoten „grau“ wurde.
 - Zeitpunkt, zu dem dieser Knoten „schwarz“ wurde

Tiefensuche (depth-first search - DFS)

Pseudocode

DFS(V, E)

```

1: for each  $v \in V$  do
2:    $v.color \leftarrow \text{WHITE}$ 
3:    $v.pred \leftarrow \text{NIL}$ 
4: end for
5:  $time \leftarrow 0$  ▷ Globale Zeit auf 0
6: for each  $v \in V$  do
7:   if  $v.color = \text{WHITE}$  then
8:     DFS-SUBTREE( $V, E, v$ )
9:   end if
10: end for

```

DFS-SUBTREE(V, E, v)

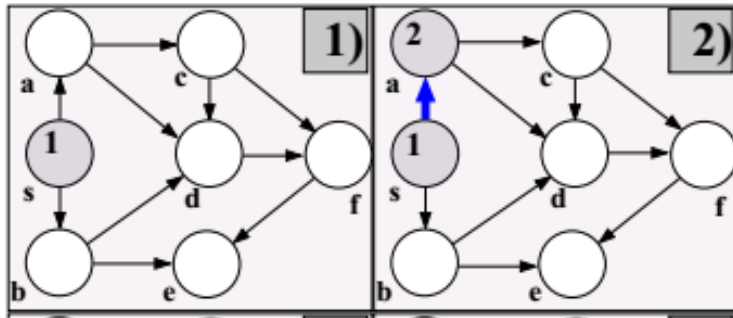
```

1:  $v.color \leftarrow \text{GRAY}$ 
2:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
3:  $v.begin \leftarrow time$ 
4: for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
5:   if  $u.color = \text{WHITE}$  then
6:      $u.pred \leftarrow v$ 
7:     DFS-SUBTREE( $V, E, u$ )
8:   end if
9: end for
10:  $v.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
12:  $v.end \leftarrow time$ 

```

Tiefensuche (depth-first search - DFS)

Beispiel



DFS-SUBTREE(V, E, v)

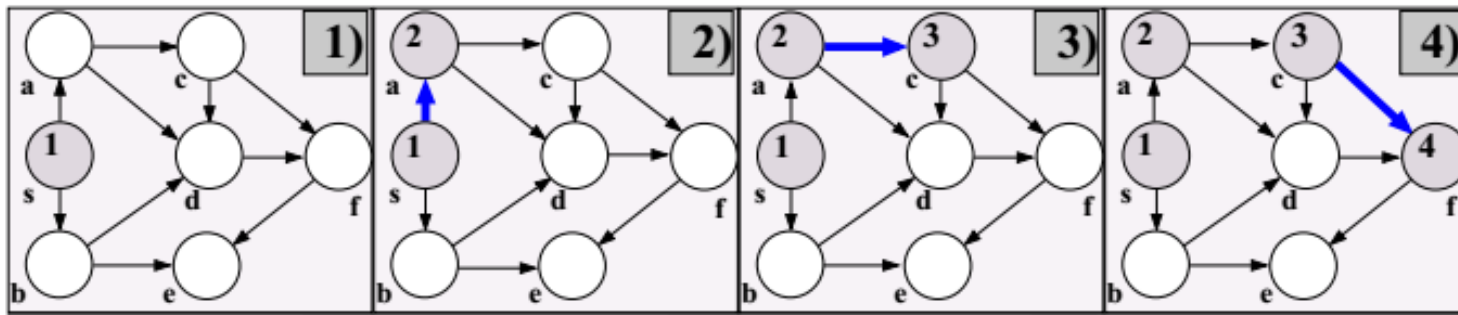
```

1:  $v.color \leftarrow \text{GRAY}$ 
2:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
3:  $v.begin \leftarrow time$ 
4: for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
5:   if  $u.color = \text{WHITE}$  then
6:      $u.pred \leftarrow v$ 
7:     DFS-SUBTREE( $V, E, u$ )
8:   end if
9: end for
10:  $v.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
12:  $v.end \leftarrow time$ 

```

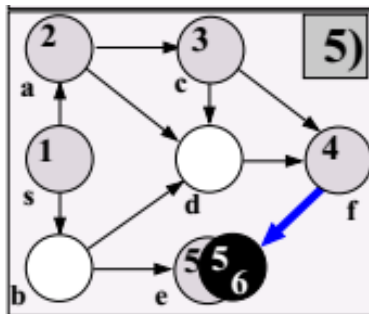
Tiefensuche (depth-first search - DFS)

Beispiel



Tiefensuche (depth-first search - DFS)

Beispiel



DFS-SUBTREE(V, E, v)

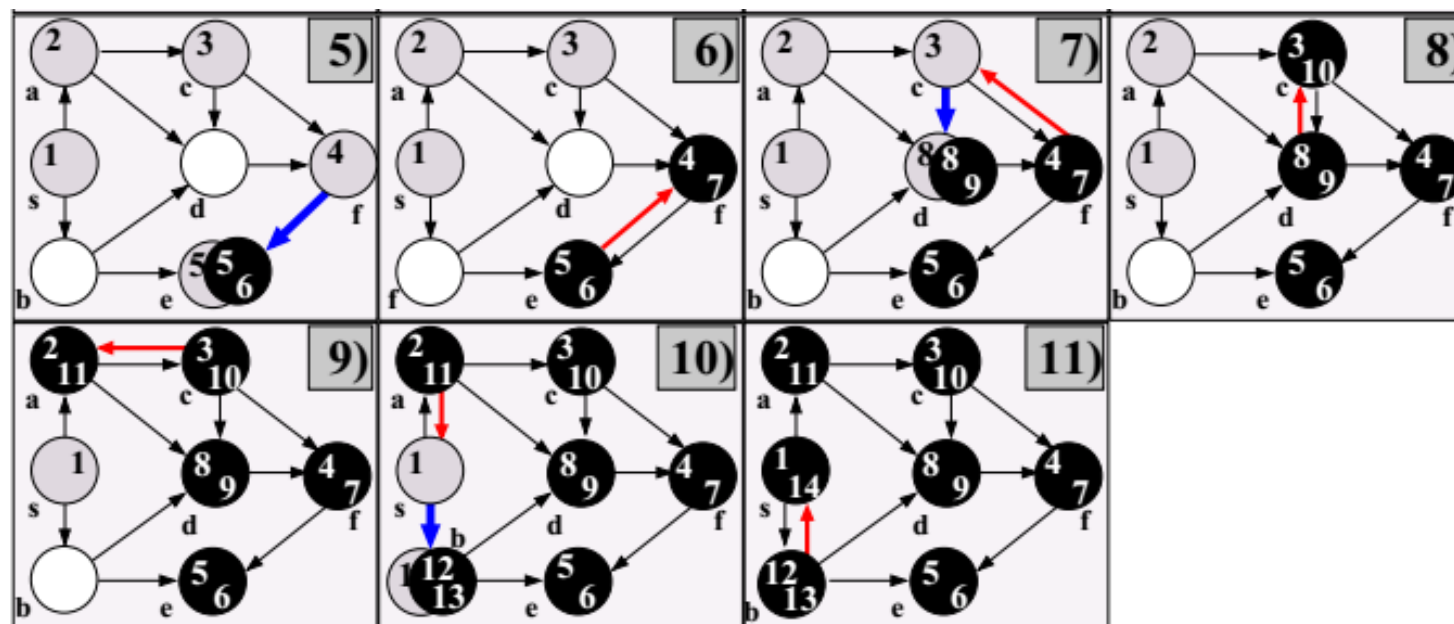
```

1:  $v.color \leftarrow \text{GRAY}$ 
2:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
3:  $v.begin \leftarrow time$ 
4: for each  $v \rightarrow u \in E$  do ▷ Alle von  $v$  ausgehenden Kanten
5:     if  $u.color = \text{WHITE}$  then
6:          $u.pred \leftarrow v$ 
7:         DFS-SUBTREE( $V, E, u$ )
8:     end if
9: end for
10:  $v.color \leftarrow \text{BLACK}$ 
11:  $time \leftarrow time + 1$  ▷ Globale Zeit erhöhen
12:  $v.end \leftarrow time$ 

```

Tiefensuche (depth-first search - DFS)

Beispiel



Tiefensuche (depth-first search - DFS)

Analyse

- Jeder Knoten wird im Rahmen der Rekursion genau einmal besucht. Dieser Teil der Lösung besitzt die Zeitkomplexität $O(|V|)$.
- Jede Kante wird nur einmal betrachtet (nämlich beim ersten Besuch des ausgehenden Knoten): $O(|E|)$
- In Summe ergibt sich auch für die Tiefensuche eine lineare Zeitkomplexität von $O(|V| + |E|)$

Tiefensuche (depth-first search - DFS)

Topologisches Sortieren

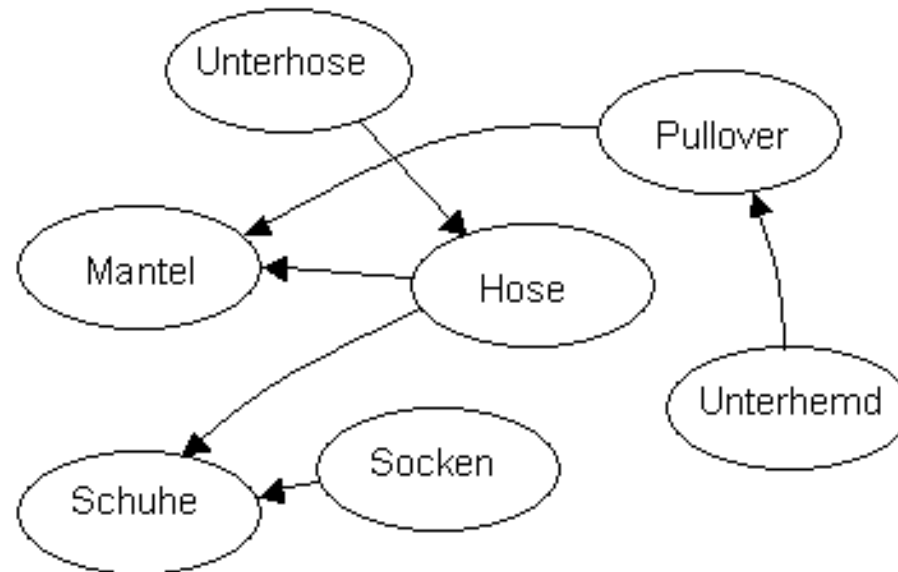
- Die Tiefensuche liefert mit ihren Zeitmarken Strukturinformationen über den Graphen, die für weitergehende Algorithmen genutzt werden können.
- Ein Anwendungsfall ist das Topologische Sortieren, bei dem alle Knoten des Graphen in eine Reihenfolge gebracht werden.
- Bei dieser Reihenfolge ist sichergestellt, dass

$$(u, v) \in E \rightarrow u < v$$

Tiefensuche (depth-first search - DFS)

Topologisches Sortieren - Beispiel

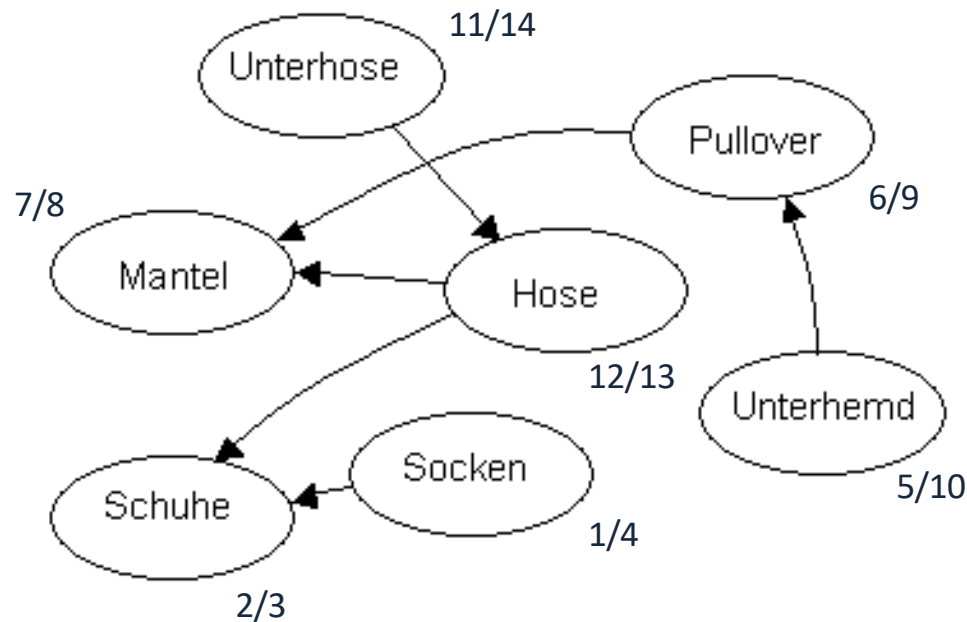
Gegeben ist ein azyklischer gerichteter Graph, der die Abhängigkeiten beim Ankleiden wiedergibt.



Tiefensuche (depth-first search - DFS)

Topologisches Sortieren - Beispiel

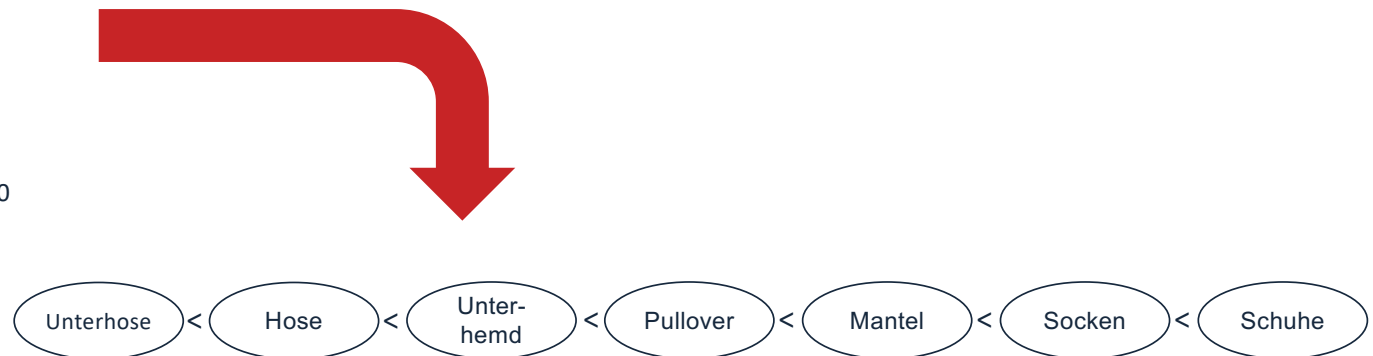
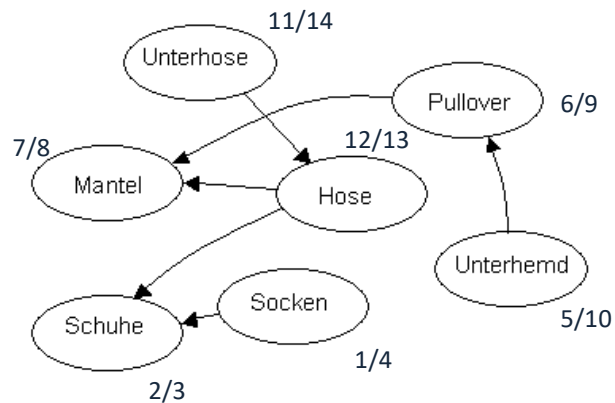
Nach einer Tiefensuche besitzt jeder Knoten zwei Zeitmarken (Zeitpunkt „grau“ und Zeitpunkt „schwarz“)



Tiefensuche (depth-first search - DFS)

Topologisches Sortieren - Beispiel

Eine mögliche topologische Sortierung ist nun die Anordnung nach den „schwarz“-Zeitmarken in absteigender Reihenfolge



Ziele der heutigen Lerneinheit

Sie verstehen...

- wie kantengewichtete Graphen formal definiert sind
- die Kernprinzipien der Graphen-Relaxation
- die grundlegenden Unterschiede zwischen gieriger Suche (Dijkstra) und systematischem Durchlaufen (Bellman-Ford).
- das Konzept einer informierten Suche

Sie können...

- den Dijkstra- und Bellman-Ford-Algorithmus schrittweise auf einem gegebenen, bewerteten Graphen ausführen
- bestehende Graphen auf negative Zyklen prüfen
- die geschätzten Gesamtkosten für den A*-Algorithmus ermitteln und den Ablauf bis zum Zielknoten darstellen

Bewertete Graphen

Bewertete Graphen

Definition

Bisher haben wir eine Kante als „einen Schritt“ im Graphen betrachtet, um die Länge von Pfaden zu ermitteln.

In einer allgemeineren Sichtweise können die Kanten aber unterschiedliche Längen haben, die z.B. zu unterschiedlichen Kosten beim Beschreiten der Kante führen können.

Ja sogar „negative Kosten“ sind denkbar, wenn beim Beschreiten eine Auszahlung erfolgt.

Graphen $G = (V, E)$, die mit Hilfe einer Bewertungsfunktion $w: E \rightarrow \mathbb{R}$ reellwertige Kantengewichte besitzen, heißen ***bewertete Graphen***.

Bewertete Graphen

Gewicht eines Pfades

Das Gewicht w eines Pfades

$$p = \langle v_0, v_1, \dots, v_k \rangle$$

ist die Summe der Gewichte aller Kanten,
aus denen er zusammengesetzt ist:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Bewertete Graphen

Kürzester Pfad

Sei P die Menge aller Pfade p von u nach v :

$$P = \{p: u \xrightarrow{p} v\}$$

Das minimale Gewicht eines Pfades von u nach v ist definiert durch:

$$\delta(u, v) = \begin{cases} \min_{p \in P} w(p), & P \neq \emptyset \\ \infty, & \text{sonst} \end{cases}$$

Ein kürzester Pfad $\hat{p}$ von u nach v besitzt dann das Gewicht

$$w(\hat{p}) = \delta(u, v)$$

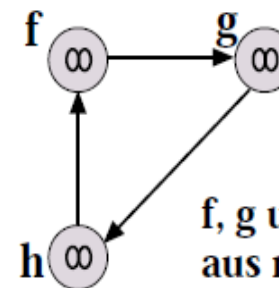
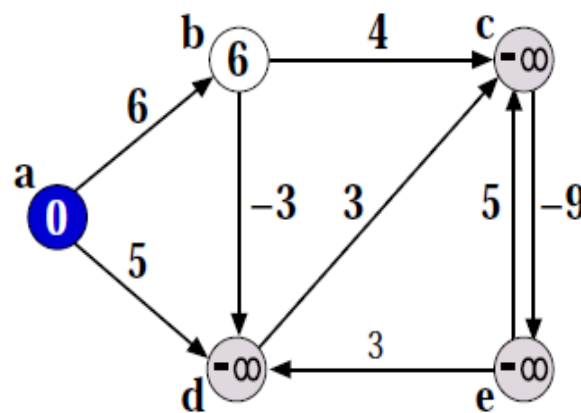
Suche nach kürzesten Pfaden in bewerteten Graphen

Suche nach kürzesten Pfaden

Problem: Negativ gewichtete Zyklen

Ist von einem Knoten s aus ein negativ gewichteter Zyklus erreichbar, dann sind Gewichte des kürzesten Pfades nicht wohl definiert, da mit jedem Durchlaufen des Zyklus der Pfad immer kürzer wird, so dass

$$\delta(s, v) = -\infty$$



f, g und h sind von Knoten a aus nicht erreichbar

Suche nach kürzesten Pfaden

Obere Schranke für das Gewicht des kürzesten Pfades

In den folgenden Algorithmen merken wir uns für jeden Knoten v eine obere Schranke d für das Gewicht des kürzesten Pfades vom Startknoten s zu v .

Zu Beginn gilt:

$$d(v) = \begin{cases} 0, & v = s \\ \infty, & v \neq s \end{cases}$$

Suche nach kürzesten Pfaden

Relaxation

Während der Ausführung der Algorithmen wird die obere Schranke als Schätzung des kürzesten Pfades schrittweise reduziert.

Dabei wird beim Besuch des Knotens u getestet, ob – sofern eine Kante (u, v) existiert – der bislang gültige Schätzwert für v aktualisiert werden muss. Diesen Schritt nennt man **Relaxieren**.

Suche nach kürzesten Pfaden

Relaxation

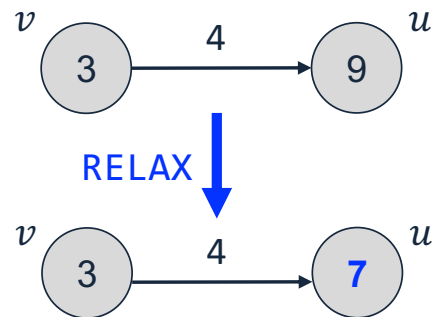
RELAX(e)

```

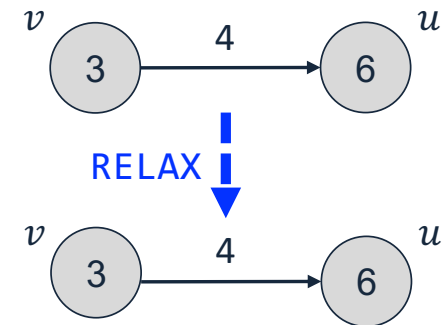
1:  $(v, u) \leftarrow e$ 
2: if  $v.dist + w(e) < u.dist$  then
3:    $u.dist \leftarrow v.dist + w(e)$ 
4:    $u.pred \leftarrow v$ 
5: end if
  
```

 $\triangleright e \in E \subseteq V \times V$

Kürzen des Pfades:



Kein Kürzen des Pfades:



Dijkstra Algorithmus

Dijkstra Algorithmus

Wer war Dijkstra?



Edsger W. Dijkstra
(1930 – 2002)

Niederländischer Informatiker

- 1960 Compiler für Algol 60
- 1965 Semaphore zur Synchronisation
- 1968 „Go To Considered Harmful“
- 1971 „Dining Philosophers“
- 1972 Turing Award
- 1979 „Separation of Concerns“

Dijkstra Algorithmus

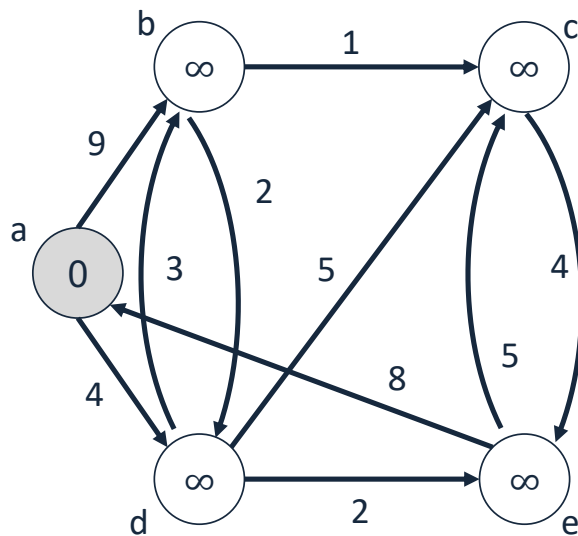
Grundidee

Ähnlich wie bei der Breitensuche werden die weiter zu untersuchenden Knoten in einer Warteschlange *queue* vorgehalten. Allerdings wird nicht das FIFO-Prinzip verwendet, sondern stets der Knoten mit dem kleinsten d -Wert ausgewählt (Vorrangwarteschlange).

Voraussetzung

Kantengewichte sind nicht negativ: $w: E \rightarrow \mathbb{R}_0^+$

Dijkstra Algorithmus



queue:

a
0

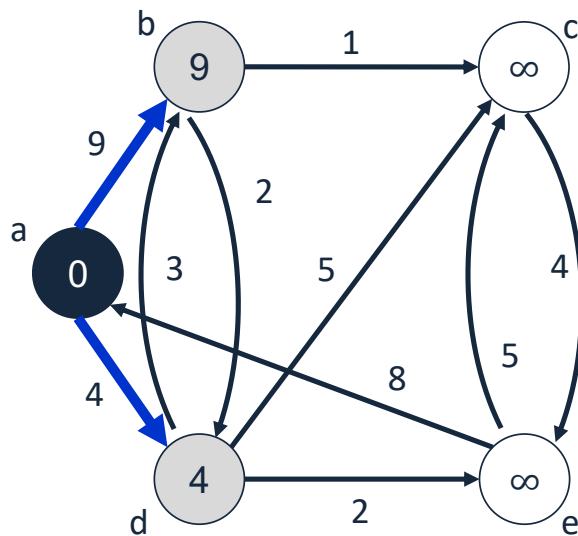
Start:

Alle Distanzen werden auf unendlich gesetzt.

Lediglich der Startknoten erhält die Distanz 0.

Daher ist er der Knoten mit der geringsten Distanz und wird als nächstes gewählt.

Dijkstra Algorithmus



queue:

b	d
9	4

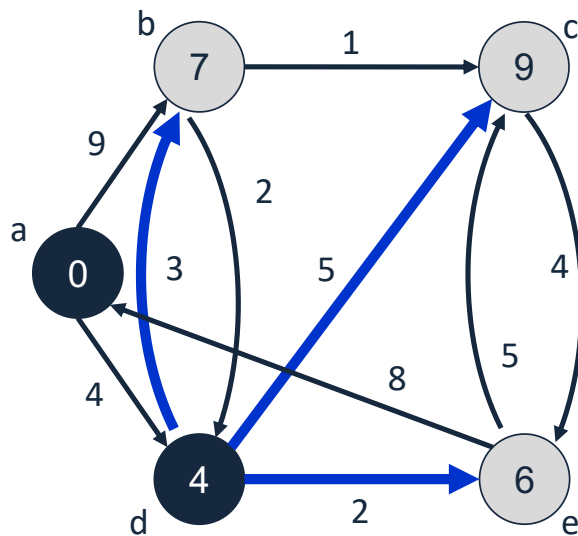
Nächster Schritt:

Entlang aller ausgehenden Kanten von a wird relaxiert.

Der Knoten a wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten d hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



queue:

b	c	e
7	9	6

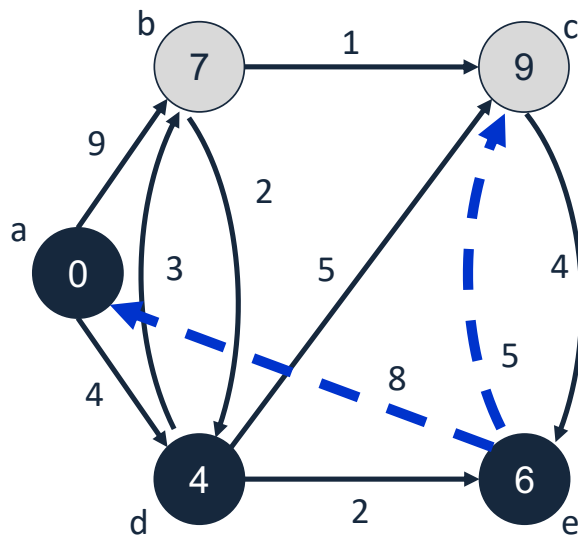
Nächster Schritt:

Entlang aller ausgehenden Kanten von *d* wird relaxiert.

Der Knoten *d* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *e* hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



queue:

b	c
7	9

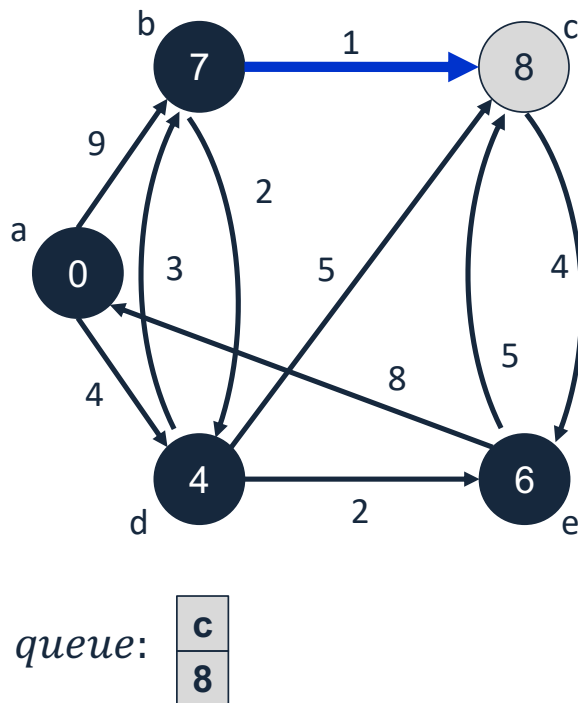
Nächster Schritt:

Die ausgehenden Kanten von *e* bringen keine Verbesserungen.

Der Knoten *e* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *b* hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



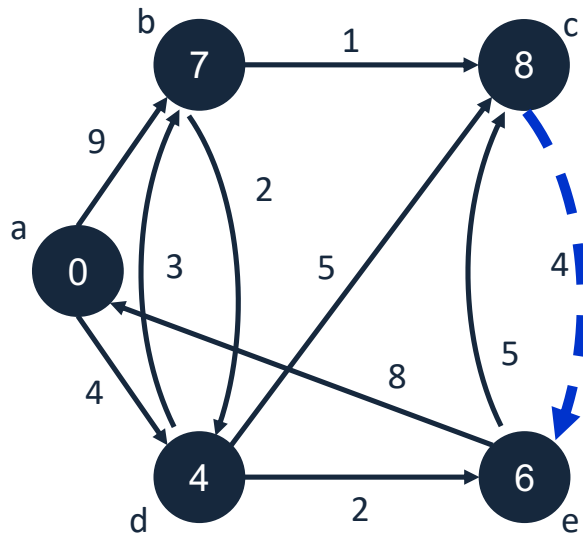
Nächster Schritt:

Entlang aller ausgehenden Kanten von *b* wird relaxiert.

Der Knoten *b* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *c* hat jetzt die geringste Distanz und wird gewählt.

Dijkstra Algorithmus



queue: $\emptyset$

Nächster Schritt:

Die ausgehenden Kanten von *c* bringen keine Verbesserungen.

Der Knoten *c* wird aus *queue* entfernt und ist abschließend berechnet.

Die *queue* ist nun leer, der Algorithmus terminiert.

Dijkstra Algorithmus

DIJKSTRA(V, E, s)

```
1: queue  $\leftarrow$  new PriorityQueue            $\triangleright$  Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$ 
4:    $v.pred \leftarrow \mathbf{NIL}$ 
5:   queue.insert( $v$ )
6: end for
7:  $s.dist \leftarrow 0$ 
8: while queue not empty do
9:    $v \leftarrow queue.extract\_min()$ 
10:  for each  $e : v \rightarrow u \in E$  do     $\triangleright$  Alle von  $v$  ausgehenden Kanten
11:    RELAX( $e$ )
12:  end for
13: end while
```

Dijkstra Algorithmus

Analyse

Jede Kante wird nur einmal betrachtet.

Jeder Knoten wird ebenfalls nur einmal betrachtet, allerdings muss jedesmal der nächste Knoten aus den verbleibenden ausgewählt werden. Bei einer Implementierung als lineare Suche in einem Array ergibt sich

$$O(|V|^2 + |E|)$$

Verwendet man für die Realisierung der Vorrangwarteschlange einen Min-Heap (vgl. Max-Heap beim Heap-Sort), so kann

$$O(|V| \log |V| + |E|)$$

erreicht werden.

Bellman-Ford Algorithmus

Bellman-Ford Algorithmus

Grundidee

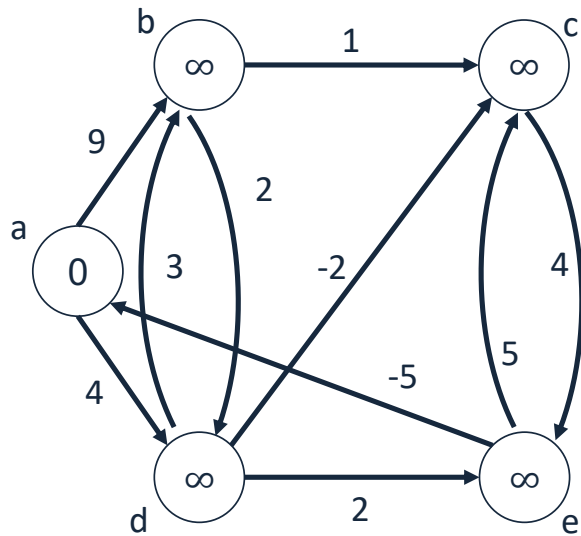
Der kürzeste Pfad ist bei $|V|$ Knoten maximal $|V| - 1$ Kanten lang. Daher werden nach einer Initialisierung analog zum Dijkstra-Algorithmus $|V| - 1$ mal alle Kanten durchlaufen und relaxiert. Ausgehend vom Startknoten sind damit alle kürzesten Pfade gesetzt.

Gibt es dann noch immer eine verbessernde Relaxation, so muss ein Zyklus mit negativem Kantengewicht enthalten sein.

somit:

Kantengewichte dürfen auch negativ sein. Das Auftreten eines Zyklus mit negativem Gewicht wird erkannt.

Bellman-Ford Algorithmus

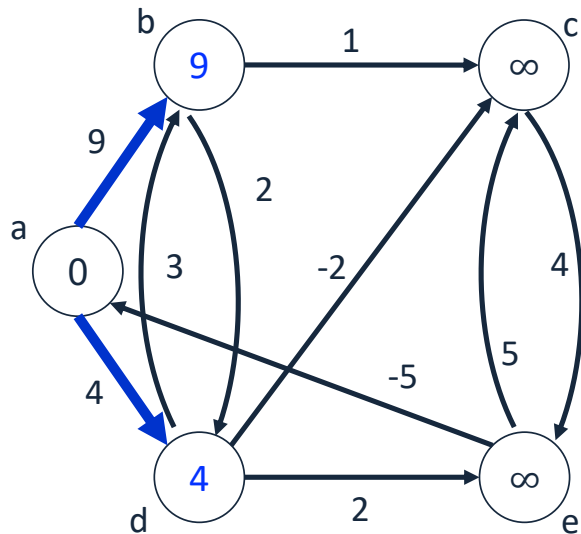


Start:

Alle Distanzen werden auf unendlich gesetzt.

Lediglich der Startknoten erhält die Distanz 0.

Bellman-Ford Algorithmus

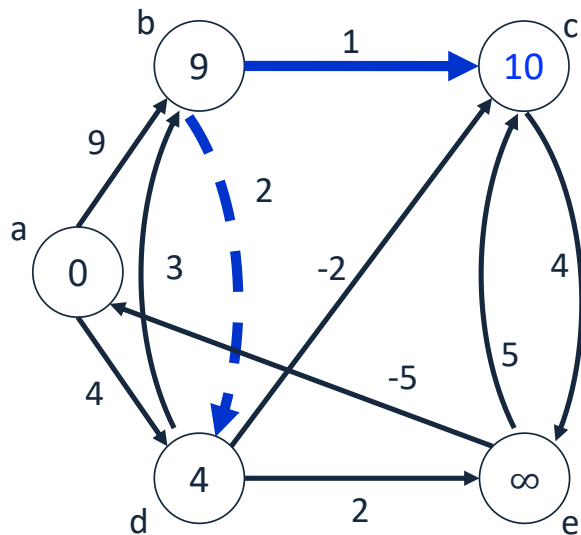


Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durch geführt werden.

1. Durchlauf bei 5 Knoten

Bellman-Ford Algorithmus

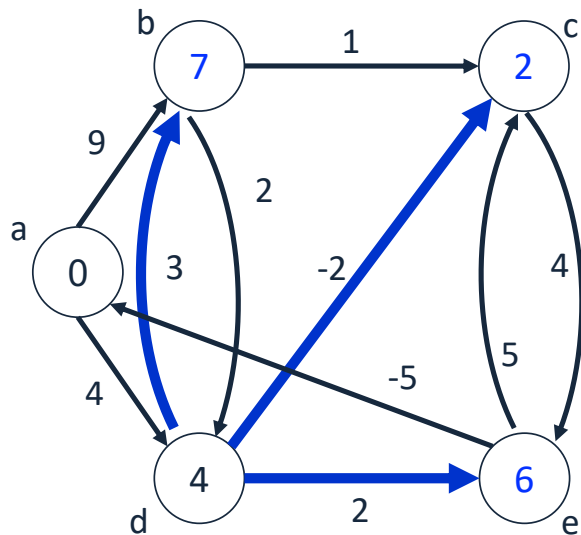


Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durchgeführt werden.

2. Durchlauf bei 5 Knoten

Bellman-Ford Algorithmus

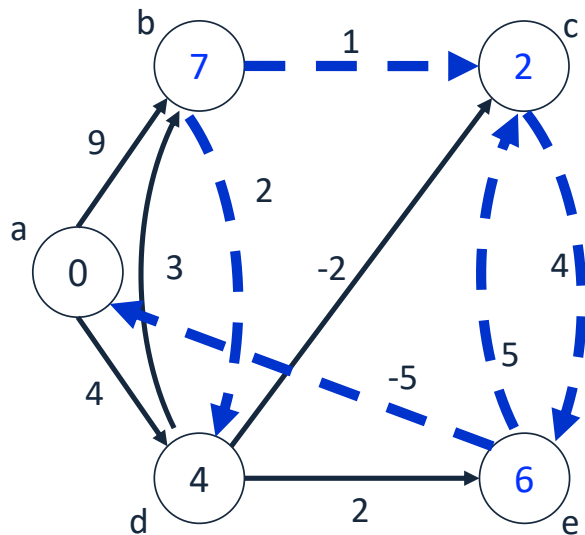


Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durch geführt werden.

2. Durchlauf bei 5 Knoten

Bellman-Ford Algorithmus



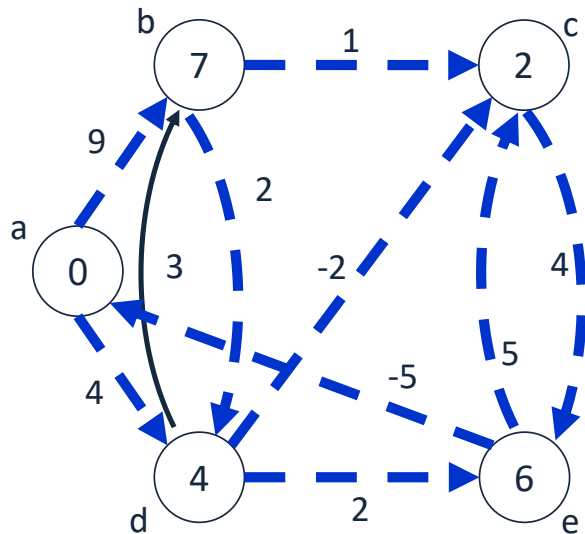
Nächster Schritt

Nur bei Knoten mit einer Distanz ungleich ∞ kann sinnvoll eine Relaxation durchgeführt werden.

3. Durchlauf bei 5 Knoten

Keine Verbesserung mehr möglich.

Bellman-Ford Algorithmus



Nächster Schritt

Auch beim 4. und 5. Durchlauf kann keine Verbesserung mehr erreicht werden.

Daher sind keine Zyklen mit negativem Gewicht enthalten.

Bellman-Ford Algorithmus

Analyse

Der Bellman-Ford-Algorithmus besitzt eine Laufzeit von

$$O(|V| \cdot |E|)$$

BELLMAN-FORD(V, E, s)

```

1: for each  $v \in V$  do
2:    $v.dist \leftarrow \infty$ 
3:    $v.pred \leftarrow \text{NIL}$ 
4: end for
5:  $s.dist \leftarrow 0$ 
6: for  $i \leftarrow 1$  to  $|V| - 1$  do
7:   for each  $e \in E$  do                                ▷ Alle Kanten
8:     RELAX( $e$ )
9:   end for
10: end for
11: for each  $e : v \rightarrow u \in E$  do                        ▷ Alle Kanten
12:   if  $v.dist + w(e) < u.dist$  then
13:     return False
14:   end if
15: end for
16: return True

```

A* Algorithmus

A* Algorithmus

Informierte Suche

Der **A* Algorithmus** findet den kürzesten Weg von einem Startknoten s zu einem Zielknoten z .

Wie der Dijkstra Algorithmus erfordert A* ausschließlich positive Kantengewichte.

Er benötigt eine **Heuristik**

$$h: V \rightarrow \mathbb{R}_0^+$$

die vorab die Kosten für das Erreichen des Zielknotens abschätzt.

Damit handelt es sich um eine **informierte Suche**.

A* Algorithmus

Heuristik für den A* Algorithmus

Die Heuristik muss **monoton** sein, d.h.

- der Schätzwert der Heuristik darf die wahren Kosten für keinen Knoten übersteigen.
- der Schätzwert für einen Vorgänger darf die geschätzten Kosten für einen Nachfolger im Pfad nicht überschreiten:

$$h(v) \leq w(v, u) + h(u)$$

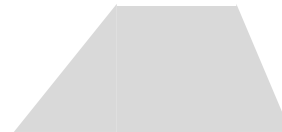
Diese Eigenschaften sind z.B. durch die Verwendung der „Luftlinien“-Entfernung zum Zielknoten gegeben.

A* Algorithmus

Geschätzte Gesamtkosten

Mit Hilfe der Heuristik und den sukzessive ermittelten Kosten für das Erreichen eines Knotens lassen sich stets die Gesamtkosten f vom Startknoten s bis zum Zielknoten z abschätzen:

$$f(v) = d(v) + h(v)$$



Kosten von s zu v



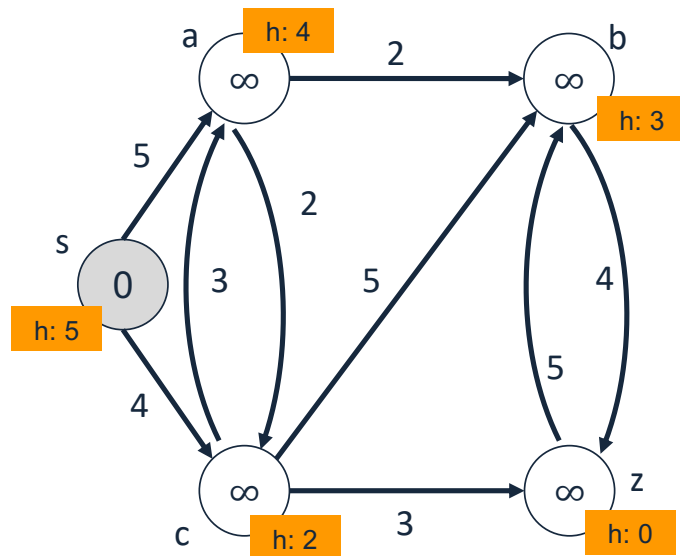
Kosten von v zu z

A* Algorithmus

Grundidee

Der A* Algorithmus ähnelt dem Verfahren von Dijkstra, allerdings wird nicht der Knoten für die weitere Expansion ausgewählt, der mit den geringsten Kosten d erreicht werden kann, sondern der Knoten, der die geringsten geschätzten Gesamtkosten f ausweist.

A* Algorithmus



queue:

s
d 0
h 5
f = d + h 5

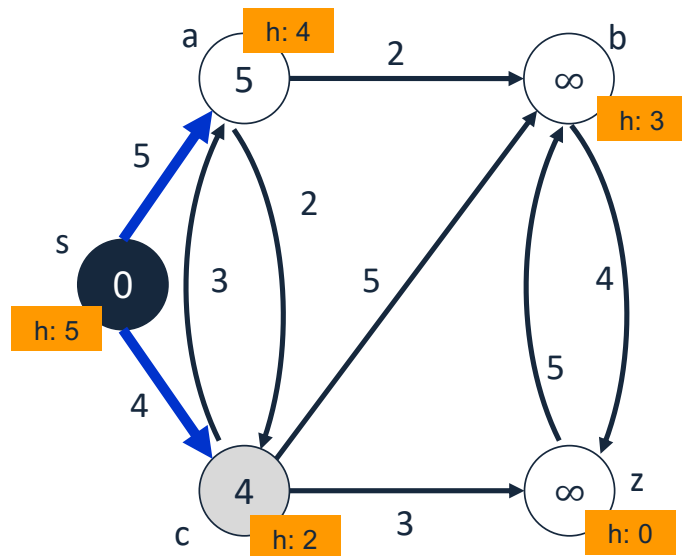
Start:

Alle Distanzen werden auf unendlich gesetzt.

Lediglich der Startknoten erhält die Distanz 0 und wird in die *queue* eingesetzt.

Dort ist er der Knoten mit den geringsten geschätzten Gesamtkosten f und wird daher als nächstes gewählt.

A* Algorithmus



queue:

	a	c
d	5	4
h	4	2
f = d + h	9	6

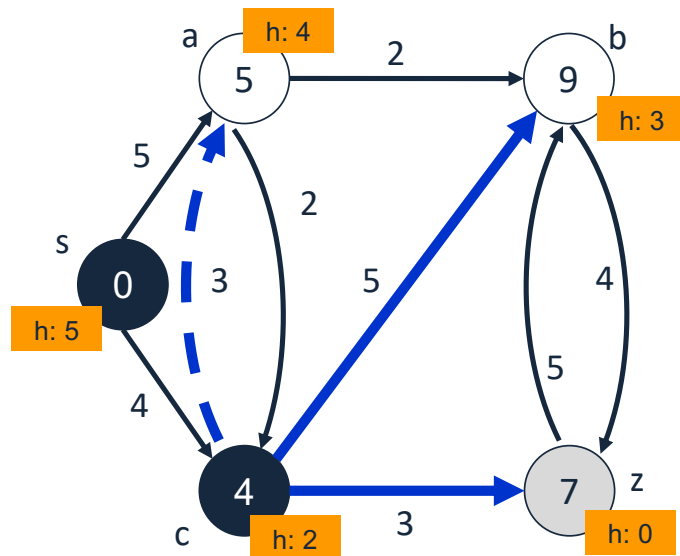
Nächster Schritt:

Entlang aller ausgehenden Kanten von *s* wird relaxiert und die erreichten Knoten in *queue* aufgenommen.

Der Knoten *s* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *c* hat jetzt die geringsten Gesamtkosten und wird expandiert.

A* Algorithmus



queue:

	a	b	z
d	5	9	7
h	4	3	0
f = d + h	9	12	7

Nächster Schritt:

Entlang aller ausgehenden Kanten von *c* wird relaxiert und die erreichten Knoten in *queue* aufgenommen.

Der Knoten *c* wird aus *queue* entfernt und ist abschließend berechnet.

Der Knoten *z* hat jetzt die geringsten Gesamtkosten. Es ist der Zielknoten und der Algorithmus terminiert.

A* Algorithmus

Korrektheitsüberlegung

Ist für den gewählten Knoten v bereits der kürzeste Pfad gefunden?

Gäbe es einen alternativen kürzeren Weg über u

$$d(u) + w(u, v) < d(v)$$

dann

$$f(u) < f(v)$$

und v wäre nicht gewählt worden.

Begründung:

$$d(u) + w(u, v) < d(v)$$

$$d(u) + h(u) + w(u, v) < d(v) + w(u, v) + h(v)$$

$$d(u) + h(u) < d(v) + h(v)$$

Aufgrund der Monotonie
 $h(u) \leq w(u, v) + h(v)$

A* Algorithmus

COST(v)

```

1: ▷ Die Kosten eines Pfades vom Start zum Ziel über  $v$ 
2: return  $v.dist + heuristic(v)$ 

```

A*(V, E, s, z)

```

1:  $queue \leftarrow$  new PriorityQueue           ▷ Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$ 
4:    $v.pred \leftarrow \text{NIL}$ 
5:    $v.color \leftarrow \text{WHITE}$ 
6: end for
7:  $s.dist \leftarrow 0$ 
8:  $s.color \leftarrow \text{GRAY}$ 
9:  $queue.insert(s)$ 
10: while  $queue$  not empty do
11:    $v \leftarrow queue.extract\_min()$            ▷ Basierend auf COST
12:   if  $v = z$  then
13:     return True
14:   end if
15:   for each  $e : v \rightarrow u \in E$  do           ▷ Alle von  $v$  ausgehenden Kanten
16:     if  $u.color = \text{BLACK}$  then
17:       continue
18:     end if
19:      $cost \leftarrow v.dist + w(e) + heuristic(u)$ 
20:     if  $u.color = \text{GRAY} \wedge cost > \text{COST}(u)$  then
21:       continue
22:     end if
23:      $u.pred \leftarrow v$ 
24:      $u.dist \leftarrow v.dist + w(e)$ 
25:      $u.color \leftarrow \text{GRAY}$ 
26:      $queue.insert\_or\_update(u)$ 
27:   end for
28:    $v.color \leftarrow \text{BLACK}$ 
29: end while
30: return False

```

A* Algorithmus

Analyse

Die Zeitkomplexität ist abhängig von der Implementierung der Auswahl des Knotens mit minimalem f . Wird hier ein Min-Heap gewählt, dann beträgt die Laufzeit im Worst-Case

$$O(|V|^2)$$

Diese schlechte Zeitkomplexität trifft jedoch im durchschnittlichen Fall nicht zu, da die Stärke des A*-Algorithmus seine zielgerichtete Suche ist und im Vergleich zur Gesamtzahl der Knoten meist nur ein geringer Teil davon untersucht wird.

Ziele der heutigen Lerneinheit

Sie verstehen...

- ... was einen Baum, einen aufspannenden Baum und einen minimalen aufspannenden Baum auszeichnet.
- ... worin sich die Algorithmen von Prim und Kruskal in ihrer Vorgehensweise unterscheiden..

Sie können...

- ... für einen gegebenen gewichteten Graphen schrittweise einen minimalen aufspannenden Baum konstruieren.
- ... den Algorithmus von Prim und den Algorithmus von Kruskal auf Beispiele anwenden.
- ... die Ergebnisse und die Laufzeit der beiden Verfahren grundlegend bewerten.

Graphen vs. Bäume

Graphen vs. Bäume

Definition (ungerichteter) Baum

Ein **(ungerichteter) Baum** ist ein spezieller Typ von Graph $G = (V, E)$, der

- zusammenhängend ist
- keine Zyklen enthält
- und keine gerichteten Kanten besitzt

Damit ergibt sich zwangsläufig, dass es von jedem Knoten eines Baums genau einen Pfad zu jedem anderen Knoten gibt.

Falls G nicht leer gilt: $|V| = |E| + 1$

Graphen vs. Bäume

Wurzel eines (ungerichteten) Baums

Man kann in einem ungerichteten Baum $G = (V, E)$ jeden Knoten $v \in V$ zur Wurzel machen.

Beweis durch ein Gedankenexperiment

Man greife den Knoten v und schüttele den gesamten Graphen. Die übrigen Knoten werden von der Schwerkraft nach unten gezogen und bilden den Baum mit der Wurzel v .

Hinweis

Bei einem gerichteten Baum, bei dem die Kanten eine Richtung haben, ist dies nicht möglich. Zur eindeutigen Beschreibung einer Hierarchie ist also ein gerichteter Baum notwendig. Die bisher betrachteten Bäume waren somit gerichtete Bäume.

Graphen vs. Bäume

Aufspannende Bäume

Gegeben ist ein ungerichteter zusammenhängender Graph $G = (V, E)$, der auch Zyklen enthalten kann.

Ein Graph $G_B = (V, E_B)$ mit $E_B \subseteq E$, der die Eigenschaften eines ungerichteten Baums besitzt (insbes. keine Zyklen), heißt **aufspannender Baum** für den Graphen G .

Hinweis

Der aufspannende Baum muss alle Knoten enthalten, und seine Kanten sind auch im Ursprungsgraph enthalten.

Graphen vs. Bäume

Aufspannende Bäume

Ein Graph kann mehrere aufspannende Bäume besitzen.

Beispiel

Ein vollständiger Graph mit 4 Knoten besitzt 16 aufspannende Bäume mit unterschiedlichen Kantenkombinationen.

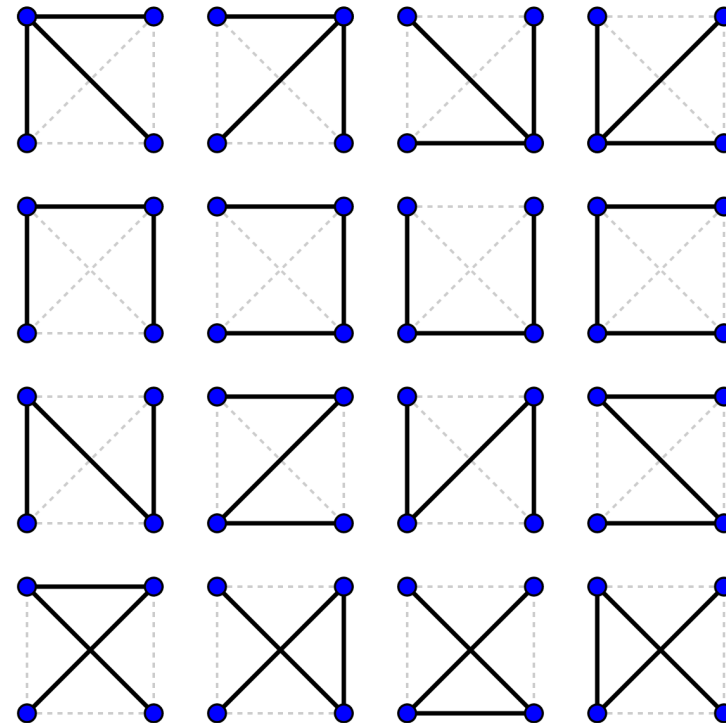


Abbildung: Wikimedia

Minimaler aufspannender Baum (engl.: minimal spanning tree, MST)

Minimaler aufspannender Baum

Definition

Gegeben ist ein bewerteter ungerichteter zusammenhängender Graph $G = (V, E)$, der auch Zyklen enthalten kann.

Ein Graph $G_B = (V, E_B)$ mit $E_B \subseteq E$, der die Eigenschaften eines ungerichteten Baums besitzt (insbes. keine Zyklen), heißt **minimaler aufspannender Baum** für den Graphen G , wenn gilt:

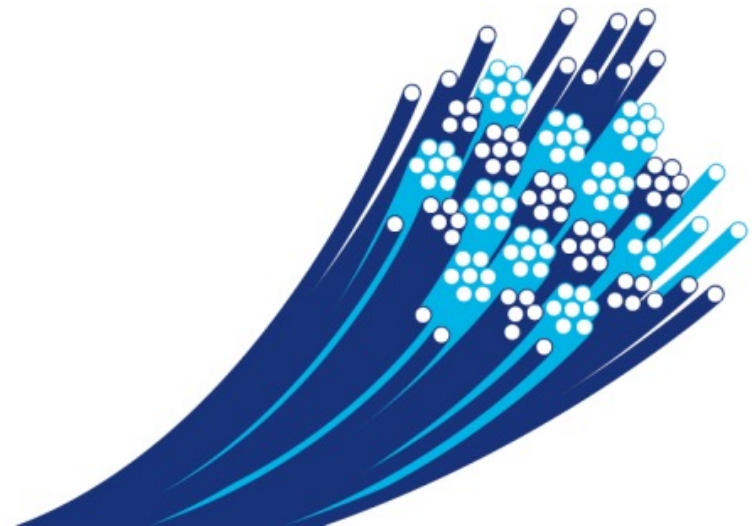
$$\sum_{(u,v) \in E_B} w(u, v) \text{ minimal}$$

Minimaler aufspannender Baum

Anwendungsbeispiel

Bei Infrastrukturmaßnahmen stellt sich gelegentlich die Frage, welche Strecken ausgebaut werden sollen, um eine flächendeckende Versorgung zu gewährleisten.

So könnten z.B. beim Ausbau des Glasfasernetzes Entscheidungen notwendig sein, zwischen welchen Städten/Ortschaften Glasfaserstrecken gebaut werden sollen, so dass anschließend alle Ortschaften mit minimalen Kosten angebunden sind.



Minimaler aufspannender Baum

Greedy-Ansatz

Sei $G = (V, E)$ ein gewichteter ungerichteter zusammenhängender Graph.
Gesucht ist die Menge der Kanten des minimal aufspannenden Baums $E_B \subseteq E$.

„Greedy-Ansatz“

Man startet mit einer leeren Menge A und fügt iterativ immer eine Kante hinzu, die **sicher** zum minimal aufspannenden Baum gehört. Nach $|V| - 1$ Schritten terminiert der Algorithmus.

Ein solcher Algorithmus heißt **greedy** (gierig), weil er sukzessive die Gesamtlösung durch Auswahl einer optimalen Alternative konstruiert, ohne frühere Entscheidungen revidieren zu müssen. Die Herausforderung dabei ist, stets eine **sichere Kante** zu ermitteln.

Algorithmus von Prim

Algorithmus von Prim

Grundidee

Der MST wird iterativ mit einem Greedy-Ansatz gefunden.

Man unterscheidet bei jedem Iterationsschritt zwischen Knoten P , die von den bereits identifizierten Kanten A des MST erreicht werden, und den Knoten $K \subseteq V$, die noch nicht erreicht werden.

Nun wählt man die Kante e mit dem geringsten Gewicht aus, die einen Knoten aus P mit einem Knoten K verbindet und fügt e zur Menge A hinzu.

Der sich nach $|V| - 1$ Schritten ergebende Baum $G_B = (V, A)$ ist der MST von $G = (V, E)$

Algorithmus von Prim

Korrektheitsüberlegung

Ist e eine sichere Kante?

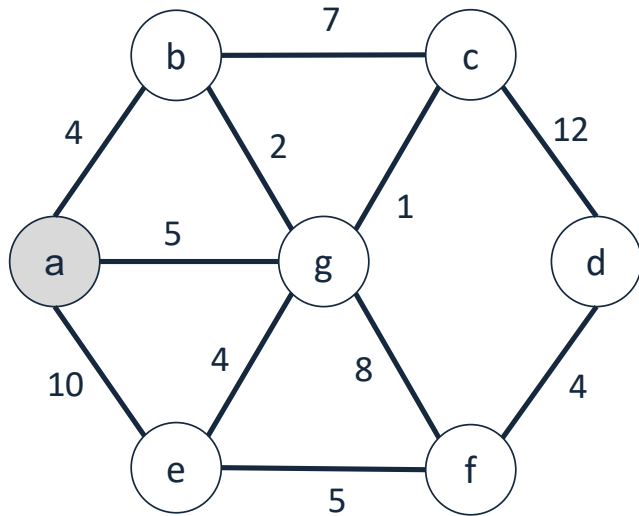
Annahme: Es gibt einen abweichenden MST G'_B , der $e = (u, v)$ nicht enthält.

In diesem MST muss es einen Pfad geben, der die Knoten u und v verbindet. Da e die Knotenteilmengen K und P verbindet, verbindet auch dieser Pfad die Teilmengen, d.h. im Pfad gibt es eine Kante f , die einen Knoten aus K mit einem Knoten aus P verbindet.

Es gilt $w(f) \geq w(e)$, sonst wäre f anstelle von e gewählt worden.

Entfernt man f aus G'_B und fügt stattdessen e ein, so erhält man wieder einen MST. Somit ist e eine sichere Kante.

Algorithmus von Prim

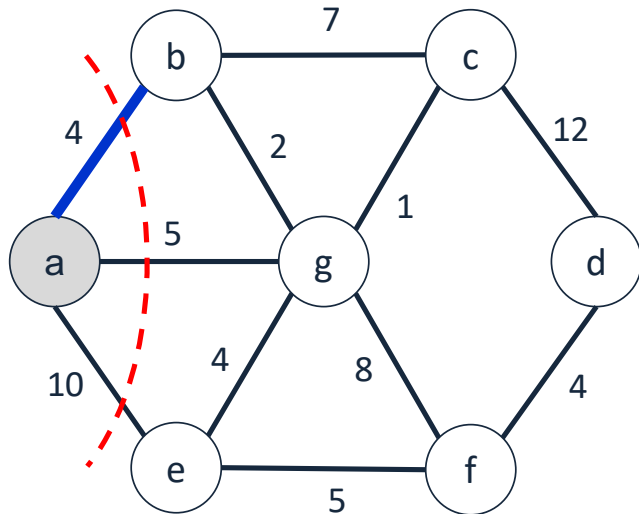


Start

Zu Beginn ist die Kantenmenge A leer.

Da am Ende alle Knoten verbunden sein müssen, kann mit einem beliebigen Knoten in der Menge P begonnen werden (hier: a). Alle anderen Knoten sind in der Menge K .

Algorithmus von Prim

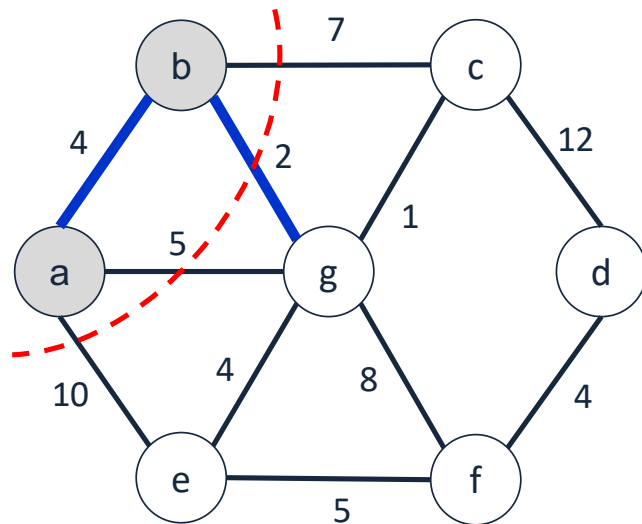


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (a, b) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

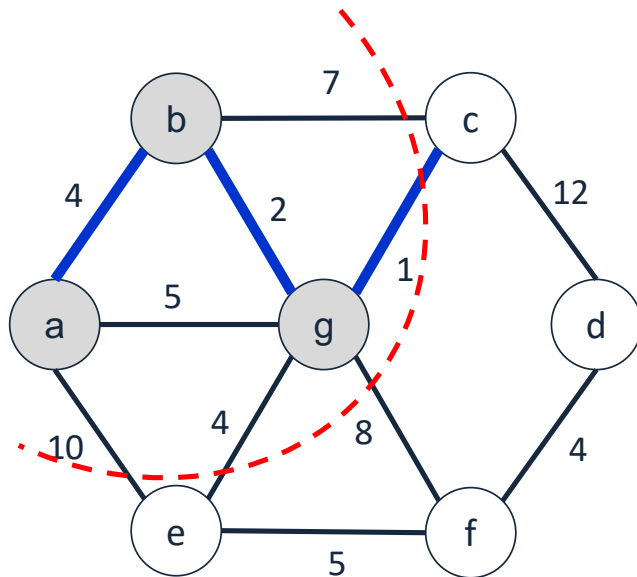


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (b, g) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

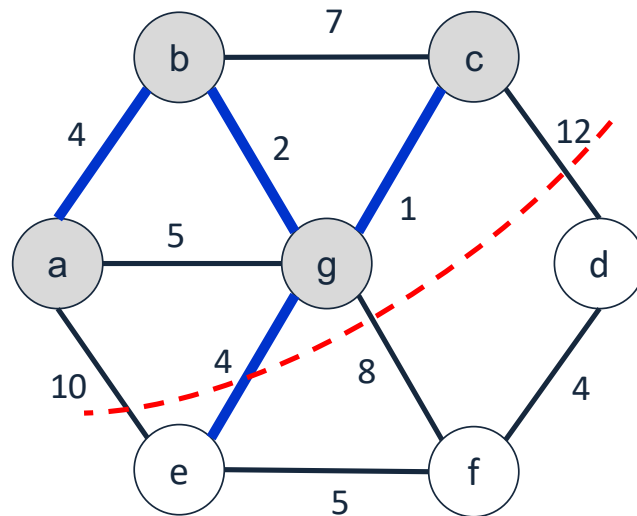


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (g, c) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

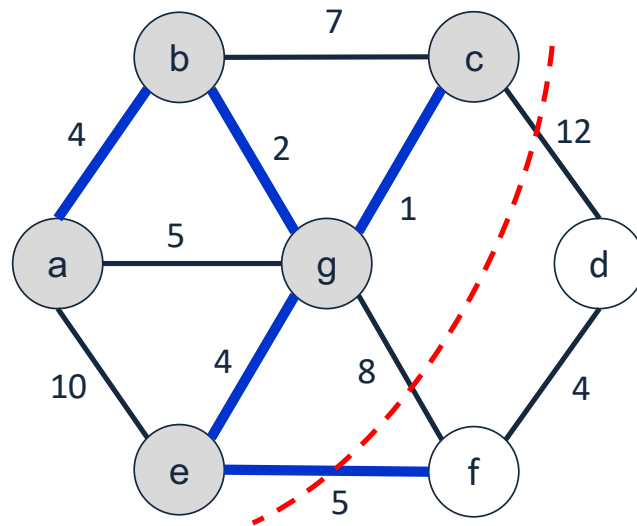


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (g, e) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

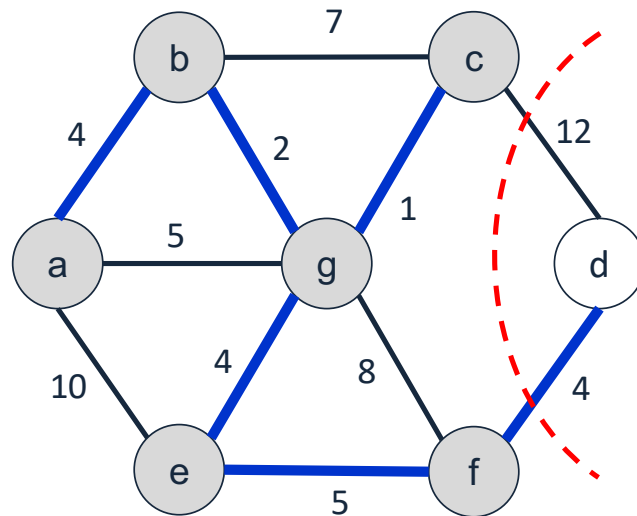


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (e, f) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim



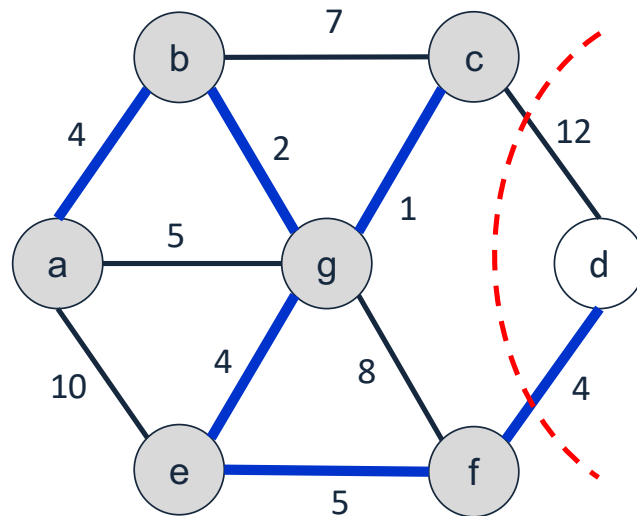
Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (f, d) .

Diese Kante wird in A aufgenommen.

Nach diesen $|V| - 1$ Schritten terminiert der Algorithmus.

Algorithmus von Prim



Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (f, d) .

Diese Kante wird in A aufgenommen.

Nach diesen $|V| - 1$ Schritten terminiert der Algorithmus.

Algorithmus von Prim

Pseudocode

Die Menge $P = V - K$ wird nicht explizit geführt, nur K .

Ausgewählt wird der Knoten v , der aus P mit minimalen Kosten erreicht werden kann.

Anschließend werden diese Kosten bei den in K verbleibenden Knoten aktualisiert.

Am Ende besitzen alle Knoten (bis auf die Wurzel) ihren Elternknoten im MST, woraus sich die Menge A ergibt.

MST-PRIM(V, E, s)

```

1: queue  $\leftarrow$  new PriorityQueue           ▷ Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$                  ▷ Abstand des Knotens  $v$  von  $P$ 
4:    $v.parent \leftarrow \mathbf{NIL}$ 
5:   queue.insert( $v$ )
6: end for
7:  $s.dist \leftarrow 0$                        ▷ Startknoten  $s$  ist in  $P$ 
8: queue.update( $s$ )
9: while queue not empty do
10:   $v \leftarrow$  queue.extract_min()          ▷ Basierend auf  $v.dist$ 
11:  for each  $e : v \rightarrow u \in E$  do      ▷ Alle von  $v$  ausgehenden Kanten
12:    if  $u \in queue \wedge u.dist > w(e)$  then
13:       $u.dist \leftarrow w(e)$ 
14:       $u.parent \leftarrow v$ 
15:      queue.update( $u$ )
16:    end if
17:  end for
18: end while

```

Algorithmus von Prim

Analyse

Die **while**-Schleife wird $|V|$ -mal durchlaufen.

Bei jedem Durchlauf muss der minimal entfernte Knoten ermittelt werden, was mit einem Min-Heap in $O(\log|V|)$ gelingen kann.

Somit ergibt sich eine Komplexität von $O(|V| \cdot \log|V|)$

Jede Kante wird anschließend einmal betrachtet, somit insgesamt

$$O(|V| \cdot \log|V| + |E|)$$

MST-PRIM(V, E, s)

```

1: queue  $\leftarrow$  new PriorityQueue           ▷ Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$                    ▷ Abstand des Knotens  $v$  von  $P$ 
4:    $v.parent \leftarrow \mathbf{NIL}$ 
5:   queue.insert( $v$ )
6: end for
7:  $s.dist \leftarrow 0$                          ▷ Startknoten  $s$  ist in  $P$ 
8: queue.update( $s$ )
9: while queue not empty do
10:   $v \leftarrow queue.extract\_min()$           ▷ Basierend auf  $v.dist$ 
11:  for each  $e : v \rightarrow u \in E$  do       ▷ Alle von  $v$  ausgehenden Kanten
12:    if  $u \in queue \wedge u.dist > w(e)$  then
13:       $u.dist \leftarrow w(e)$ 
14:       $u.parent \leftarrow v$ 
15:      queue.update( $u$ )
16:    end if
17:  end for
18: end while

```

Union - Find

Union - Find

Zielsetzung

Für den folgenden Algorithmus benötigen wir eine Möglichkeit,

1. viele einzelne Objekte zunächst als einelementige Mengen zu verstehen (Operation: MAKE-SET),
2. anschließend die Mengen zu vereinigen (Operation: UNION) und
3. schließlich feststellen zu können, ob zwei Objekte Elemente der gleichen Menge sind.

Für die letzte Anforderung definieren wir je Menge ein kanonisches Element (Musterelement), das wir von jedem Element der Menge aus ermitteln können (Operation: FIND-SET). Zwei Objekte sind dann in der gleichen Menge, wenn sie dasselbe kanonische Element besitzen.

Union - Find

Pseudocode

MAKE-SET(v)

1: $\text{parent}[v] \leftarrow v$

FIND-SET(v)

1: **if** $\text{parent}[v] = v$ **then**
2: **return** v
3: **else**
4: **return** **FIND-SET**($\text{parent}[v]$)
5: **end if**

UNION(u, v)

1: $r_u \leftarrow \text{FIND-SET}(u)$
2: $r_v \leftarrow \text{FIND-SET}(v)$
3: $\text{parent}[r_v] \leftarrow r_u$

Algorithmus von Kruskal

Algorithmus von Kruskal

Grundidee

Der MST wird iterativ mit einem Greedy-Ansatz gefunden.

Man beginnt wieder mit $A = \emptyset$ und erweitert A sukzessive durch Auswahl der Kante $e \in (E - A)$ mit dem geringsten Gewicht, die mit den Kanten aus A keinen Zyklus bildet.

Temporär können demnach getrennte Teilbäume entstehen. Am Ende ergibt sich jedoch ein zusammenhängender Baum, weil der Ausgangsgraph zusammenhängend war, also eine verbindende Kante zwischen Teilbäumen existiert, deren Wahl zu keinem Zyklus führt.

Algorithmus von Kruskal

Korrektheitsüberlegung

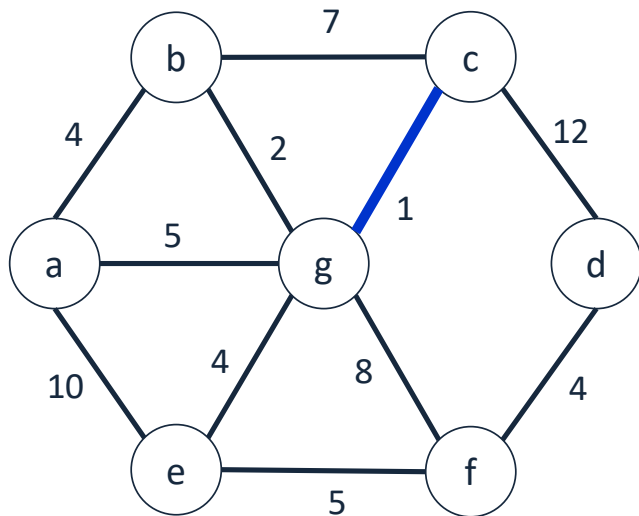
Ist ein so konstruierter $G = (V, A)$ wirklich ein MST?

Beweis durch Induktion, dass während der Konstruktion stets gilt: $A \subseteq E_B$ wobei $G_B = (V, E_B)$ minimaler aufspannender Baum.

Für $A = \emptyset$ ist die Aussage trivial.

Sei e eine nach Vorschrift gewählte Kante. Dann ist auch $A \cup \{e\} \subseteq E_B$, denn andernfalls besäße $E_B \cup \{e\}$ einen Zyklus mit einer Kante f . Entfernt man f aus G_B und fügt stattdessen e ein, so erhält man wieder einen MST. Somit ist e eine sichere Kante.

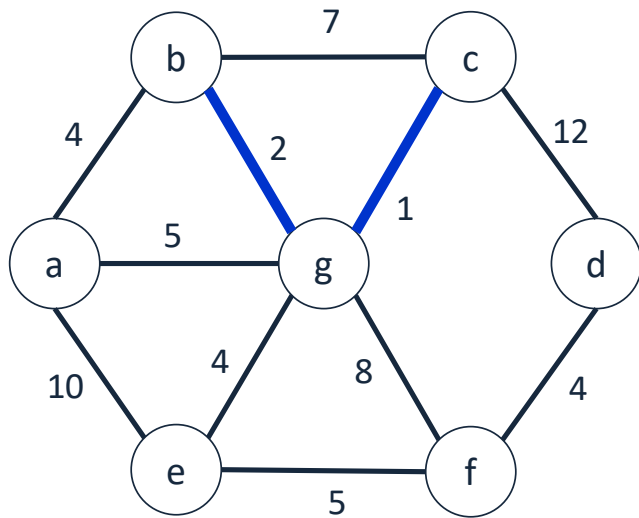
Algorithmus von Kruskal



Start

Im ersten Schritt wird die Kante mit dem geringsten Gewicht ausgewählt und zu A hinzugefügt.

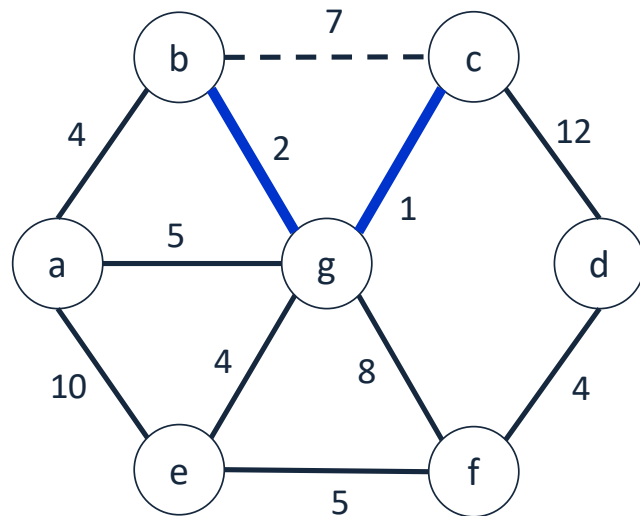
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

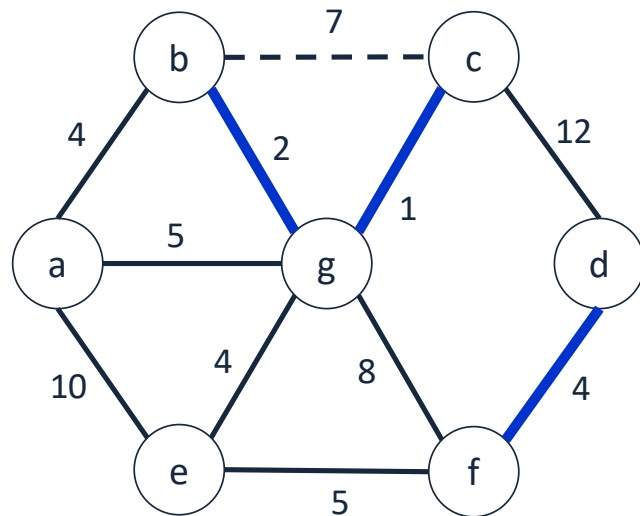
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

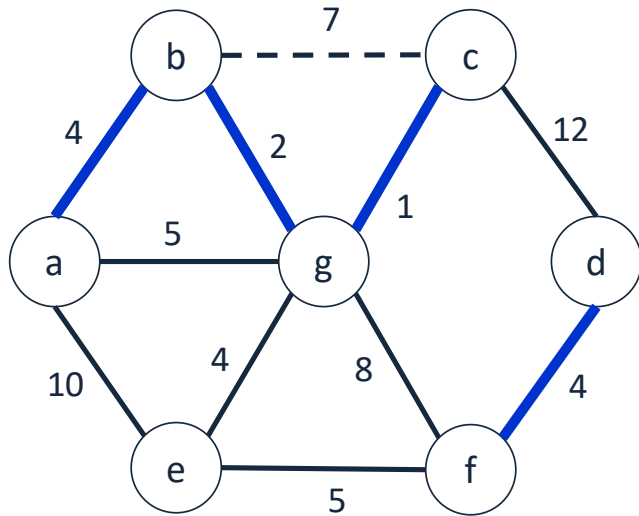
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

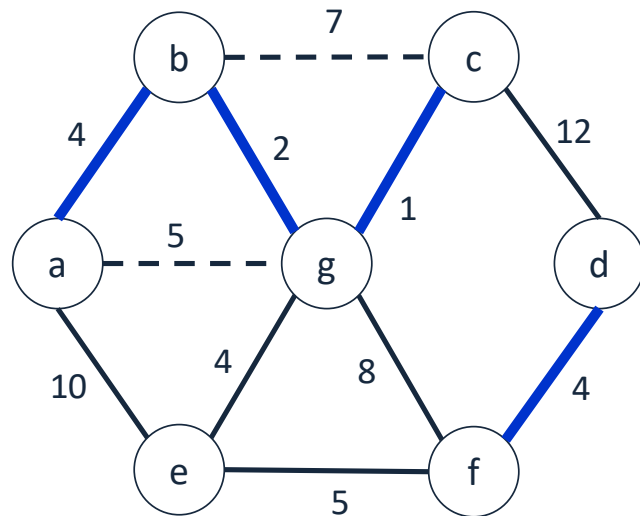
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

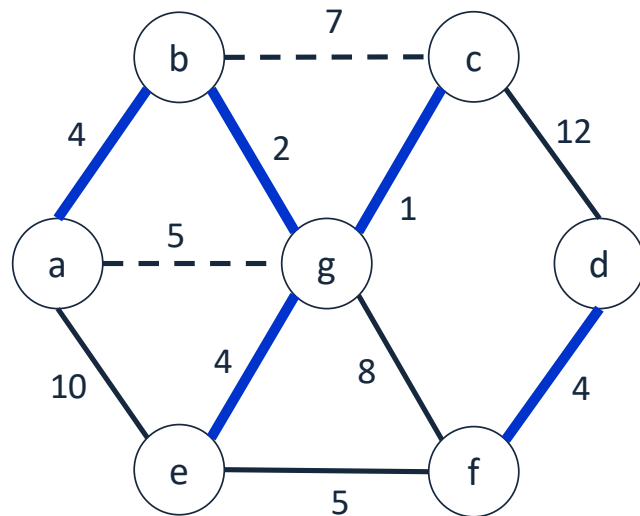
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

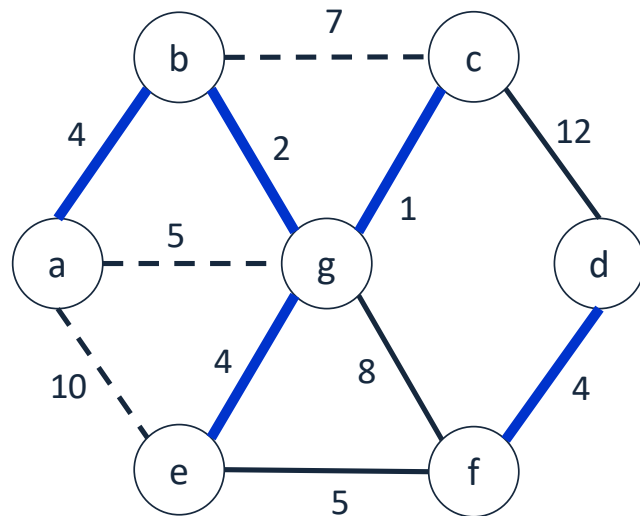
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

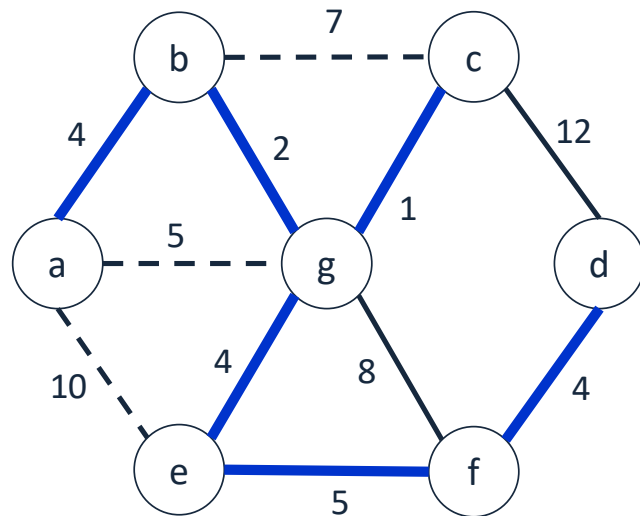
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

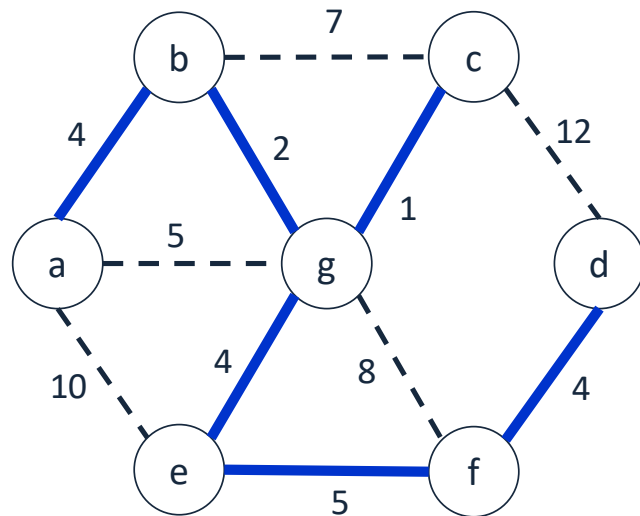
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

Algorithmus von Kruskal



Finden der nächsten Kante

Kann keine weitere Kante aufgenommen werden, ohne einen Zyklus zu erzeugen, terminiert der Algorithmus.

Algorithmus von Kruskal

Pseudocode

Die Laufzeit wird dominiert durch das Sortieren in Zeile 5, das $O(|E| \log|E|)$ benötigt.

Bei effizienter Implementierung der Mengenoperationen erhält man insgesamt $O(|E| \log|V|)$

MST-KRUSKAL(V, E)

```

1:  $A \leftarrow \emptyset$ 
2: for each  $v \in V$  do
3:   MAKE-SET( $v$ )
4: end for
5: SORT( $E$ )                                     ▷ Basierend auf  $w$ 
6: for each  $e : v \rightarrow u \in E$  do             ▷ Alle Kanten
7:   if FIND-SET( $v$ )  $\neq$  FIND-SET( $u$ ) then
8:      $A \leftarrow A \cup \{e\}$ 
9:     UNION( $v, u$ )
10:  end if
11: end for

```

Ziele der heutigen Lerneinheit

Sie verstehen...

- ... den Begriff der Berechenbarkeit und die Grenzen algorithmischer Lösbarkeit.
- ... die Bedeutung von Komplexitätsklassen wie P und NP.
- ... die Relevanz NP-vollständiger Probleme für die Informatik.

Sie können...

- ... erklären, warum RAM und Turing-Maschine gleich mächtig sind.
- ... berechenbare von nicht berechenbaren Problemen unterscheiden.
- ... das Halteproblem als Beispiel eines unentscheidbaren Problems erläutern.

Berechnungsmodelle

Berechnungsmodelle

Random Access Machine (RAM)



Bei den Komplexitätsbetrachtungen wird ein einfaches, aber utopisches Rechnermodell zugrunde gelegt: die ***Random Access Machine (RAM)***:

- Unendlich viele Speicherzellen, die in konstanter Zeit gelesen oder beschrieben werden
- Überschaubarer Befehlssatz (arithm. Operationen, Kontrollstrukturen, Speicherzugriff)

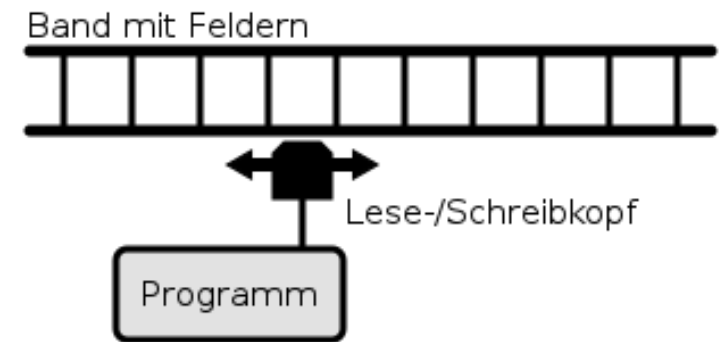
Berechnungsmodelle

Deterministische Turing-Maschine (TM)

Eine Turingmaschine ist ein **alternatives Berechnungsmodell**.

Sie besteht aus:

- Einem unendlich langen Band, auf dem Daten gespeichert werden können.
- Einem Schreib-Lese-Kopf, der sich in beiden Richtungen über das Band bewegen kann und jeweils ein Zeichen lesen und schreiben kann.
- Einem endl. Zustandsautomaten, dessen Übergänge vom aktuell gelesenen Zeichen determiniert sind („Programm“). Bei einem Übergang kann ein Zeichen geschrieben und der Schreib-Lese-Kopf um eine Position nach links oder rechts bewegt werden.



Der Input ist die initiale Beschriftung des Bands, der Output ist die Beschriftung des Bands, wenn ein Endzustand erreicht wird.

Berechnungsmodelle

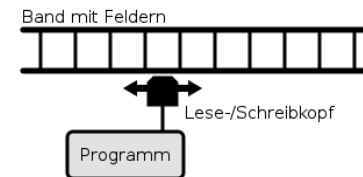
Gleichwertigkeit von RAM und TM

Es wurde gezeigt, dass RAM und TM gleich mächtig sind, d.h. alles, was mit einer RAM berechnet werden kann, kann auch von einer TM berechnet werden und umgekehrt.



RAM

Können sich
wechselseitig
simulieren



TM

Berechnungsmodelle

Turing Complete

Ein Berechnungsmodell, das genauso mächtig ist wie eine TM, wird als ***Turing-vollständiges System*** oder auch als ***Turing Complete*** bezeichnet.

Eine RAM ist also ***Turing Complete***.

Weitere Beispiele:

- TeX und damit auch LaTeX sind turing-vollständig. Das hat schon der Erfinder Donald Knuth bewiesen.
- Das "Game of Life" von John Conway ist turing-vollständig. Als Input dient die Startpopulation.
- Das Spiel „Minecraft“ ist turing-vollständig.

Berechenbarkeit

Berechenbarkeit

Grenzen

Gibt es Probleme, die sich durch die gezeigten Berechnungsmodelle, die ja über endliche Computer hinausgehen, nicht berechnet werden können?

Ja! Denn es gibt schon mehr Funktionen $f: \mathbb{Z} \rightarrow \mathbb{Z}$ als es Programme für eine Turing Maschine geben kann. D.h. es gibt Funktionen, für die es kein Programm gibt.

Beweis:

Wir zeigen: es gibt **überabzählbar unendlich** viele Funktionen $f: \mathbb{Z} \rightarrow \mathbb{Z}$,
aber nur **abzählbar unendlich** viele TM-Programme

Berechenbarkeit

Grenzen - Beweis

Es gibt überabzählbar unendlich viele Funktionen $f: \mathbb{Z} \rightarrow \mathbb{Z}$

Widerspruchsbeweis:

Sollte es nur abzählbar unendlich viele solche Funktionen geben, dann könnte man deren Wertetabellen in eine unendliche Liste untereinander schreiben.

	1	2	3	4	5
f_1	4+1	2	3	7	...
f_2	1	9+1	3	1	...
f_3	7	3	3+1	6	...
f_4	6	2	7	1+1	...
f_5	2	8	5	5	...
...					

Dann ist aber die Funktion, die sich durch Addition von 1 zu jedem Element der Diagonale ergibt, nicht enthalten...

Berechenbarkeit

Grenzen - Beweis

Es gibt nur abzählbar unendlich viele TM-Programme

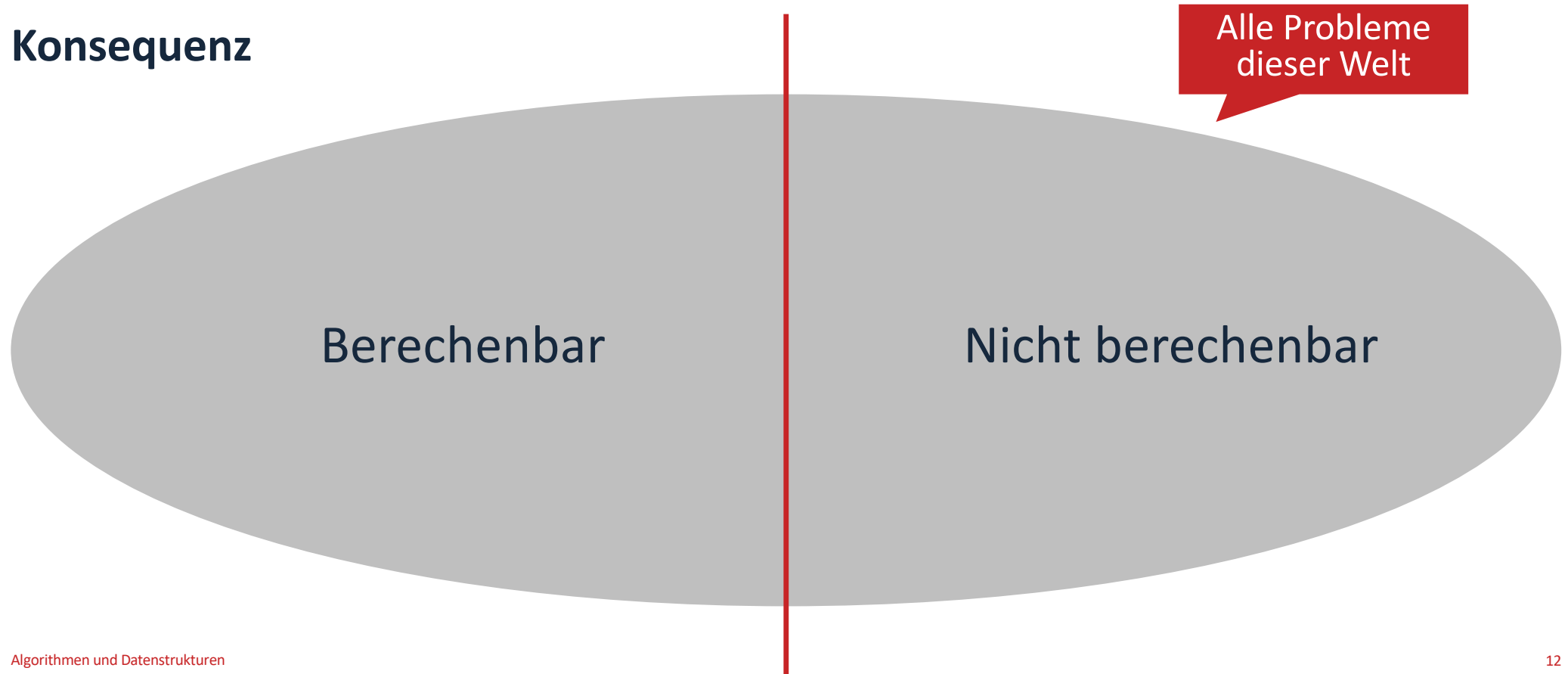
Beweis:

Betrachtet wird das gleich mächtige RAM-Berechnungsmodell. Alle Programme bestehen aus einem endlichen Satz von Befehlen, die man zwar zu unendlich vielen Programmen zusammensetzen kann. Diese können aber problemlos in eine unendliche Liste überführt werden, indem man alle Programme als String betrachtet und alphabetisch sortiert.

→ Die Anzahl der Programme ist abzählbar unendlich.

Berechenbarkeit

Konsequenz



Berechenbarkeit

Beispiel für eine nicht berechenbare Funktion

Behauptung

Sei X ein Programm und y ein Input, dann ist die Funktion

$$H(X, y) = \begin{cases} 0 & \text{falls } X(y) \text{ terminiert} \\ 1 & \text{sonst} \end{cases}$$

nicht berechenbar.

Berechenbarkeit

Beispiel für eine nicht berechenbare Funktion

Widerspruchsbeweis

Wenn so eine Funktion $H(X, y)$ berechnet werden könnte, dann könnte man sie in folgenden Programm Z verwenden:

 $Z(y)$

1: **repeat**

2: **until** $H(Z, y) = 1$

Falls Z beim Input y terminiert, liefert $H(X, y)$ den Wert 0 und die Schleife wird wiederholt, d.h. Z terminiert nicht.

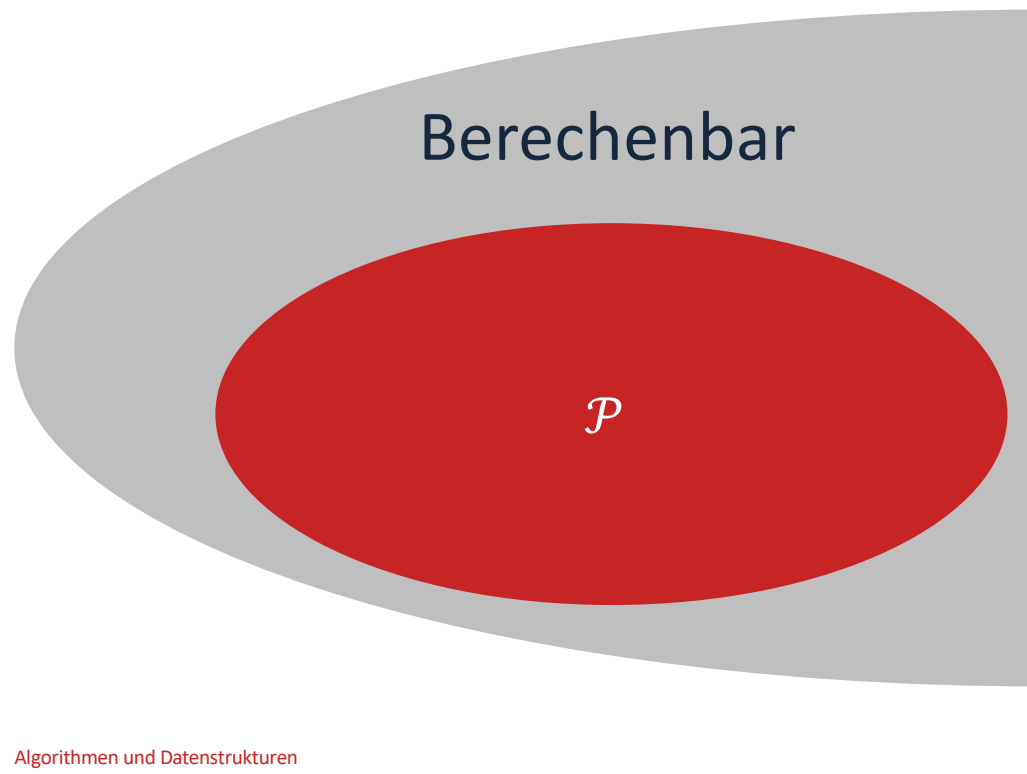
Falls Z aber beim Input y nicht terminiert, liefert $H(X, y)$ den Wert 1 und das Programm endet.

→ Das Halteproblem ist nicht entscheidbar,
 H nicht berechenbar

Komplexitätsklassen

Komplexitätsklassen

Effizient lösbare Probleme



Gegenstand der Vorlesung waren Fragestellungen, für die es Algorithmen mit der Zeitkomplexität

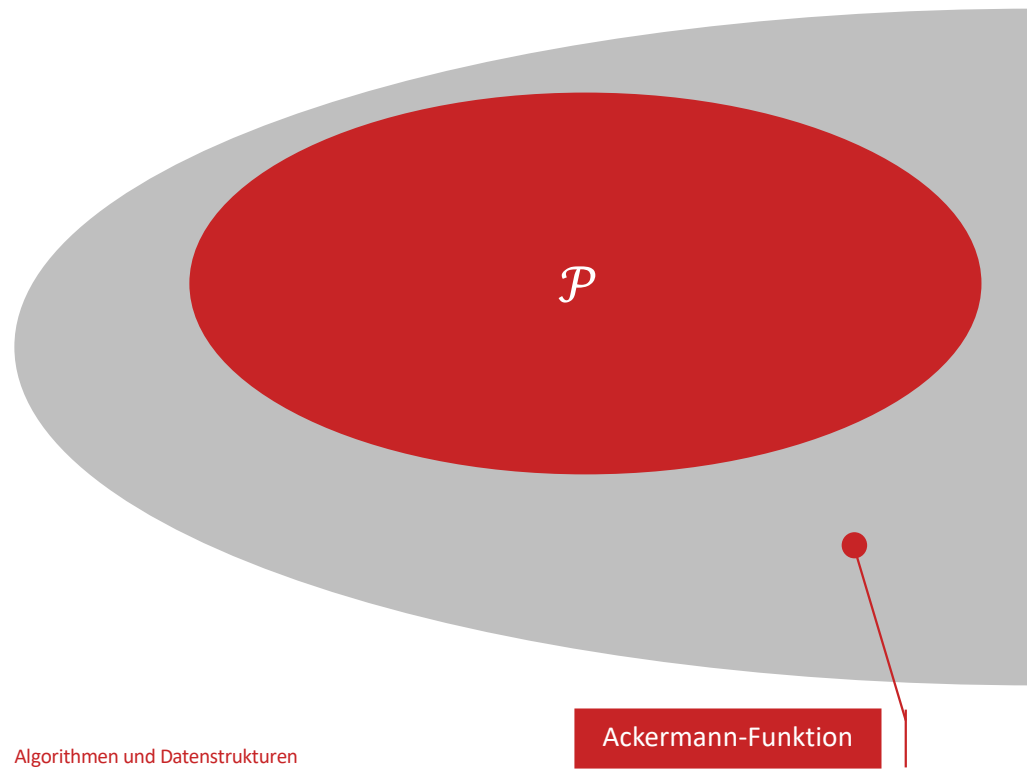
$$O(n^i) \text{ mit } i \in \mathbb{N}_0$$

gibt.

Diese Fragestellungen gehören zu $\mathcal{P}$, der Klasse der in polynomialer Zeit von einer TM lösbaren Probleme. Solche Fragestellungen werden auch als effizient lösbar angesehen.

Komplexitätsklassen

Ackermann-Funktion



Definition:

$$\begin{aligned} A(0, n) &= n + 1 \\ A(m, 0) &= A(m - 1, 1) \\ A(m, n) &= A(m - 1, A(m, n - 1)) \end{aligned}$$

Selbst für kleine Werte von m und n kann die Berechnung von $A(m, n)$ eine erhebliche Anzahl von Rechenschritten erfordern, die exponentiell in Bezug auf m und n wachsen.

Komplexitätsklassen

Nicht-Deterministische Turing-Maschine (NTM)

Eine Nicht-Deterministische Turingmaschine ist eine Erweiterung der Turingmaschine. In einer NTM

- kann es für einen Zustand mehrere Übergänge bei gleichem gelesenen Zeichen geben
- werden alle diese Übergänge gleichberechtigt parallel ausgeführt
- ist es ausreichend, wenn einer der parallelen Pfade einen Endzustand erreicht



Die Grenze der Berechenbarkeit wird durch NTM nicht verschoben.

Aber: durch die Parallelität kann sich die Zeitkomplexität verändern: Es können u.U. Probleme mit NTM in polynomialer Zeit gelöst werden, die nicht in $\mathcal{P}$ waren.

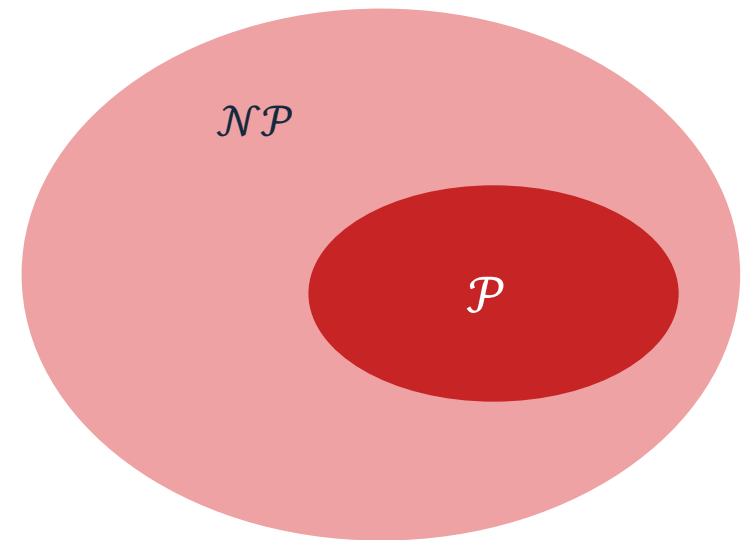
Komplexitätsklassen

P = NP ?

Alle von einer NTM in polynomialer Zeit lösbaren Probleme werden auch als $\mathcal{NP}$ bezeichnet.

Es ist klar, dass $\mathcal{P} \subseteq \mathcal{NP}$.

Erstaunlicherweise hat man aber noch nicht zeigen können, dass es Probleme gibt, die in $\mathcal{NP}$ enthalten sind, nicht aber in $\mathcal{P}$. Es könnte also $\mathcal{P} = \mathcal{NP}$ sein.



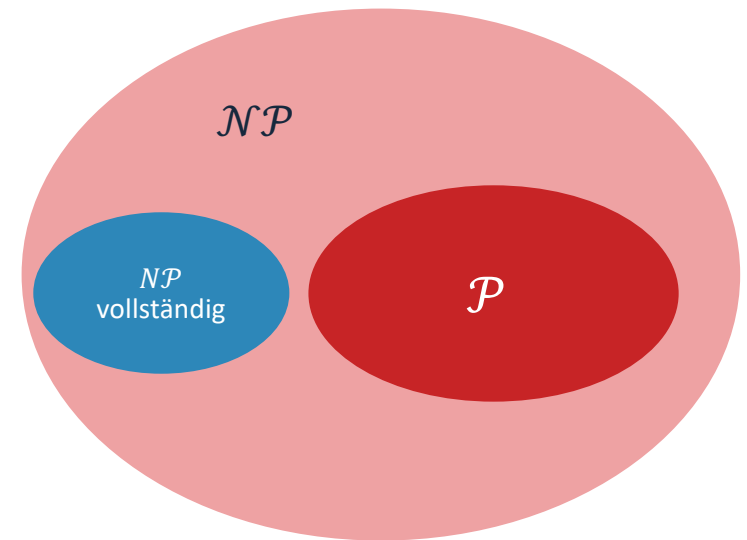
Komplexitätsklassen

NP-vollständige Probleme

Man kann zeigen, dass es in NP einige Probleme gibt, auf die sich alle anderen Probleme in NP in polynomialer Zeit reduzieren lassen.

Diese Probleme stellen sozusagen die schwierigsten Probleme in NP dar und heißen **NP vollständig**.

Wenn für eines dieser NP-vollständigen Probleme ein Algorithmus mit polynomialer Laufzeit gefunden werden könnte, wäre $NP=P$ bewiesen.



Komplexitätsklassen

NP-vollständige Probleme

Beispiele:

Traveling Salesman Problem

Was ist die kostengünstigste Rundreise durch n Städte?

Erfüllbarkeitsproblem

Gibt es eine Belegung der n Variablen einer booleschen Formel, die den Ausdruck wahr macht?

Komplexitätsklassen

Offene Frage

In der theoretischen Informatik ist bis heute unklar:

- Gilt $P = NP$?
- Und damit: existieren effiziente Algorithmen für NP-vollständige Probleme?

" $P = NP$?" ist eines der sechs noch ungelösten Millennium-Probleme und für seine Lösung ist ein Preis von 1 Mio. US-Dollar ausgesetzt."